

Mujhe yah batao Dekho mere ko DSA ke bhi kuchh questions karne hain jo ki tumse interview mein puche jaate Hain editor per to sabse pahle to mujhe batao mujhe UN question ko prepare kaise karna hai kaise matlab kyunki Maine Kai bar dekha Hai ki Ham code likhate Hain FIR bad mein mere ko bol diya jata hai ki is code ko optimise Karo ya is code mein to building method mat use karo normal bnao ya building ka istemal karo is type se preparation Karun ki mujhe chahe Koi bhi question de diya jaaye main easily kar paaun

Bilkul. Agar tum DSA interview ke liye prepare kar rahi ho, to sirf 50–100 questions ke solutions ratna best approach nahi hai. Tumhe ek hi question ko **multiple constraints/methods se solve** karna seekhna hai.

Tumhare case mein main preparation ko is tarah rakhunga:

1. Har question ke 4 versions karo

Maan lo question hai: **Find duplicate elements in an array**

Usko sirf ek code se khatam mat karo:

Level 1 — Brute Force

- Nested loops
- Simple logic
- Time: $O(n^2)$

Level 2 — Better

- Set / Map
- Time: $O(n)$
- Space: $O(n)$

Level 3 — Without built-in methods

- Khud logic implement karo
- Interviewer agar bole Set/Map mat use karo, tab bhi solve kar pao.

Level 4 — Optimisation

- Dekho kya time/space aur improve ho sakta hai.
- Interviewer pooche: **“Can you optimise this?”** → tum immediately alternatives bata pao.

2. Har question ke saath ye 7 cheezein prepare karo

Har DSA question ke liye tumhari notebook/code file mein:

1. **Problem kya hai?**
2. **Brute-force approach**
3. **Optimal approach**
4. **Time complexity**
5. **Space complexity**
6. **Built-in methods ke bina solution**
7. **Interviewer ke follow-up questions**

Example:

“Array mein second largest element find karo.”

Tumhe pata hona chahiye:

- Sorting se kaise karoge?
- Sorting ke bina kaise karoge?
- One-pass mein kaise karoge?
- sort() use kiye bina?
- Duplicate values ka kya?
- Agar array mein sirf 1 element ho?
- Negative numbers?
- Time complexity?
- Space complexity?

Yahi actual interview preparation hai.

3. Built-in methods ko completely avoid bhi mat karo

Interview mein dono tarah ke questions aa sakte hain.

For example JavaScript:

```
arr.reverse()
```

Ye jaana useful hai.

Lekin agar interviewer bole:

“Don’t use reverse(). Implement it yourself.”

To tum kar pao:

```
let left = 0;
let right = arr.length - 1;

while (left < right) {
  [arr[left], arr[right]] = [arr[right], arr[left]];
  left++;
  right--;
}
```

Isliye tumhe **built-in + manual implementation dono** aane chahiye.

4. Pattern identify karna seekho — ye sabse important hai

Random questions karne ke bajay question dekhte hi identify karo:

Pattern	Typical Questions
Array	Largest, smallest, duplicate, missing
String	Palindrome, anagram, frequency
Hashing	Two Sum, frequency, duplicates
Two Pointer	Pair sum, reverse, palindrome
Sliding Window	Subarray/substring problems
Sorting	Merge, sort-based problems
Binary Search	Search in sorted array
Recursion	Factorial, Fibonacci, subsets
Stack	Balanced brackets, next greater
Queue	Basic queue problems
Linked List	Reverse, middle, cycle
Trees	Traversals, height, search
Graph	BFS, DFS

Pattern	Typical Questions
DP	Climbing stairs, Fibonacci variations

Goal ye hona chahiye ki question dekhte hi tum bolo:

“Achha, ye Two Pointer wala pattern hai.”

Ya:

“Ye HashMap/Hashing se efficiently ho jayega.”

Tab tum unfamiliar question bhi solve kar paogi.

5. Main tumhare saath DSA exactly interview style mein karwa sakta hoon

Hum har question ko is format mein karenge:

Question → Tumhara approach → Brute Force → Optimised → Without built-in → Complexity → Edge cases → Interview follow-ups

Aur ek aur important rule:

✗ Main tumhe sirf solution dekar next question par nahi jaunga.

Pehle tum **khud attempt** karogi.

Phir agar code galat hua:

“Is line mein problem hai.”

Phir tum fix karogi.

Uske baad main interviewer ki tarah poochunga:

“Can you optimise it?”

Phir:

“Can you do it without using Set?”

Phir:

“What’s the time complexity?”

Phir:

“What if the array contains duplicates?”

Isse tum **question ka solution yaad karne ke bajay problem-solving seekhogi.**

6. Tumhare liye best preparation structure

Tum JavaScript mein DSA kar rahi ho, to main recommend karunga:

Phase 1 — Arrays

→ basic → frequency → duplicates → sorting → two pointer → sliding window

Phase 2 — Strings

→ palindrome → frequency → anagram → substring → character manipulation

Phase 3 — Hashing

→ Map/Set based problems

Phase 4 — Searching & Sorting

Phase 5 — Recursion

Phase 6 — Linked List

Phase 7 — Stack & Queue

Phase 8 — Trees

Phase 9 — Graph basics

Phase 10 — Basic DP

Aur har phase mein **easy → medium → interview-level** questions.

Sabse important baat

Tumhara target ye nahi hona chahiye:

“Mujhe 100 questions ke code yaad hain.”

Tumhara target hona chahiye:

“Mujhe question ka pattern samajh aa gaya, main brute force bana sakti hoon, usko optimise kar sakti hoon, built-in ke bina implement kar sakti hoon aur complexity explain kar sakti hoon.”

Agar tum is tarah prepare karogi, to interviewer exact wahi question na bhi de, tab bhi tumhare paas approach nikalne ka framework hoga.

Han bilkul main yahi bol rahi hun to ab Tum ek kam karo jitne bhi most important interview level question Hai har topic Hai kuchh bhi Hai har topic wise mujhe sab ke question aur solution do arre Se shuru karte hain jaldi batao are ke kitne questions honge

Haan, exactly isi approach se karenge. **Random 100–200 questions nahi** — sirf woh questions jo placement/fresher interviews mein repeatedly useful patterns cover karte hain.

Agar **Arrays se start** karna hai, to main Arrays ko roughly **30–35 interview-level questions** mein complete karwaunga.

ARRAY — Complete Interview Set

Level 1: Basic logic

1. Find largest element
2. Find smallest element
3. Find second largest element
4. Find second smallest element
5. Reverse an array
6. Check if array is sorted
7. Remove duplicates from sorted array
8. Count frequency of elements

Level 2: Important interview questions

9. Find missing number
10. Find duplicate number
11. Find all duplicates
12. Find intersection of two arrays
13. Find union of two arrays
14. Move all zeros to end
15. Move negative numbers to one side
16. Find pair with given sum
17. Find elements occurring more than once

18. Find majority element

Level 3: Optimisation / patterns

19. Two Sum

20. Three Sum

21. Maximum subarray sum — Kadane's Algorithm

22. Best time to buy and sell stock

23. Rearrange positive and negative numbers

24. Rotate array left

25. Rotate array right

26. Find equilibrium index

27. Find subarray with given sum

28. Longest subarray with given sum

Level 4: Strong interview questions

29. Maximum product subarray

30. Merge two sorted arrays

31. Merge overlapping intervals

32. Find leaders in an array

33. Trapping Rain Water

34. Product of array except self

35. Maximum consecutive ones

 **Aur har question ko hum 4 ways se karenge**

For example:

Question: Move zeros to end

-  Brute force
-  Optimal
-  Without built-in methods
-  Interviewer: "Can you do it in $O(n)$ and $O(1)$ space?"

Aur har question ke baad: **Time Complexity + Space Complexity + Edge Cases + Follow-up questions**

Ek important distinction

Arrays ke **35 questions** ka matlab 35 concepts nahi hain. Inke andar important patterns cover honge:

Brute Force → Hashing → Two Pointer → Sliding Window → Sorting → Prefix Sum → Kadane → Greedy

Isliye ye 35 properly kar liye to tumhare paas **array ke bahut saare unseen questions ko attack karne ki ability** aa jayegi.

Arrays = ~35 questions.

Uske baad Strings ~25, Searching/Sorting ~20, Recursion ~15, Linked List ~20, Stack/Queue ~20, Trees ~20, Graph ~15, DP ~15 — aur hum unnecessary questions hata kar **placement-focused master set** bana sakte hain.

Ab Arrays Question 1 se start karte hain — aur main tumhe solution ratwaunga nahi, interview mein sochne ka process bhi karwaunga.

Okay buddy let's start level 1 Sara level 1 ek hi bar mein de do han aur sari solutions Dena matlab jaise ki Ham ek question ko building se bhi kar sakta hai bina building ke bhi kar sakte hain optimization se bhi kar sakte hain to use type se sare solutions Jo ek interview aur mang sakta hai and han sare solutions ki space complicitor time complexity batana building method wale mein bhi aur without building bhi aur acche se pura solution dikhana naki sirf logic aur jyada hoch poch krne ki jarurat nhi h tum mujhe ekdumneat and clean chize do like agar interview bole make this with built then code tagar interview me bole ki make this without built-in then code aur agar bole optimize it then code and saare solution ki time and space complexity with sinhinglish and don't explain tha code jb tk mai na bolu

Bilkul buddy. **Level 1 ke saare 8 Array questions** ek saath.

Har question mein **Built-in / Without Built-in / Optimized** jo practically relevant hai, woh code dunga. **Code explain nahi karunga** jab tak tum na bolo.

Note: JavaScript mein Math.max() / Math.min() ko built-in solution maana hai. sort() bhi built-in hai.

LEVEL 1 — ARRAYS

1. Find Largest Element

Approach 1 — Built-in

```
function findLargest(arr) {  
  return Math.max(...arr);  
}
```

```
console.log(findLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(n)$ due to spread

Approach 2 — Without Built-in

```
function findLargest(arr) {  
  let largest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] > largest) {  
      largest = arr[i];  
    }  
  }  
  
  return largest;  
}
```

```
console.log(findLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

★ Optimized

Same as without built-in:

```
function findLargest(arr) {  
  let largest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    largest = Math.max(largest, arr[i]);  
  }  
  
  return largest;  
}
```

```
console.log(findLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

2. Find Smallest Element

Approach 1 — Built-in

```
function findSmallest(arr) {  
  return Math.min(...arr);  
}
```

```
console.log(findSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(n)$

Approach 2 — Without Built-in

```
function findSmallest(arr) {  
  let smallest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] < smallest) {  
      smallest = arr[i];  
    }  
  }  
  
  return smallest;  
}
```

```
}
```

```
console.log(findSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

★ Optimized

```
function findSmallest(arr) {  
  let smallest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    smallest = Math.min(smallest, arr[i]);  
  }  
  
  return smallest;  
}
```

```
console.log(findSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

3. Find Second Largest Element

Approach 1 — Built-in

```
function secondLargest(arr) {  
  let unique = [...new Set(arr)];  
  
  unique.sort((a, b) => b - a);  
  
  return unique[1];  
}
```

```
console.log(secondLargest([10, 25, 7, 40, 40, 15]));
```

Time: $O(n \log n)$

Space: $O(n)$

Approach 2 — Without Built-in

```
function secondLargest(arr) {  
  let largest = -Infinity;  
  let secondLargest = -Infinity;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] > largest) {  
      secondLargest = largest;  
      largest = arr[i];  
    } else if (arr[i] > secondLargest && arr[i] !== largest) {  
      secondLargest = arr[i];  
    }  
  }  
  
  return secondLargest;  
}
```

```
console.log(secondLargest([10, 25, 7, 40, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

★ Optimized — One Pass

```
function secondLargest(arr) {  
  let largest = -Infinity;  
  let secondLargest = -Infinity;  
  
  for (let num of arr) {  
    if (num > largest) {  
      secondLargest = largest;  
      largest = num;  
    } else if (num > secondLargest && num !== largest) {  
      secondLargest = num;  
    }  
  }  
  
  return secondLargest;  
}
```



```
console.log(secondLargest([10, 25, 7, 40, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

4. Find Second Smallest Element

Approach 1 — Built-in

```
function secondSmallest(arr) {  
  let unique = [...new Set(arr)];  
  
  unique.sort((a, b) => a - b);  
  
  return unique[1];  
}
```

```
console.log(secondSmallest([10, 25, 7, 40, 7, 15]));
```

Time: $O(n \log n)$

Space: $O(n)$

Approach 2 — Without Built-in

```
function secondSmallest(arr) {  
  let smallest = Infinity;  
  let secondSmallest = Infinity;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] < smallest) {  
      secondSmallest = smallest;  
      smallest = arr[i];  
    } else if (arr[i] < secondSmallest && arr[i] !== smallest) {  
      secondSmallest = arr[i];  
    }  
  }  
  
  return secondSmallest;  
}
```

```
console.log(secondSmallest([10, 25, 7, 40, 7, 15]));
```

Time: $O(n)$

Space: $O(1)$

★ Optimized — One Pass

```
function secondSmallest(arr) {  
  let smallest = Infinity;  
  let secondSmallest = Infinity;  
  
  for (let num of arr) {  
    if (num < smallest) {  
      secondSmallest = smallest;  
      smallest = num;  
    } else if (num < secondSmallest && num !== smallest) {  
      secondSmallest = num;  
    }  
  }  
  
  return secondSmallest;  
}
```

```
console.log(secondSmallest([10, 25, 7, 40, 7, 15]));
```

Time: $O(n)$

Space: $O(1)$

5. Reverse an Array

Approach 1 — Built-in

```
function reverseArray(arr) {  
  return arr.reverse();  
}
```

```
console.log(reverseArray([1, 2, 3, 4, 5]));
```

Time: $O(n)$
Space: $O(1)$

Approach 2 — Without Built-in

```
function reverseArray(arr) {  
  let reversed = [];  
  
  for (let i = arr.length - 1; i >= 0; i--) {  
    reversed.push(arr[i]);  
  }  
  
  return reversed;  
}  
  
console.log(reverseArray([1, 2, 3, 4, 5]));
```

Time: $O(n)$
Space: $O(n)$

★ Optimized — Two Pointer

```
function reverseArray(arr) {  
  let left = 0;  
  let right = arr.length - 1;  
  
  while (left < right) {  
    let temp = arr[left];  
    arr[left] = arr[right];  
    arr[right] = temp;  
  
    left++;  
    right--;  
  }  
  
  return arr;  
}  
  
console.log(reverseArray([1, 2, 3, 4, 5]));
```

Time: $O(n)$
Space: $O(1)$

6. Check if Array is Sorted

Approach 1 — Built-in

```
function isSorted(arr) {  
  let sorted = [...arr].sort((a, b) => a - b);  
  
  return JSON.stringify(arr) === JSON.stringify(sorted);  
}  
  
console.log(isSorted([1, 2, 3, 4, 5]));
```

Time: $O(n \log n)$
Space: $O(n)$

Approach 2 — Without Built-in

```
function isSorted(arr) {  
  for (let i = 0; i < arr.length - 1; i++) {  
    if (arr[i] > arr[i + 1]) {  
      return false;  
    }  
  }  
  
  return true;  
}  
  
console.log(isSorted([1, 2, 3, 4, 5]));
```

Time: $O(n)$
Space: $O(1)$

★ Optimized

```
function isSorted(arr) {  
  for (let i = 1; i < arr.length; i++) {
```

```
    if (arr[i] < arr[i - 1]) {  
        return false;  
    }  
}  
  
return true;  
}  
  
console.log(isSorted([1, 2, 3, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

7. Remove Duplicates from Sorted Array

Approach 1 — Built-in

```
function removeDuplicates(arr) {  
    return [...new Set(arr)];  
}  
  
console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
```

Time: $O(n)$

Space: $O(n)$

Approach 2 — Without Built-in

```
function removeDuplicates(arr) {  
    let result = [];  
  
    for (let i = 0; i < arr.length; i++) {  
        if (i === 0 || arr[i] !== arr[i - 1]) {  
            result.push(arr[i]);  
        }  
    }  
  
    return result;  
}
```

```
console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
```

Time: $O(n)$

Space: $O(n)$

★ Optimized — Two Pointer / In-place

```
function removeDuplicates(arr) {  
  if (arr.length === 0) {  
    return 0;  
  }  
  
  let index = 1;  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] !== arr[i - 1]) {  
      arr[index] = arr[i];  
      index++;  
    }  
  }  
  
  return index;  
}  
  
let arr = [1, 1, 2, 2, 3, 4, 4];  
  
let length = removeDuplicates(arr);  
  
console.log(arr.slice(0, length));
```

Time: $O(n)$

Space: $O(1)$

8. Count Frequency of Elements

Approach 1 — Built-in

```
function frequency(arr) {  
  return arr.reduce((freq, num) => {
```

```
    freq[num] = (freq[num] || 0) + 1;
    return freq;
  }, {});
}

console.log(frequency([1, 2, 2, 3, 1, 2, 3]));
```

Time: $O(n)$

Space: $O(n)$

Approach 2 — Without Built-in reduce()

```
function frequency(arr) {
  let freq = {};

  for (let i = 0; i < arr.length; i++) {
    if (freq[arr[i]]) {
      freq[arr[i]]++;
    } else {
      freq[arr[i]] = 1;
    }
  }

  return freq;
}

console.log(frequency([1, 2, 2, 3, 1, 2, 3]));
```

Time: $O(n)$

Space: $O(n)$

★ Without Map / Set / Reduce

```
function frequency(arr) {
  let result = [];

  for (let i = 0; i < arr.length; i++) {
    let count = 0;
    let alreadyCounted = false;

    for (let j = 0; j < i; j++) {
```

```

    if (arr[i] === arr[j]) {
        alreadyCounted = true;
        break;
    }
}

if (alreadyCounted) {
    continue;
}

for (let j = 0; j < arr.length; j++) {
    if (arr[i] === arr[j]) {
        count++;
    }
}

result.push([arr[i], count]);
}

return result;
}

console.log(frequency([1, 2, 2, 3, 1, 2, 3]));

```

Time: $O(n^2)$

Space: $O(n)$

Level 1 — Quick Revision

#	Question	Best Time	Best Space
1	Largest	$O(n)$	$O(1)$
2	Smallest	$O(n)$	$O(1)$
3	Second Largest	$O(n)$	$O(1)$
4	Second Smallest	$O(n)$	$O(1)$
5	Reverse Array	$O(n)$	$O(1)$
6	Check Sorted	$O(n)$	$O(1)$

#	Question	Best Time	Best Space
7	Remove Duplicates	O(n)	O(1)
8	Frequency	O(n)	O(n)

🔥 Interview ke liye Level 1 mein particularly pakke karo:

Second Largest → Reverse → Check Sorted → Remove Duplicates → Frequency

Inmein **built-in version + without built-in + optimized version + complexity** teeno confidently aane chahiye.

Are yaar main aise Nahin bol rahi hai tum interview ka question bhi to likho ki agar interview ko bolane question ki aise likho to mujhe ek word likhna hai agar aise bolate to yah code likhna Hai is type se likho bhi to samajh mein bhi nahin a raha kya hai aur agar koi solution optimise h phele se hi to tum uss question ke answer me same as this..... Aisa kuch likhdo naki dubara code do

Haan 🤔 ab samajh gaya tum exactly kya chahti ho. **Question ko interviewer ki language mein likhna hai**, phir clearly batana hai **interviewer ne kya bola → kaunsa code likhna hai**. Aur agar Built-in aur Optimized same hain, to code repeat nahi karna.

Chalo **Level 1 ko proper interview format mein** dobara karte hain.

🟢 ARRAY — LEVEL 1

1. Find Largest Element

🎤 Interviewer asks:

“Write a JavaScript function to find the largest element in an array.”

Agar interviewer bole: **“You can use built-in methods.”**

```
function findLargest(arr) {
  return Math.max(...arr);
}
```

```
console.log(findLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(n)$

Agar interviewer bole: **“Don’t use any built-in method.”**

```
function findLargest(arr) {  
  let largest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] > largest) {  
      largest = arr[i];  
    }  
  }  
  
  return largest;  
}
```

```
console.log(findLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

Agar interviewer bole: **“Can you optimize it?”**

Same as the without built-in solution.

Already optimal: $O(n)$ time and $O(1)$ space.

2. Find Smallest Element



Interviewer asks:

“Write a function to find the smallest element in an array.”

Built-in allowed:

```
function findSmallest(arr) {  
  return Math.min(...arr);  
}
```

```
console.log(findSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in:

```
function findSmallest(arr) {  
  let smallest = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] < smallest) {  
      smallest = arr[i];  
    }  
  }  
  
  return smallest;  
}  
  
console.log(findSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

Optimize:

Same as without built-in.

Already optimal: $O(n)$ / $O(1)$.

3. Find Second Largest Element



Interviewer asks:

“Find the second largest element in an array.”

Example:

Input: [10, 25, 7, 40, 15]

Output: 25

Built-in allowed:

```
function secondLargest(arr) {  
  let unique = [...new Set(arr)];  
  
  unique.sort((a, b) => b - a);  
  
  return unique[1];  
}
```

```
}
```

```
console.log(secondLargest([10, 25, 7, 40, 15]));
```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in:

```
function secondLargest(arr) {  
  let largest = -Infinity;  
  let secondLargest = -Infinity;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] > largest) {  
      secondLargest = largest;  
      largest = arr[i];  
    } else if (arr[i] > secondLargest && arr[i] !== largest) {  
      secondLargest = arr[i];  
    }  
  }  
  
  return secondLargest;  
}
```

```
console.log(secondLargest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

Agar interviewer bole: “**Optimize it.**”

Same as the without built-in solution.

Already optimal: **one pass** → $O(n)$ time, $O(1)$ space.

4. Find Second Smallest Element



Interviewer asks:

“Find the second smallest element in an array.”

Example:

Input: [10, 25, 7, 40, 15]

Output: 10

Built-in allowed:

```
function secondSmallest(arr) {  
  let unique = [...new Set(arr)];  
  
  unique.sort((a, b) => a - b);  
  
  return unique[1];  
}  
  
console.log(secondSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in:

```
function secondSmallest(arr) {  
  let smallest = Infinity;  
  let secondSmallest = Infinity;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] < smallest) {  
      secondSmallest = smallest;  
      smallest = arr[i];  
    } else if (arr[i] < secondSmallest && arr[i] !== smallest) {  
      secondSmallest = arr[i];  
    }  
  }  
  
  return secondSmallest;  
}  
  
console.log(secondSmallest([10, 25, 7, 40, 15]));
```

Time: $O(n)$

Space: $O(1)$

Optimize:

Same as without built-in.

Already optimal: $O(n)$ / $O(1)$.

5. Reverse an Array



Interviewer asks:

“Reverse an array.”

Example:

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

Agar interviewer bole: **“Use a built-in method.”**

```
function reverseArray(arr) {  
  return arr.reverse();  
}
```

```
console.log(reverseArray([1, 2, 3, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Agar bole: **“Don’t use built-in methods.”**

```
function reverseArray(arr) {  
  let left = 0;  
  let right = arr.length - 1;  
  
  while (left < right) {  
    let temp = arr[left];  
    arr[left] = arr[right];  
    arr[right] = temp;  
  
    left++;  
    right--;  
  }  
  
  return arr;  
}
```

```
console.log(reverseArray([1, 2, 3, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Agar bole: “**Optimize it.**”

Same as the without built-in solution.

Ye already **in-place optimal solution** hai.

6. Check Whether Array Is Sorted



Interviewer asks:

“Write a function to check whether an array is sorted in ascending order.”

Example:

`[1, 2, 3, 4, 5] → true`

`[1, 3, 2, 4, 5] → false`

Built-in allowed:

```
function isSorted(arr) {  
  let sorted = [...arr].sort((a, b) => a - b);  
  
  return JSON.stringify(arr) === JSON.stringify(sorted);  
}
```

```
console.log(isSorted([1, 2, 3, 4, 5]));
```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in:

```
function isSorted(arr) {  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] < arr[i - 1]) {  
      return false;  
    }  
  }  
  
  return true;  
}
```

```
}
```

```
console.log(isSorted([1, 2, 3, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Agar interviewer bole: **“Optimize it.”**

Same as without built-in solution.

$O(n)$ se better possible nahi hai because array ke elements check karne pad sakte hain.

7. Remove Duplicates From a Sorted Array



Interviewer asks:

“Remove duplicates from a sorted array.”

Example:

Input: [1, 1, 2, 2, 3, 4, 4]

Output: [1, 2, 3, 4]

Built-in allowed:

```
function removeDuplicates(arr) {  
  return [...new Set(arr)];  
}
```

```
console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in:

```
function removeDuplicates(arr) {  
  let result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    if (i === 0 || arr[i] !== arr[i - 1]) {  
      result.push(arr[i]);  
    }  
  }  
}
```



```
    }  
  }  
  
  return result;  
}  
  
console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
```

Time: $O(n)$

Space: $O(n)$

Agar interviewer bole: “Can you optimize space?”

```
function removeDuplicates(arr) {  
  if (arr.length === 0) {  
    return 0;  
  }  
  
  let index = 1;  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] !== arr[i - 1]) {  
      arr[index] = arr[i];  
      index++;  
    }  
  }  
  
  return index;  
}  
  
let arr = [1, 1, 2, 2, 3, 4, 4];  
  
let length = removeDuplicates(arr);  
  
console.log(arr.slice(0, length));
```

Time: $O(n)$

Space: $O(1)$

8. Count Frequency of Each Element



Interviewer asks:

“Count the frequency of each element in an array.”

Example:

Input: [1, 2, 2, 3, 1, 2]

Output: {1: 2, 2: 3, 3: 1}

Built-in allowed:

```
function frequency(arr) {  
  return arr.reduce((freq, num) => {  
    freq[num] = (freq[num] || 0) + 1;  
    return freq;  
  }, {});  
}  
  
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Time: O(n)

Space: O(n)

Without reduce():

```
function frequency(arr) {  
  let freq = {};  
  
  for (let i = 0; i < arr.length; i++) {  
    if (freq[arr[i]]) {  
      freq[arr[i]]++;  
    } else {  
      freq[arr[i]] = 1;  
    }  
  }  
  
  return freq;  
}  
  
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Time: O(n)

Space: O(n)

Agar interviewer bole: “**Optimize it.**”

Same as without reduce() solution.

O(n) time is already optimal for checking every element.

🔥 LEVEL 1 — Tumhe exactly kya yaad rehna chahiye?

Question	Built-in	Without Built-in	Optimized
Largest	Math.max()	Loop	Same
Smallest	Math.min()	Loop	Same
Second Largest	Set + sort()	One pass	Same
Second Smallest	Set + sort()	One pass	Same
Reverse	reverse()	Two Pointer	Same
Sorted	sort()	One loop	Same
Remove Duplicates	Set	Loop	Two Pointer
Frequency	reduce()	Object + loop	Same

Bas isi format mein aage bhi karenge:

Interviewer ka exact question → Built-in bole to code → Without built-in bole to code → Optimize bole to sirf naya code (agar genuinely different hai) → har approach ki Time + Space Complexity.

Code ki explanation **tab tak nahi dunga jab tak tum specifically “explain” nahi bolti.**

7 ka optimize sol smjha de and 8 ka buit in easy language me ki ho kya rha h

Bilkul. Dono ko **line-by-line simple Hinglish** mein samjhte hain.

7. Remove Duplicates — Optimized Solution

Code:

```
function removeDuplicates(arr) {  
  if (arr.length === 0) {  
    return 0;  
  }  
  
  let index = 1;  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] !== arr[i - 1]) {  
      arr[index] = arr[i];  
      index++;  
    }  
  }  
  
  return index;  
}  
  
let arr = [1, 1, 2, 2, 3, 4, 4];  
  
let length = removeDuplicates(arr);  
  
console.log(arr.slice(0, length));
```

Pehle ye samjho

Array **sorted** hai:

[1, 1, 2, 2, 3, 4, 4]

Sorted hone ki wajah se duplicate values **side-by-side** hain.

Hum 2 pointers use kar rahe hain:

- $i \rightarrow$ array ko check karega
- $index \rightarrow$ next unique element ko kahan rakhna hai

Starting:

```
let index = 1;
```

Matlab pehla element 1 already unique hai, isliye usko chhod diya.

Ab:

```
i = 1
```

```
index = 1
```

```
[1, 1, 2, 2, 3, 4, 4]
```

```
  ↑  ↑
```

```
arr[i] = 1
```

```
arr[i - 1] = 1
```

Same hain:

```
if (arr[i] !== arr[i - 1])
```

Condition false → kuch nahi karenge.

Next:

```
i = 2
```

```
[1, 1, 2, 2, 3, 4, 4]
```

```
  ↑
```

Ab 2 aur previous 1 different hain.

```
arr[index] = arr[i];
```

Yaani:

```
arr[1] = 2
```

Array internally:

```
[1, 2, 2, 2, 3, 4, 4]
```

Phir:

```
index++;
```

```
index = 2
```

Next i = 3:

```
2 vs 2
```

Same → duplicate → skip.

Next i = 4:

3 vs 2

Different → 3 ko arr[index] par rakh do.

Result:

```
[1, 2, 3, 2, 3, 4, 4]
      ↑
```

Isi tarah end tak:

```
[1, 2, 3, 4, 3, 4, 4]
```

Actual **unique portion** sirf:

```
[1, 2, 3, 4]
```

Isliye function index return karta hai:

```
return index;
```

```
index = 4
```

Aur:

```
arr.slice(0, length)
```

ka matlab:

Array ke **0 se length-1 tak** elements le aao.

So:

```
[1, 2, 3, 4]
```

🔥 Isko optimize kya banata hai?

Normal solution mein:

```
let result = [];
```

alag array bana rahe the → **O(n)** extra space

Optimized mein **same original array ke andar** unique elements ko aage shift kar diya.

Isliye:

Time: $O(n)$

Space: $O(1)$ 

8. Frequency — Built-in `reduce()` Easy Explanation

Code:

```
function frequency(arr) {  
  return arr.reduce((freq, num) => {  
    freq[num] = (freq[num] || 0) + 1;  
    return freq;  
  }, {});  
}  
  
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Input:

[1, 2, 2, 3, 1, 2]

Hume chahiye:

```
{  
  1: 2,  
  2: 3,  
  3: 1  
}
```

`reduce()` actually kar kya raha hai?

`reduce()` poore array ke elements ko **ek single result** mein convert karta hai.

Yahan initial result hai:

```
{}
```

Matlab starting mein empty object:

```
{}
```

Pehla element: 1

```
freq[1] = (freq[1] || 0) + 1;
```

Abhi freq[1] exist nahi karta.

So:

```
undefined || 0
```

gives:

```
0
```

Then:

```
0 + 1 = 1
```

Object:

```
{ 1: 1 }
```

Dusra element: 2

2 pehli baar aaya:

```
{ 1: 1, 2: 1 }
```

Teesra element: 2

Ab freq[2] already 1 hai.

```
1 + 1 = 2
```

So:

```
{ 1: 1, 2: 2 }
```

Next: 3

```
{ 1: 1, 2: 2, 3: 1 }
```

Next: 1

1 already 1 hai:

```
1 + 1 = 2
```

So:

```
{ 1: 2, 2: 2, 3: 1 }
```

Last: 2

2 already 2 hai:

```
2 + 1 = 3
```

Final:

```
{  
  1: 2,  
  2: 3,  
  3: 1  
}
```

Bas ek line yaad rakho:

```
freq[num] = (freq[num] || 0) + 1;
```

Iska simple meaning:

“Agar number pehle se hai to uski count +1 karo, warna 1 se start karo.”

Aur:

```
return freq;
```

har iteration ke baad updated object ko next iteration mein bhej raha hai.

Time: O(n)

Space: O(n)

Yahi frequency counting ka **sabse common JavaScript pattern** hai.

7 wale me kuch bhi nhi aaya smjh

Haan, 7 wala thoda confusing hai 😓. **Two-pointer/in-place** wala abhi side mein **rakho**. Pehle ekdum basic se samjho ki problem kya hai, phir optimized code.

Question

Interviewer bolta hai:

“Remove duplicates from a sorted array.”

Input:

```
[1, 1, 2, 2, 3, 4, 4]
```

Output:

```
[1, 2, 3, 4]
```

Sabse important baat

Array **sorted** hai, isliye same numbers paas-paas hain:

```
1 1 2 2 3 4 4
↑ ↑
same
```

Hume har number ki **sirf ek copy** rakhni hai.

Pehle normal solution samjho

```
function removeDuplicates(arr) {
  let result = [];

  for (let i = 0; i < arr.length; i++) {
    if (i === 0 || arr[i] !== arr[i - 1]) {
      result.push(arr[i]);
    }
  }

  return result;
}
```

Yahan simple hai:

1

Pehla element hai → rakh lo

```
result = [1]
```

Dusra 1

Previous bhi 1 hai:

```
1 !== 1 → false
```

To nahi rakhenge.

```
result = [1]
```

2

Previous 1 hai:

```
2 !== 1 → true
```

Rakh lo:

```
result = [1, 2]
```

Dusra 2

Previous bhi 2:

```
2 !== 2 → false
```

Skip.

Aise end tak:

```
[1, 2, 3, 4]
```

Ye wala samajh aaya? Isi logic ko optimize karna hai.

Ab optimized kya kar raha hai?

Problem normal solution mein ye hai:

```
let result = [];
```

Humne **naya array** bana diya.

Interviewer bolta hai:

“Can you do it without using extra space?”

To hum **naya array nahi banayenge**.

Original array ko hi use karenge.

Ye code dekho

```
function removeDuplicates(arr) {  
  let index = 1;  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] !== arr[i - 1]) {  
      arr[index] = arr[i];  
      index++;  
    }  
  }  
  
  return index;  
}
```

Ab sirf **ek concept** samjho:

i ka kaam

i array ko **check** karega.

index ka kaam

index batayega:

“Next unique number ko kahan rakhna hai?”

Starting

```
arr = [1, 1, 2, 2, 3, 4, 4]
```

```
index = 1
```

```
i = 1
```

Pehla 1 already apni jagah hai.

`i = 1`

```
arr[i] = 1  
arr[i - 1] = 1
```

Same hain.

So:

kuch nahi karo

`i = 2`

```
arr[i] = 2  
arr[i - 1] = 1
```

Different hain.

Matlab **2 naya/unique number hai.**

Ab:

```
arr[index] = arr[i];
```

Means:

```
arr[1] = arr[2];
```

Yaani:

```
[1, 2, 2, 2, 3, 4, 4]
```

Aur:

```
index++;
```

Ab:

```
index = 2
```

`i = 3`

```
arr[i] = 2  
arr[i - 1] = 2
```

Same → duplicate → **kuch nahi karo.**

`i = 4`

```
arr[i] = 3  
arr[i - 1] = 2
```

Different → unique.

```
arr[index] = arr[i];
```

Ab:

```
arr[2] = arr[4];
```

Array:

```
[1, 2, 3, 2, 3, 4, 4]
```

index becomes 3.

`i = 5`

```
4 != 3
```

Unique.

```
arr[3] = arr[5];
```

Array ka starting part:

```
[1, 2, 3, 4, ...]
```

End mein

Original array technically aisa ho sakta hai:

```
[1, 2, 3, 4, 3, 4, 4]
```

Lekin hume sirf starting ke 4 elements chahiye.

```
[1, 2, 3, 4]
```

Isliye index return kiya.

🔥 Ek line mein pura concept

Normal:

Original array → check → NEW array mein unique values

Optimized:

Original array → check → SAME array ke starting portion mein unique values

Isliye:

Normal: $O(n)$ extra space

Optimized: $O(1)$ extra space ✓

Aur **time dono ka $O(n)$** hai.

Tumhe abhi bas ye 2 cheezein pakadni hain:

`i` = check karega

`index` = unique element ki next position

Bas. **index ko “unique values rakhne ki jagah” samjho.**

`arr[index] = arr[i];` ye line bilkul smjh nhi aari

Haan, **bas isi line ko samajhte hain.** Baaki code bhool jao abhi.

`arr[index] = arr[i];`

Iska simple meaning hai:

Right side ki value ko left side wali position par rakh do.

Ek simple example:

`let arr = [1, 1, 2, 2, 3];`

Maan lo:

`i = 2;`

`index = 1;`

To:

`arr[i]`

means:

```
arr[2]
```

Aur arr[2] hai:

```
2
```

Aur:

```
arr[index]
```

means:

```
arr[1]
```

To line:

```
arr[index] = arr[i];
```

actually ban gayi:

```
arr[1] = arr[2];
```

Yaani:

```
arr[1] = 2
```

Pehle array tha:

```
[1, 1, 2, 2, 3]
```

```
  ↑   ↑  
index i
```

Ab arr[1] mein arr[2] ki value daal di:

```
[1, 2, 2, 2, 3]
```

```
  ↑  
unique 2
```

🤔 “Lekin 2 ko copy karne ki zarurat kya thi?”

YAH! main point hai.

Hume duplicates ko physically delete karne ki zarurat nahi hai.

Hume bas array ke **starting part ko unique banana hai:**

[1, 2, 3, 4, ?, ?, ?]

To jab bhi naya unique number mile, usko starting mein next available position par rakh do.

Example:

Original:

[1, 1, 2, 2, 3, 4, 4]

↑
i

Naya unique 2 mila.

index = 1 hai.

To:

```
arr[1] = arr[2];
```

means:

"Jo 2 mujhe position 2 par mila hai,
use position 1 par rakh do."

Result:

[1, 2, 2, 2, 3, 4, 4]

Phir index aage badhega.

Ek aur super-simple example

Socho tumhare paas seats hain:

Seat 0 → 1

Seat 1 → duplicate

Seat 2 → 2

Seat 3 → duplicate

Seat 4 → 3

Tum bol rahi ho:

“Mujhe unique values starting seats mein rakhni hain.”

To 2 mila seat 2 par, lekin next empty/required unique position **seat 1** hai.

```
arr[1] = arr[2];
```

Bas **2 ko position 2 se position 1 par copy kar diya.**

Yaad rakhne ka formula:

```
arr[index] = arr[i];
```

= **“i wali value ko index wali jagah par rakh do.”**

Bas itna hi. i **dhundh raha hai**, index **rakh raha hai**.

Done next questions easy sa hi dena yrr bhot problem hoti h aise

Haan yaar 🤔 ab se **easy language + clean code + ekdum step-by-step**, unnecessary complicated tricks nahi. Pehle question ka basic solution, phir interviewer agar **built-in / without built-in / optimize** bole tab woh version.

🟢 Question 8 — Count Frequency of Each Element

🎤 Interviewer asks:

“Given an array, count how many times each element occurs.”

Example:

Input: [1, 2, 2, 3, 1, 2]

Output:

1 → 2 times

2 → 3 times

3 → 1 time

Agar interviewer bole: **“Use built-in methods.”**

```
function frequency(arr) {  
  return arr.reduce((count, num) => {  
    count[num] = (count[num] || 0) + 1;  
    return count;  
  }, {});  
}
```

```
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Time: $O(n)$

Space: $O(n)$

Agar interviewer bole: **“Don’t use built-in methods.”**

```
function frequency(arr) {  
  let count = {};  
  
  for (let i = 0; i < arr.length; i++) {  
    if (count[arr[i]]) {  
      count[arr[i]]++;  
    } else {  
      count[arr[i]] = 1;  
    }  
  }  
  
  return count;  
}
```

```
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Time: $O(n)$

Space: $O(n)$

Agar interviewer bole: **“Optimize it.”**

Same as without built-in solution.

$O(n)$ time already optimal hai.

Ab Level 2 — Important Array Questions

Next hum **Question 9: Find Missing Number** se start karenge.

Ye bhi easy hai aur interview mein **bahut common** hai.

Format same rahega:

Question → Built-in → Without built-in → Optimize → Time & Space Complexity.

Aur haan, abhi se unnecessary complicated approach nahi dunga. 👍

Saare dedo level 2 ke aur hard wordings mat dena ek dum simple sober code ho means $j=i+1$ ko tum directly 1 bhi to likh skte ho agar i 0 h to iss type se simple code do naki ko confusing 3 type ke code do ab built in bhi optimise bhi and without built in bhi

Haan yaar, ab **exactly simple style** mein karte hain. 😭

Aur ek correction: **Level 2 ke 10 questions** honge. Har question mein sirf wahi approaches dunga jo interview mein actually useful hain. Agar **without built-in aur optimized same** hai, to dobara code nahi dunga.

● ARRAY — LEVEL 2

9. Find Missing Number

🎤 Interviewer asks:

“Find the missing number from an array containing numbers from 1 to n.”

Input: [1, 2, 3, 5]

Output: 4

Built-in allowed

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  
  for (let i = 1; i <= n; i++) {  
    if (!arr.includes(i)) {  
      return i;  
    }  
  }  
}  
  
console.log(missingNumber([1, 2, 3, 5]));
```

Time: $O(n^2)$

Space: $O(1)$

Without built-in

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  let sum = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    sum += arr[i];  
  }  
  
  let total = n * (n + 1) / 2;  
  
  return total - sum;  
}  
  
console.log(missingNumber([1, 2, 3, 5]));
```

Time: $O(n)$

Space: $O(1)$

Optimize

Same as without built-in.

$O(n)$ time and $O(1)$ space already optimal hai.

10. Find Duplicate Number



Interviewer asks:

“Find the duplicate number in an array.”

Input: [1, 3, 4, 2, 2]

Output: 2

Built-in allowed

```
function findDuplicate(arr) {  
  let seen = new Set();
```

```
for (let num of arr) {  
  if (seen.has(num)) {  
    return num;  
  }  
  
  seen.add(num);  
}  
  
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function findDuplicate(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] === arr[j]) {  
        return arr[i];  
      }  
    }  
  }  
}  
  
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n^2)$

Space: $O(1)$

Optimize

```
function findDuplicate(arr) {  
  let seen = {};  
  
  for (let i = 0; i < arr.length; i++) {  
    if (seen[arr[i]]) {  
      return arr[i];  
    }  
  
    seen[arr[i]] = true;  
  }  
}
```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

11. Find All Duplicate Elements



Interviewer asks:

“Find all duplicate elements in an array.”

Input: [1, 2, 3, 2, 4, 1, 5]

Output: [2, 1]

Built-in allowed

```
function findDuplicates(arr) {  
  let count = {};  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  return Object.keys(count)  
    .filter(num => count[num] > 1)  
    .map(Number);  
}
```

```
console.log(findDuplicates([1, 2, 3, 2, 4, 1, 5]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function findDuplicates(arr) {  
  let result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] === arr[j]) {  
        result.push(arr[i]);  
      }  
    }  
  }  
}
```

```
        break;
    }
}

return result;
}

console.log(findDuplicates([1, 2, 3, 2, 4, 1, 5]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimize

```
function findDuplicates(arr) {
    let seen = {};
    let result = [];

    for (let i = 0; i < arr.length; i++) {
        if (seen[arr[i]]) {
            result.push(arr[i]);
        } else {
            seen[arr[i]] = true;
        }
    }

    return result;
}

console.log(findDuplicates([1, 2, 3, 2, 4, 1, 5]));
```

Time: $O(n)$

Space: $O(n)$

12. Find Intersection of Two Arrays



Interviewer asks:

“Find the common elements between two arrays.”

Input:

[1, 2, 3, 4]

[3, 4, 5, 6]

Output:

[3, 4]

Built-in allowed

```
function intersection(arr1, arr2) {  
  return arr1.filter(num => arr2.includes(num));  
}
```

```
console.log(intersection([1, 2, 3, 4], [3, 4, 5, 6]));
```

Time: $O(n \times m)$

Space: $O(n)$

Without built-in

```
function intersection(arr1, arr2) {  
  let result = [];  
  
  for (let i = 0; i < arr1.length; i++) {  
    for (let j = 0; j < arr2.length; j++) {  
      if (arr1[i] === arr2[j]) {  
        result.push(arr1[i]);  
        break;  
      }  
    }  
  }  
}
```

```
  return result;  
}
```

```
console.log(intersection([1, 2, 3, 4], [3, 4, 5, 6]));
```

Time: $O(n \times m)$

Space: $O(n)$

Optimize

```
function intersection(arr1, arr2) {  
  let set = new Set(arr2);  
  let result = [];
```

```

for (let i = 0; i < arr1.length; i++) {
  if (set.has(arr1[i])) {
    result.push(arr1[i]);
  }
}

return result;
}

console.log(intersection([1, 2, 3, 4], [3, 4, 5, 6]));

```

Time: $O(n + m)$

Space: $O(m)$

13. Find Union of Two Arrays

 Interviewer asks:

“Find the union of two arrays without repeating elements.”

Input:

[1, 2, 3]

[2, 3, 4]

Output:

[1, 2, 3, 4]

Built-in allowed

```

function union(arr1, arr2) {
  return [...new Set([...arr1, ...arr2])];
}

```

```

console.log(union([1, 2, 3], [2, 3, 4]));

```

Time: $O(n + m)$

Space: $O(n + m)$

Without built-in

```

function union(arr1, arr2) {
  let result = [];

```

```
for (let i = 0; i < arr1.length; i++) {  
  if (!result.includes(arr1[i])) {  
    result.push(arr1[i]);  
  }  
}  
  
for (let i = 0; i < arr2.length; i++) {  
  if (!result.includes(arr2[i])) {  
    result.push(arr2[i]);  
  }  
}  
  
return result;  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O((n + m)^2)$

Space: $O(n + m)$

Optimize

```
function union(arr1, arr2) {  
  let set = new Set();  
  
  for (let num of arr1) {  
    set.add(num);  
  }  
  
  for (let num of arr2) {  
    set.add(num);  
  }  
  
  return [...set];  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n + m)$

Space: $O(n + m)$

14. Move All Zeros to the End

 Interviewer asks:

“Move all zeros to the end of the array.”

Input: [0, 1, 0, 3, 12]

Output: [1, 3, 12, 0, 0]

Built-in allowed

```
function moveZeros(arr) {  
  let nonZero = arr.filter(num => num !== 0);  
  let zeros = arr.filter(num => num === 0);  
  
  return [...nonZero, ...zeros];  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: O(n)

Space: O(n)

Without built-in

```
function moveZeros(arr) {  
  let result = [];  
  let zeros = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] !== 0) {  
      result.push(arr[i]);  
    } else {  
      zeros++;  
    }  
  }  
  
  for (let i = 0; i < zeros; i++) {  
    result.push(0);  
  }  
  
  return result;  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(n)$

★ Optimize — Same Array

```
function moveZeros(arr) {  
  let index = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] !== 0) {  
      arr[index] = arr[i];  
      index++;  
    }  
  }  
  
  while (index < arr.length) {  
    arr[index] = 0;  
    index++;  
  }  
  
  return arr;  
}  
  
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(1)$

15. Move Negative Numbers to One Side



Interviewer asks:

“Move all negative numbers to one side of the array.”

Input: [1, -2, 3, -4, 5]

Output: [-2, -4, 1, 3, 5]

Built-in allowed

```
function moveNegative(arr) {  
  return [  
    ...arr.filter(num => num < 0),  
    ...arr.filter(num => num >= 0)  
  ]  
}
```

```
    ];  
  }  
  
  console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in / Optimized

```
function moveNegative(arr) {  
  let index = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] < 0) {  
      let temp = arr[index];  
      arr[index] = arr[i];  
      arr[i] = temp;  
  
      index++;  
    }  
  }  
  
  return arr;  
}
```

```
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Optimize: Same solution. Already $O(n)$ / $O(1)$.

16. Find Pair With Given Sum



Interviewer asks:

“Find two numbers whose sum is equal to the given target.”

Input:

[2, 7, 11, 15]

Target = 9

Output:

[2, 7]

Built-in allowed

```
function findPair(arr, target) {  
  for (let num of arr) {  
    let other = target - num;  
  
    if (arr.includes(other) && other !== num) {  
      return [num, other];  
    }  
  }  
}
```

```
console.log(findPair([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

Without built-in

```
function findPair(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] + arr[j] === target) {  
        return [arr[i], arr[j]];  
      }  
    }  
  }  
}
```

```
console.log(findPair([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

★ Optimize

```
function findPair(arr, target) {  
  let seen = {};  
  
  for (let i = 0; i < arr.length; i++) {  
    let other = target - arr[i];
```

```
    if (seen[other]) {  
        return [other, arr[i]];  
    }  
  
    seen[arr[i]] = true;  
}  
}  
  
console.log(findPair([2, 7, 11, 15], 9));
```

Time: $O(n)$

Space: $O(n)$

17. Find Majority Element



Interviewer asks:

“Find the element that appears more than half of the time.”

Input: [2, 2, 1, 1, 1, 2, 2]

Output: 2

Built-in allowed

```
function majorityElement(arr) {  
    let count = {};  
  
    for (let num of arr) {  
        count[num] = (count[num] || 0) + 1;  
    }  
  
    return Object.keys(count).find(  
        num => count[num] > arr.length / 2  
    );  
}  
  
console.log(majorityElement([2, 2, 1, 1, 1, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function majorityElement(arr) {
  for (let i = 0; i < arr.length; i++) {
    let count = 0;

    for (let j = 0; j < arr.length; j++) {
      if (arr[i] === arr[j]) {
        count++;
      }
    }

    if (count > arr.length / 2) {
      return arr[i];
    }
  }
}

console.log(majorityElement([2, 2, 1, 1, 1, 2, 2]));
```

Time: $O(n^2)$

Space: $O(1)$

★ Optimize — Boyer-Moore

```
function majorityElement(arr) {
  let candidate = arr[0];
  let count = 1;

  for (let i = 1; i < arr.length; i++) {
    if (arr[i] === candidate) {
      count++;
    } else {
      count--;
    }

    if (count === 0) {
      candidate = arr[i];
      count = 1;
    }
  }

  return candidate;
}
```

```
console.log(majorityElement([2, 2, 1, 1, 1, 2, 2]));
```

Time: $O(n)$

Space: $O(1)$ ★

18. Find Elements That Occur More Than Once



Interviewer asks:

“Print all elements that occur more than once.”

Ye **Question 11** jaisa hi hai.

Same as Question 11.

19. Find Pair With Given Difference



Interviewer asks:

“Find two numbers whose difference is equal to the given number.”

Input:

[5, 20, 3, 2, 50, 80]

Difference = 78

Output:

2, 80

Built-in allowed

```
function findDifference(arr, target) {  
  for (let num of arr) {  
    if (arr.includes(num + target)) {  
      return [num, num + target];  
    }  
  }  
}
```

```
console.log(findDifference([5, 20, 3, 2, 50, 80], 78));
```

Time: $O(n^2)$

Space: $O(1)$

Without built-in

```
function findDifference(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (Math.abs(arr[i] - arr[j]) === target) {  
        return [arr[i], arr[j]];  
      }  
    }  
  }  
}
```

```
console.log(findDifference([5, 20, 3, 2, 50, 80], 78));
```

Time: $O(n^2)$

Space: $O(1)$

★ Optimize

```
function findDifference(arr, target) {  
  let seen = new Set();  
  
  for (let num of arr) {  
    if (seen.has(num + target)) {  
      return [num, num + target];  
    }  
  
    if (seen.has(num - target)) {  
      return [num - target, num];  
    }  
  
    seen.add(num);  
  }  
}
```

```
console.log(findDifference([5, 20, 3, 2, 50, 80], 78));
```

Time: $O(n)$

Space: $O(n)$

20. Find Maximum Consecutive 1s

 Interviewer asks:

“Find the maximum number of consecutive 1s in an array.”

Input: [1, 1, 0, 1, 1, 1]

Output: 3

Built-in allowed

```
function maxOnes(arr) {  
  let groups = arr.join("").split("0");  
  let max = 0;  
  
  for (let group of groups) {  
    max = Math.max(max, group.length);  
  }  
  
  return max;  
}
```

```
console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: O(n)

Space: O(n)

Without built-in

```
function maxOnes(arr) {  
  let count = 0;  
  let max = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === 1) {  
      count++;  
    } else {  
      count = 0;  
    }  
  
    if (count > max) {  
      max = count;  
    }  
  }  
}
```

```
    return max;
}

console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

Optimize

Same as without built-in.

Already optimal: $O(n)$ / $O(1)$.

Level 2 mein sabse important

Inko **extra strong** karna:

Missing Number → Duplicate → Intersection → Move Zeros → Two Sum/Pair Sum → Majority Element

Aur ek important cheez: **Question 18 ko alag se prepare karne ki zarurat nahi**, kyunki woh Question 11 ka duplicate hai. Isliye actual unique questions **19 hain, 18 nahi**.

Aage **Level 3** mein hum Array ke genuinely important patterns karenge: **Two Pointer, Sliding Window, Kadane, Rotate Array, Subarray Sum, Stock, Merge Sorted Array, Leaders, etc.**

Abhi bhi strong hi h duplicate find wale question me hum directly frequency count krte then if condition lgake check krte ki iss element ki frequency 1 se jyada h kya agar h to return number Tum faltu mein Aisa kyon kar rahe ho Jo hamare pass ek simple Sa pyara Sa code Hai To Tum mujhe vahi do na ham extra strong kyu kre

Haan 🥲 **bilkul sahi bol rahi ho tum**. Main unnecessarily approaches badha raha tha. Interview preparation mein bhi **simple, readable aur standard code** best hai—jab tak interviewer specifically koi constraint na de.

For **Find Duplicate**, tumhara wala approach perfectly valid hai:

 Interviewer:

“Find the duplicate element in an array.”

Simple & clean solution — Frequency Count

```
function findDuplicate(arr) {  
  let count = {};  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > 1) {  
      return num;  
    }  
  }  
}
```

console.log(findDuplicate([1, 3, 4, 2, 2]));

Time: $O(n)$

Space: $O(n)$

Agar interviewer bole:

“Optimize it.”

Tabhi hum alternative approach dekhenge, **warna yehi code use karo.**

Ab se mera rule:

1. Pehle simplest correct solution.
2. Built-in specifically bole → built-in.
3. Without built-in specifically bole → simple manual code.
4. Optimize specifically bole → tab optimized approach.
5. Agar simple solution already optimal hai → “Same solution” — dobara code nahi.
6. Unnecessarily 3–4 approaches nahi.

Aur “extra strong question” bhi tabhi bolunga jab genuinely koi important concept ho. Har question ko unnecessarily difficult nahi banaunga. 👍

Han ab dobara se level 2 do vo bhi 3no code ke sath building without builtin and optimize start Aur bhot hi Easy answers Dena faltu confusion nhi aur code accha ho na chaiye near and clean naki sb chiz gusani h aur ek hi jgaha saari chize mat likho code ekdum easy peasy hona chaiye tum bhot strong logic use krre ho unnecessary hard krre ho bekaar ho bilkul tum

Haan yaar 🤔 ab **bilkul simple + clean + interview-level** rakhenge.

Level 2 ko dobara karte hain, aur har question me:

- **Built-in**
 - **Without built-in**
 - **Optimized**
 - **Time + Space Complexity**
 - Code **short aur easy**
 - Koi faltu strong logic nahi.
-

● LEVEL 2 — Arrays

Q9. Find the Missing Number

Question: Find the missing number from an array containing numbers from 1 to n.

1 Built-in

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  
  let total = Array.from({ length: n }, (_, i) => i + 1)  
    .reduce((sum, num) => sum + num, 0);  
  
  let current = arr.reduce((sum, num) => sum + num, 0);  
  
  return total - current;  
}  
  
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  let total = 0;  
  let current = 0;  
  
  for (let i = 1; i <= n; i++) {  
    total += i;  
  }  
  
  for (let num of arr) {  
    current += num;  
  }  
  
  return total - current;  
}  
  
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

3 Optimized

Same as without built-in.

Upar wala hi optimal simple solution hai.

Q10. Find Duplicate Number

Question: Find the duplicate number in an array.

1 Built-in

```
function findDuplicate(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
}
```



```
});

return Object.keys(count).find(num => count[num] > 1);
}
```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function findDuplicate(arr) {
  let count = {};

  for (let num of arr) {
    count[num] = (count[num] || 0) + 1;
  }

  for (let num of arr) {
    if (count[num] > 1) {
      return num;
    }
  }
}
```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

3 Optimized

Same as without built-in.

Simple frequency count hi rakhenge. 👍

Q11. Find All Duplicate Elements

Question: Find all elements that appear more than once.

1 Built-in

```
function findDuplicates(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
  
  return Object.keys(count)  
    .filter(num => count[num] > 1)  
    .map(Number);  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > 1 && !result.includes(num)) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n^2)$

Space: $O(n)$

3 Optimized

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num in count) {  
    if (count[num] > 1) {  
      result.push(Number(num));  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n)$

Space: $O(n)$

Q12. Find Intersection of Two Arrays

Question: Find the common elements between two arrays.

1 Built-in

```
function intersection(arr1, arr2) {  
  return arr1.filter(num => arr2.includes(num));  
}  
  
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

2 Without Built-in

```
function intersection(arr1, arr2) {  
  let result = [];  
  
  for (let num of arr1) {  
    for (let value of arr2) {  
      if (num === value) {  
        result.push(num);  
        break;  
      }  
    }  
  }  
  
  return result;  
}  
  
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

3 Optimized

```
function intersection(arr1, arr2) {  
  let set = new Set(arr2);  
  let result = [];  
  
  for (let num of arr1) {  
    if (set.has(num)) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

Q13. Find Union of Two Arrays

Question: Find all unique elements from two arrays.

1 Built-in

```
function union(arr1, arr2) {  
    return [...new Set([...arr1, ...arr2])];  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function union(arr1, arr2) {  
    let result = [];  
  
    for (let num of arr1) {  
        if (!result.includes(num)) {  
            result.push(num);  
        }  
    }  
  
    for (let num of arr2) {  
        if (!result.includes(num)) {  
            result.push(num);  
        }  
    }  
  
    return result;  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

3 Optimized

```
function union(arr1, arr2) {  
    let set = new Set();
```

```
for (let num of arr1) {  
  set.add(num);  
}  
  
for (let num of arr2) {  
  set.add(num);  
}  
  
return [...set];  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: O(n)

Space: O(n)

Q14. Move All Zeros to End

Question: Move all 0s to the end of the array.

1 Built-in

```
function moveZeros(arr) {  
  let zeros = arr.filter(num => num === 0);  
  let numbers = arr.filter(num => num !== 0);  
  
  return [...numbers, ...zeros];  
}  
  
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: O(n)

Space: O(n)

2 Without Built-in

```
function moveZeros(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num !== 0) {  
      result.push(num);  
    }  
  }  
}
```

```
    }  
  }  
  
  for (let num of arr) {  
    if (num === 0) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(n)$

3 Optimized

```
function moveZeros(arr) {  
  let index = 0;  
  
  for (let num of arr) {  
    if (num !== 0) {  
      arr[index] = num;  
      index++;  
    }  
  }  
  
  while (index < arr.length) {  
    arr[index] = 0;  
    index++;  
  }  
  
  return arr;  
}  
  
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(1)$

Q15. Move Negative Numbers to One Side

Question: Move all negative numbers to one side of the array.

1 Built-in

```
function moveNegative(arr) {  
  let negative = arr.filter(num => num < 0);  
  let positive = arr.filter(num => num >= 0);  
  
  return [...negative, ...positive];  
}  
  
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: O(n)

Space: O(n)

2 Without Built-in

```
function moveNegative(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num < 0) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr) {  
    if (num >= 0) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: O(n)

Space: O(n)

3 Optimized

```
function moveNegative(arr) {  
  let index = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] < 0) {  
      [arr[index], arr[i]] = [arr[i], arr[index]];  
      index++;  
    }  
  }  
  
  return arr;  
}  
  
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Q16. Find Pair With Given Sum

Question: Find two numbers whose sum is equal to the given target.

1 Built-in

```
function pairSum(arr, target) {  
  let result = arr.find((num, i) =>  
    arr.includes(target - num, i + 1)  
  );  
  
  if (result !== undefined) {  
    return [result, target - result];  
  }  
}
```

```
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

2 Without Built-in

```
function pairSum(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] + arr[j] === target) {  
        return [arr[i], arr[j]];  
      }  
    }  
  }  
}
```

```
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

3 Optimized

```
function pairSum(arr, target) {  
  let seen = {};  
  
  for (let num of arr) {  
    let needed = target - num;  
  
    if (seen[needed]) {  
      return [needed, num];  
    }  
  
    seen[num] = true;  
  }  
}
```

```
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n)$

Space: $O(n)$

Q17. Find Majority Element

Question: Find the element that appears more than $n/2$ times.

1 Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
  
  return Object.keys(count).find(num => count[num] > arr.length / 2);  
}  
  
console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > arr.length / 2) {  
      return num;  
    }  
  }  
}  
  
console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

3 Optimized

```
function majorityElement(arr) {  
  let count = 0;  
  let candidate = null;  
  
  for (let num of arr) {
```

```

    if (count === 0) {
        candidate = num;
    }

    count += num === candidate ? 1 : -1;
}

return candidate;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));

```

Time: $O(n)$

Space: $O(1)$

Q18. Find Pair With Given Difference

Question: Find two numbers whose difference is equal to the given value.

1 Built-in

```

function pairDifference(arr, target) {
    return arr.find(num =>
        arr.includes(num + target)
    );
}

console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));

```

Time: $O(n^2)$

Space: $O(1)$

2 Without Built-in

```

function pairDifference(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (Math.abs(arr[i] - arr[j]) === target) {
                return [arr[i], arr[j]];
            }
        }
    }
}

```

```
}
```

```
console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n^2)$

Space: $O(1)$

3 Optimized

```
function pairDifference(arr, target) {  
  let seen = new Set();  
  
  for (let num of arr) {  
    if (seen.has(num - target) || seen.has(num + target)) {  
      return true;  
    }  
  
    seen.add(num);  
  }  
  
  return false;  
}
```

```
console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n)$

Space: $O(n)$

Q19. Find Maximum Consecutive 1s

Question: Find the maximum number of consecutive 1s in an array.

1 Built-in

```
function maxOnes(arr) {  
  let groups = arr.join('').split('0');  
  
  return Math.max(...groups.map(group => group.length));  
}
```

```
console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(n)$

2 Without Built-in

```
function maxOnes(arr) {  
  let count = 0;  
  let max = 0;  
  
  for (let num of arr) {  
    if (num === 1) {  
      count++;  
    } else {  
      count = 0;  
    }  
  
    if (count > max) {  
      max = count;  
    }  
  }  
  
  return max;  
}
```

```
console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

3 Optimized

Same as without built-in.

✓ Level 2 Done

Q9 → Q19 = 11 important questions

Aur ab se **same simple pattern** rakhenge. Koi Floyd, weird pointer tricks, unnecessary advanced logic **nahi ghusayenge** jab tak interviewer specifically optimize na bole.

Abhi bhi bhot bekaar h bhai kuch smjh bhi nhi aata such btau jo tum building in wala code dere ho na vo kaafi hard lgira mujhe baaki to sb shi h fir bhi ab dubara do pura level 2

Haan bhai 🤔 samajh gaya. **Built-in wale bhi bilkul basic** rakhunga — koi reduce, find, filter + includes ka combo nahi. Jitna simple ho sakta hai utna simple.

Aur ek correction: **built-in ka matlab easy built-in method use karna**, complicated one-liner banana nahi.

● LEVEL 2 — Arrays

Q9. Find the Missing Number

Question: Find the missing number from 1 to n.

Built-in

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  
  let total = (n * (n + 1)) / 2;  
  let sum = arr.reduce((a, b) => a + b, 0);  
  
  return total - sum;  
}  
  
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function missingNumber(arr) {  
  let n = arr.length + 1;  
  let total = 0;  
  let sum = 0;  
  
  for (let i = 1; i <= n; i++) {  
    total += i;  
  }  
}
```

```
    for (let num of arr) {  
        sum += num;  
    }  
  
    return total - sum;  
}  
  
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

Q10. Find Duplicate Number

Question: Find the number that appears more than once.

Built-in

```
function findDuplicate(arr) {  
    let unique = new Set(arr);  
  
    for (let num of arr) {  
        if (!unique.has(num)) {  
            return num;  
        }  
    }  
}
```

Time: $O(n)$

Space: $O(n)$

Actually, simple frequency counting is clearer here, so **interview ke liye wahi use karna best hai.**

Without Built-in

```
function findDuplicate(arr) {  
    let count = {};
```



```
for (let num of arr) {  
  count[num] = (count[num] || 0) + 1;  
}  
  
for (let num of arr) {  
  if (count[num] > 1) {  
    return num;  
  }  
}  
}  
  
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

Q11. Find All Duplicate Elements

Question: Find all numbers that appear more than once.

Built-in

```
function findDuplicates(arr) {  
  let unique = [...new Set(arr)];  
  let result = [];  
  
  for (let num of unique) {  
    if (arr.filter(x => x === num).length > 1) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n^2)$

Space: $O(n)$

Without Built-in

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > 1 && !result.includes(num)) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num in count) {  
    if (count[num] > 1) {  
      result.push(Number(num));  
    }  
  }  
  
  return result;  
}
```

```
}
```

```
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n)$

Space: $O(n)$

Q12. Find Intersection of Two Arrays

Question: Find the common elements in two arrays.

Built-in

```
function intersection(arr1, arr2) {  
    return arr1.filter(num => arr2.includes(num));  
}
```

```
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Without Built-in

```
function intersection(arr1, arr2) {  
    let result = [];  
  
    for (let num of arr1) {  
        for (let value of arr2) {  
            if (num === value) {  
                result.push(num);  
                break;  
            }  
        }  
    }  
  
    return result;  
}
```

```
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function intersection(arr1, arr2) {  
  let set = new Set(arr2);  
  let result = [];  
  
  for (let num of arr1) {  
    if (set.has(num)) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}
```

```
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

Q13. Find Union of Two Arrays

Question: Combine two arrays and remove duplicate elements.

Built-in

```
function union(arr1, arr2) {  
  return [...new Set(arr1.concat(arr2))];  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function union(arr1, arr2) {  
  let result = [];
```

```
for (let num of arr1) {  
  if (!result.includes(num)) {  
    result.push(num);  
  }  
}  
  
for (let num of arr2) {  
  if (!result.includes(num)) {  
    result.push(num);  
  }  
}  
  
return result;  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function union(arr1, arr2) {  
  let set = new Set();  
  
  for (let num of arr1) {  
    set.add(num);  
  }  
  
  for (let num of arr2) {  
    set.add(num);  
  }  
  
  return [...set];  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

Q14. Move All Zeros to End

Question: Move all zeros to the end of the array.

Built-in

```
function moveZeros(arr) {  
  let numbers = arr.filter(num => num !== 0);  
  let zeros = arr.filter(num => num === 0);  
  
  return numbers.concat(zeros);  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: O(n)

Space: O(n)

Without Built-in

```
function moveZeros(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num !== 0) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr) {  
    if (num === 0) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: O(n)

Space: O(n)

Optimized

```
function moveZeros(arr) {  
  let index = 0;  
  
  for (let num of arr) {  
    if (num !== 0) {  
      arr[index] = num;  
      index++;  
    }  
  }  
  
  while (index < arr.length) {  
    arr[index] = 0;  
    index++;  
  }  
  
  return arr;  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(1)$

Q15. Move Negative Numbers to One Side

Question: Move all negative numbers to one side.

Built-in

```
function moveNegative(arr) {  
  let negative = arr.filter(num => num < 0);  
  let positive = arr.filter(num => num >= 0);  
  
  return negative.concat(positive);  
}
```

```
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function moveNegative(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num < 0) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr) {  
    if (num >= 0) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function moveNegative(arr) {  
  let index = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] < 0) {  
      [arr[index], arr[i]] = [arr[i], arr[index]];  
      index++;  
    }  
  }  
  
  return arr;  
}  
  
console.log(moveNegative([1, -2, 3, -4, 5]));
```


Time: $O(n)$
Space: $O(1)$

Q16. Find Pair With Given Sum

Question: Find two numbers whose sum is equal to the target.

Built-in

```
function pairSum(arr, target) {  
  for (let num of arr) {  
    let other = target - num;  
  
    if (arr.includes(other)) {  
      return [num, other];  
    }  
  }  
}  
  
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$
Space: $O(1)$

Without Built-in

```
function pairSum(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] + arr[j] === target) {  
        return [arr[i], arr[j]];  
      }  
    }  
  }  
}  
  
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$
Space: $O(1)$

Optimized

```
function pairSum(arr, target) {  
  let seen = {};  
  
  for (let num of arr) {  
    let other = target - num;  
  
    if (seen[other]) {  
      return [other, num];  
    }  
  
    seen[num] = true;  
  }  
}  
  
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: O(n)

Space: O(n)

Q17. Find Majority Element

Question: Find the element that appears more than $n/2$ times.

Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
  
  for (let num in count) {  
    if (count[num] > arr.length / 2) {  
      return Number(num);  
    }  
  }  
}
```

```
console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > arr.length / 2) {  
      return num;  
    }  
  }  
}
```

```
console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function majorityElement(arr) {  
  let candidate = arr[0];  
  let count = 1;  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] === candidate) {  
      count++;  
    } else {  
      count--;  
    }  
  }  
  
  if (count === 0) {  
    candidate = arr[i];  
    count = 1;  
  }  
}
```

```
}

return candidate;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Time: $O(n)$

Space: $O(1)$

Q18. Find Pair With Given Difference

Question: Find two numbers whose difference is equal to the target.

Built-in

```
function pairDifference(arr, target) {
  for (let num of arr) {
    if (arr.includes(num + target)) {
      return [num, num + target];
    }
  }
}

console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n^2)$

Space: $O(1)$

Without Built-in

```
function pairDifference(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (Math.abs(arr[i] - arr[j]) === target) {
        return [arr[i], arr[j]];
      }
    }
  }
}

console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n^2)$

Space: $O(1)$

Optimized

```
function pairDifference(arr, target) {  
  let seen = new Set();  
  
  for (let num of arr) {  
    if (seen.has(num - target) || seen.has(num + target)) {  
      return true;  
    }  
  
    seen.add(num);  
  }  
  
  return false;  
}  
  
console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n)$

Space: $O(n)$

Q19. Find Maximum Consecutive 1s

Question: Find the maximum number of consecutive 1s.

Built-in

```
function maxOnes(arr) {  
  let count = 0;  
  let max = 0;  
  
  arr.forEach(num => {  
    if (num === 1) {  
      count++;  
    } else {  
      count = 0;  
    }  
  })  
}
```

```
    max = Math.max(max, count);
  });

  return max;
}

console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function maxOnes(arr) {
  let count = 0;
  let max = 0;

  for (let num of arr) {
    if (num === 1) {
      count++;
    } else {
      count = 0;
    }

    if (count > max) {
      max = count;
    }
  }

  return max;
}

console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

✓ Level 2 Final List

1. Missing Number
2. Duplicate Number
3. All Duplicate Elements
4. Intersection
5. Union
6. Move Zeros to End
7. Move Negative Numbers
8. Pair With Given Sum
9. Majority Element
10. Pair With Given Difference
11. Maximum Consecutive 1s

Ab ab isi difficulty pe challenge. Built-in me bhi simple methods, without-built-in me basic loops, aur optimization tabhi jab actual improvement ho.

Sach bataun bilkul bhi samajh mein Nahin a Raha tumne itne heavy heavy logic use kiye Bina mere ko kuchh samajh mein Nahin a Raha

Haan yaar, **meri galti hai**. Main “3 approaches” dene ke chakkar mein code ko unnecessarily difficult bana raha tha. 😭

Ab ekdum **zero-confusion rule**:

- Pehle **sirf basic/simple code**.
- `reduce()`, `Set`, `filter()`, `find()`, `includes()` ke combinations **nahi**.
- Simple for loop + if condition.
- Har question ka **ek hi easy code**.

- Complexity bhi saath.
- **Optimization baad mein**, sirf jab tum bolo “optimize karo”.
- Built-in bhi tabhi dunga jab tum specifically bolo “**built-in wala do**”.

Example — Duplicate Number

Question: Find the duplicate number.

```
function findDuplicate(arr) {
  let count = {};

  for (let num of arr) {
    count[num] = (count[num] || 0) + 1;
  }

  for (let num of arr) {
    if (count[num] > 1) {
      return num;
    }
  }
}

console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Bas itna hi. Isme koi extra logic nahi.

Aur honestly, **Level 2 ko abhi dobara 11 questions ke 3-3 versions mein dena galat hoga**, kyunki wahi phir overload ho jayega.

👉 Ab hum **Level 2 ek-ek question karke** karenge.

Main **Q9 se start karunga**, sirf **easy code + complexity**. Phir tum “next” bolna.

Tumumje ab dubara level 2 do yrr par ab sb easy hoga cool and calm ha 3no method buil in bhi do without bhi optimize bhi par easy means jyada mat ghusao chizo ko

Haan yrr ❤️ **ab cool & calm**. Is baar main complexity nahi ghusaunga.

3 methods rahenge, but har method **jitna simple ho sakta hai utna simple** hoga.

Ek aur rule: agar optimized code bhi same simple code hai, to “Same as without built-in” likhunga.

LEVEL 2 — ARRAYS

Q9. Find Missing Number

Question: Find the missing number from 1 to n.

Built-in

```
function missingNumber(arr) {  
    let n = arr.length + 1;  
  
    let total = (n * (n + 1)) / 2;  
    let sum = arr.reduce((a, b) => a + b, 0);  
  
    return total - sum;  
}  
  
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function missingNumber(arr) {  
    let n = arr.length + 1;  
    let total = 0;  
    let sum = 0;  
  
    for (let i = 1; i <= n; i++) {  
        total += i;  
    }  
  
    for (let num of arr) {  
        sum += num;  
    }  
  
    return total - sum;  
}
```

```
console.log(missingNumber([1, 2, 4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

Ye already simple aur optimal hai.

Q10. Find Duplicate Number

Question: Find the number that appears more than once.

Built-in

```
function findDuplicate(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
  
  for (let num of arr) {  
    if (count[num] > 1) {  
      return num;  
    }  
  }  
}
```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function findDuplicate(arr) {  
  let count = {};  
  
  for (let num of arr) {
```

```

    count[num] = (count[num] || 0) + 1;
  }

  for (let num of arr) {
    if (count[num] > 1) {
      return num;
    }
  }
}

```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

Q11. Find All Duplicate Elements

Question: Find all numbers that appear more than once.

Built-in

```

function findDuplicates(arr) {
  let count = {};

  arr.forEach(num => {
    count[num] = (count[num] || 0) + 1;
  });

  return Object.keys(count)
    .filter(num => count[num] > 1)
    .map(Number);
}

console.log(findDuplicates([1, 2, 3, 2, 4, 3]));

```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > 1) {  
      if (!result.includes(num)) {  
        result.push(num);  
      }  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num in count) {  
    if (count[num] > 1) {  
      result.push(Number(num));  
    }  
  }  
  
  return result;  
}
```

```
console.log(findDuplicates([1, 2, 3, 2, 4, 3]));
```

Time: $O(n)$

Space: $O(n)$

Q12. Find Intersection of Two Arrays

Question: Find the common elements of two arrays.

Built-in

```
function intersection(arr1, arr2) {  
  return arr1.filter(num => arr2.includes(num));  
}
```

```
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Without Built-in

```
function intersection(arr1, arr2) {  
  let result = [];  
  
  for (let num of arr1) {  
    for (let value of arr2) {  
      if (num === value) {  
        result.push(num);  
        break;  
      }  
    }  
  }  
  
  return result;  
}
```

```
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function intersection(arr1, arr2) {  
  let result = [];  
  let count = {};  
  
  for (let num of arr2) {  
    count[num] = true;  
  }  
  
  for (let num of arr1) {  
    if (count[num]) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(intersection([1, 2, 3], [2, 3, 4]));
```

Time: O(n)

Space: O(n)

Q13. Find Union of Two Arrays

Question: Combine two arrays and remove duplicates.

Built-in

```
function union(arr1, arr2) {  
  return [...new Set(arr1.concat(arr2))];  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: O(n)

Space: O(n)

Without Built-in

```
function union(arr1, arr2) {  
  let result = [];  
  
  for (let num of arr1) {  
    if (!result.includes(num)) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr2) {  
    if (!result.includes(num)) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function union(arr1, arr2) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr1) {  
    count[num] = true;  
    result.push(num);  
  }  
  
  for (let num of arr2) {  
    if (!count[num]) {  
      count[num] = true;  
      result.push(num);  
    }  
  }  
  
  return result;  
}
```

```
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));
```

Time: $O(n)$

Space: $O(n)$

Q14. Move All Zeros to End

Question: Move all 0s to the end.

Built-in

```
function moveZeros(arr) {  
  let numbers = arr.filter(num => num !== 0);  
  let zeros = arr.filter(num => num === 0);  
  
  return numbers.concat(zeros);  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function moveZeros(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num !== 0) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr) {  
    if (num === 0) {  
      result.push(num);  
    }  
  }  
}
```



```
    return result;
}

console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function moveZeros(arr) {
    let index = 0;

    for (let num of arr) {
        if (num !== 0) {
            arr[index] = num;
            index++;
        }
    }

    while (index < arr.length) {
        arr[index] = 0;
        index++;
    }

    return arr;
}

console.log(moveZeros([0, 1, 0, 3, 12]));
```

Time: $O(n)$

Space: $O(1)$

Q15. Move Negative Numbers to One Side

Question: Move all negative numbers to one side.

Built-in

```
function moveNegative(arr) {
    let negative = arr.filter(num => num < 0);
    let positive = arr.filter(num => num >= 0);
```

```
    return negative.concat(positive);  
}
```

```
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function moveNegative(arr) {  
    let result = [];
```

```
    for (let num of arr) {  
        if (num < 0) {  
            result.push(num);  
        }  
    }  
}
```

```
    for (let num of arr) {  
        if (num >= 0) {  
            result.push(num);  
        }  
    }  
}
```

```
    return result;  
}
```

```
console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function moveNegative(arr) {  
    let index = 0;
```

```
    for (let i = 0; i < arr.length; i++) {  
        if (arr[i] < 0) {  
            [arr[index], arr[i]] = [arr[i], arr[index]];  
            index++;  
        }  
    }
```

```
}

return arr;
}

console.log(moveNegative([1, -2, 3, -4, 5]));
```

Time: $O(n)$

Space: $O(1)$

Q16. Find Pair With Given Sum

Question: Find two numbers whose sum is equal to the target.

Built-in

```
function pairSum(arr, target) {
  for (let num of arr) {
    if (arr.includes(target - num)) {
      return [num, target - num];
    }
  }
}
```

```
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

Without Built-in

```
function pairSum(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
}
```

```
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n^2)$

Space: $O(1)$

Optimized

```
function pairSum(arr, target) {  
  let seen = {};  
  
  for (let num of arr) {  
    let needed = target - num;  
  
    if (seen[needed]) {  
      return [needed, num];  
    }  
  
    seen[num] = true;  
  }  
}  
  
console.log(pairSum([2, 7, 11, 15], 9));
```

Time: $O(n)$

Space: $O(n)$

Q17. Find Majority Element

Question: Find the element that appears more than $n/2$ times.

Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  arr.forEach(num => {  
    count[num] = (count[num] || 0) + 1;  
  });  
  
  for (let num in count) {  
    if (count[num] > arr.length / 2) {  
      return Number(num);  
    }  
  }  
}
```

```
}  
}  
  
console.log(majorityElement([2, 2, 1, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function majorityElement(arr) {  
  let count = {};  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num of arr) {  
    if (count[num] > arr.length / 2) {  
      return num;  
    }  
  }  
}
```

```
console.log(majorityElement([2, 2, 1, 2, 2]));
```

Time: $O(n)$

Space: $O(n)$

Optimized

Isko abhi simple hi rakho.

Frequency-count wala solution hi easy aur interview ke liye enough hai.

Q18. Find Pair With Given Difference

Question: Find two numbers whose difference is equal to the target.

Built-in

```
function pairDifference(arr, target) {  
  for (let num of arr) {
```

```

        if (arr.includes(num + target)) {
            return [num, num + target];
        }
    }
}

console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));

```

Time: $O(n^2)$

Space: $O(1)$

Without Built-in

```

function pairDifference(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (Math.abs(arr[i] - arr[j]) === target) {
                return [arr[i], arr[j]];
            }
        }
    }
}

console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));

```

Time: $O(n^2)$

Space: $O(1)$

Optimized

```

function pairDifference(arr, target) {
    let seen = {};

    for (let num of arr) {
        if (seen[num - target] || seen[num + target]) {
            return true;
        }

        seen[num] = true;
    }

    return false;
}

```

```
console.log(pairDifference([5, 20, 3, 2, 50, 80], 45));
```

Time: $O(n)$

Space: $O(n)$

Q19. Maximum Consecutive 1s

Question: Find the maximum number of consecutive 1s.

Built-in

```
function maxOnes(arr) {  
  let count = 0;  
  let max = 0;  
  
  arr.forEach(num => {  
    if (num === 1) {  
      count++;  
    } else {  
      count = 0;  
    }  
  
    max = Math.max(max, count);  
  });  
  
  return max;  
}
```

```
console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function maxOnes(arr) {  
  let count = 0;  
  let max = 0;  
  
  for (let num of arr) {  
    if (num === 1) {
```

```
        count++;
    } else {
        count = 0;
    }

    if (count > max) {
        max = count;
    }
}

return max;
}

console.log(maxOnes([1, 1, 0, 1, 1, 1]));
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

✓ LEVEL 2 COMPLETE

Q9 → Q19

Ab isko hi tumhara **Level 2 final version** maan lo.

Aur next levels mein bhi **isi difficulty ka code** rahega — **simple, clean, samajhne layak**, na ki unnecessarily “strong” logic.

Ek kaam kroarray baad me krte h searching sorting ke questions kitne h btao and start kro

Haan, **Arrays abhi side mein rakhte hain.** 😊

Searching + Sorting ko placement/interview ke hisaab se roughly **20 important questions** mein kar lenge:

🔍 Searching — 8

1. Linear Search

2. Binary Search
3. Search an element in sorted array
4. Find first occurrence
5. Find last occurrence
6. Count occurrences of an element
7. Find floor/ceil of an element
8. Search in rotated sorted array

Sorting — 12

9. Bubble Sort
10. Selection Sort
11. Insertion Sort
12. Merge Sort
13. Quick Sort
14. Sort an array of 0s, 1s and 2s
15. Sort array in ascending order
16. Sort array in descending order
17. Find kth smallest element
18. Find kth largest element
19. Sort strings
20. Check if array is sorted

Important: Searching/sorting mein hum **actual interview mein commonly asked wale** karenge, random questions nahi.

Aur tumhare format mein: **Built-in → Without built-in → Optimized → Time/Space Complexity**

Lekin code **ekdum basic**. Chalo start karte hain. 📌

SEARCHING

Q1. Linear Search

Question: Find an element in an array using Linear Search.

Built-in

```
function search(arr, target) {  
    return arr.indexOf(target);  
}
```

```
}
```

```
console.log(search([10, 20, 30, 40], 30));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function search(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
}
```

```
  return -1;  
}
```

```
console.log(search([10, 20, 30, 40], 30));
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

Linear Search ke liye ye already best simple solution hai.

Q2. Binary Search

Question: Find an element in a **sorted array** using Binary Search.

Built-in

```
function search(arr, target) {  
  return arr.indexOf(target);  
}
```

```
console.log(search([10, 20, 30, 40, 50], 40));
```

Time: $O(n)$

Space: $O(1)$

Without Built-in

```
function search(arr, target) {  
  let start = 0;  
  let end = arr.length - 1;  
  
  while (start <= end) {  
    let middle = Math.floor((start + end) / 2);  
  
    if (arr[middle] === target) {  
      return middle;  
    }  
  
    if (arr[middle] < target) {  
      start = middle + 1;  
    } else {  
      end = middle - 1;  
    }  
  }  
  
  return -1;  
}
```

```
console.log(search([10, 20, 30, 40, 50], 40));
```

Time: $O(\log n)$

Space: $O(1)$

Optimized

Same as without built-in.

Binary Search ka ye hi standard optimized solution hai.

But tumne binary wale me without builtin me math . Floor ka use karliya 🙄

Haan 🙄 bilkul sahi pakda! Math.floor() bhi built-in method hai.

But **Binary Search ke liye Math.floor() use karna normal hai**, kyunki hume middle index integer chahiye. Agar interviewer specifically bole “**koi bhi built-in method use mat karo**”, to Math.floor() bhi avoid karna padega.

Simple way:

```
let middle = (start + end) >> 1;
```

Lekin >> bhi thoda confusing hai, isliye **interview ke liye Math.floor() wala code hi best/simple hai.**

Aur ek important correction: **JavaScript me direct built-in Binary Search method hota hi nahi hai**, isliye indexOf() ko Binary Search ka built-in version bolna sahi nahi tha. indexOf() internally linear search karta hai.

So Binary Search ke liye hum **sirf standard manual code** rakhenge. No unnecessary jugaad. 😞

Yaar kya ho gaya tumhen tum meri baten kyon Nahin samajh Pa rahe ho tum samajh hi nahin Rahe Ho Main tumne subah Se mera dimag kharab kar diya itna time nahin hai mere pass

Yaar, **meri galti hai.** 😞 Tum baar-baar clearly bata rahi ho ki tumhare paas time kam hai aur tumhe **simple, interview-focused, fixed format** me preparation karni hai, aur main unnecessary cheezein add karke confuse kar raha hoon.

Ab se **exactly tumhare format me** challenge:

- **Question** → interviewer jaise poochega
- **Built-in allowed** → simplest built-in code
- **Without built-in** → genuinely without built-in
- **Optimized** → sirf jab actual improvement ho
- **Time + Space Complexity** → har code ke saath
- **No unnecessary explanation**
- **No fancy algorithms**
- Agar built-in version exist nahi karta → **seedha bolunga**, fake approach nahi dunga.
- Math.floor() jaisi cheez ko “without built-in” nahi bolunga.

Aur sabse important: **jo questions genuinely placement ke liye important hain wahi karenge, list ko unnecessarily bada nahi karenge.**

Tumhara time waste hua — **sorry. Ab seedha preparation pe aate hain.**

Ab jaldi se jitne bhi searching aur shorting question Hai sare ek bar mein hi de do

Haan. Ab ekdum final, placement-focused list de rahi hoon — **Searching + Sorting ke saare important questions ek hi jagah.** Random ya unnecessarily hard questions nahi.

SEARCHING — 10 Questions

1. Find an element in an array using Linear Search
 2. Find an element in a sorted array using Binary Search
 3. Find the first occurrence of an element in a sorted array
 4. Find the last occurrence of an element in a sorted array
 5. Count the occurrences of an element in a sorted array
 6. Find the floor of a given element in a sorted array
 7. Find the ceil of a given element in a sorted array
 8. Search an element in a rotated sorted array
 9. Find the position of an element in a sorted array
 10. Find the missing number using searching logic
-

SORTING — 15 Questions

Basic Sorting Algorithms

11. Sort an array using Bubble Sort
12. Sort an array using Selection Sort
13. Sort an array using Insertion Sort
14. Sort an array using Merge Sort
15. Sort an array using Quick Sort

Common Interview Problems

16. Sort an array in ascending order
17. Sort an array in descending order
18. Sort an array containing only 0s, 1s and 2s
19. Find the kth smallest element
20. Find the kth largest element
21. Sort an array of strings alphabetically
22. Sort an array of objects based on a property
23. Merge two sorted arrays
24. Check whether an array is already sorted

25. Find the median of a sorted array

★ Placement ke liye priority

Agar **time bahut kam hai**, toh inko sabse pehle karo:

Searching

- Linear Search ★★
- Binary Search ★★
- First Occurrence ★★
- Last Occurrence ★★
- Count Occurrences ★
- Rotated Sorted Array ★

Sorting

- Bubble Sort ★★
- Selection Sort ★★
- Insertion Sort ★★
- Merge Sort ★★
- Quick Sort ★★
- 0,1,2 Sort ★
- kth Smallest ★
- kth Largest ★
- Merge Two Sorted Arrays ★
- Check Sorted ★

Total: 25 questions.

Aur haan — **ab in 25 ko hi karenge**. Har question me tumhare fixed format ke according **simple code** + **complexity**, bina faltu ki complexity ke.

Ek kam karo pahle tum mujhe sare important question bata do matlab array mein kitne question hai kaun kaun se string mein kaun se hai kitne hai is type most important Jo interview mein aate Hain service based company mein target kar rahi hun okay

Haan, **ab pehle poora DSA roadmap/final question list decide kar lete hain**, phir ek-ek question karenge. Tumhara target **TCS, Infosys, Accenture, Cognizant, Capgemini, Wipro jaise service-based companies ke fresher placements** hain, isliye main product-company level ke random hard questions nahi daalungi. Current 2026 prep sources bhi arrays, strings, hashing, basic sorting/searching ko highest priority dete hain. ([Phiny](#))

FINAL DSA QUESTION LIST — SERVICE-BASED FRESHER

1. ARRAY — 25 Questions ★★ ★

Basic — must do

1. Find largest element
2. Find smallest element
3. Find second largest element
4. Find second smallest element
5. Reverse an array
6. Check if array is sorted
7. Remove duplicates from sorted array
8. Count frequency of elements
9. Find missing number
10. Find duplicate element
11. Find all duplicate elements
12. Find element that appears only once

Array manipulation

13. Move all zeros to the end
14. Move all negative elements to one side
15. Left rotate array by one position
16. Left rotate array by K positions
17. Right rotate array by K positions
18. Find union of two arrays
19. Find intersection of two arrays

Important interview logic

20. Find pair with given sum — Two Sum
21. Find pair with given difference

22. Find majority element
23. Find maximum consecutive 1s
24. Find maximum subarray sum — Kadane's Algorithm
25. Merge two sorted arrays

These kinds of array questions are repeatedly emphasized in current service-company prep material, including duplicates, missing numbers, sorting, searching, rotations, Two Sum, majority element, Kadane and merging sorted arrays. ([Scribd](#))

2. STRING — 20 Questions

Basic

1. Reverse a string
2. Check if a string is palindrome
3. Count vowels and consonants
4. Count frequency of characters
5. Find first non-repeating character
6. Find first repeating character
7. Remove duplicate characters
8. Remove spaces from a string
9. Count words in a string
10. Find length of string without using length

Very important

11. Check if two strings are anagrams
12. Check if two strings are equal
13. Convert lowercase to uppercase
14. Convert uppercase to lowercase
15. Find ASCII value of a character
16. Replace a character in a string
17. Remove special characters
18. Reverse words in a string
19. Find longest word in a string
20. Find longest common prefix

These basic string operations—anagram, palindrome, reversal, character frequency, duplicate removal, whitespace/special-character handling—show up frequently in service-company question sets. ([Scribd](#))

3. SEARCHING — 6 Questions ★★★★★

1. Linear Search
2. Binary Search
3. First occurrence of an element
4. Last occurrence of an element
5. Count occurrences of an element
6. Search in a rotated sorted array

बस. Searching ko unnecessarily 10–15 questions ka nahi banayenge. Service-based fresher level ke liye ye enough important set hai. Current guides specifically linear/binary search ko core focus rakhte hain. ([InterviewMantra](#))

4. SORTING — 10 Questions ★★★★★

Algorithms — MUST KNOW

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort

Problems

6. Sort array in ascending order
7. Sort array in descending order
8. Sort array of 0s, 1s and 2s
9. Find kth smallest element
10. Find kth largest element

Sorting algorithms are worth knowing for both coding and technical interviews; recent 2026 reports also show selection-sort/sorting-based questions appearing in TCS-style assessments. ([Reddit](#))

5. HASHING / FREQUENCY — 8 Questions ★★

Ye Arrays + Strings ke saath bahut important hai.

1. Count frequency of array elements
 2. Count frequency of characters
 3. Find duplicate elements
 4. Find first non-repeating element
 5. Find first repeating element
 6. Two Sum
 7. Check if two arrays have same elements
 8. Check if two strings are anagrams
-

6. LINKED LIST — 10 Questions ★★

Service-based interviews ke liye basic level:

1. Create a linked list
2. Traverse a linked list
3. Insert at beginning
4. Insert at end
5. Delete a node
6. Search an element
7. Reverse a linked list
8. Find middle node
9. Detect a cycle
10. Merge two sorted linked lists

Linked-list basics, especially reverse, middle, cycle and merge, are commonly included after arrays/strings. ([Scribd](#))

7. STACK & QUEUE — 8 Questions ★★

Stack

1. Implement stack using array
2. Push and Pop operations
3. Check balanced parentheses
4. Reverse a string using stack

Queue

5. Implement queue using array
 6. Enqueue and Dequeue
 7. Implement circular queue
 8. Difference between Stack and Queue + basic implementation
-

8. RECURSION — 7 Questions ★★

1. Factorial
 2. Fibonacci
 3. Sum of first N numbers
 4. Reverse a number
 5. Reverse a string
 6. Check palindrome using recursion
 7. Power of a number
-

9. BASIC MATH / NUMBER LOGIC — 15 Questions ★★★

Service companies me ye **bahut useful** hai.

1. Check even or odd
2. Check positive/negative
3. Find largest of 2/3 numbers
4. Reverse a number
5. Check palindrome number

6. Count digits
7. Sum of digits
8. Armstrong number
9. Prime number
10. Prime numbers in a range
11. Factorial
12. Fibonacci
13. GCD
14. LCM
15. Power of a number

Service-company coding rounds often include basic math/number logic alongside arrays and strings. ([InterviewMantra](#))

10. PATTERN PRINTING — 10 Important Patterns ★★

1. Square pattern
 2. Right triangle
 3. Inverted triangle
 4. Pyramid
 5. Inverted pyramid
 6. Number triangle
 7. Same number triangle
 8. Floyd's triangle
 9. 0-1 triangle
 10. Diamond
-

 **TOTAL**

Topic	Questions
Arrays	25
Strings	20

Topic	Questions
Searching	6
Sorting	10
Hashing	8
Linked List	10
Stack & Queue	8
Recursion	7
Basic Math	15
Patterns	10
TOTAL	119

But important 📌

119 questions ka matlab ye nahi ki tumhe 119 hard LeetCode questions karne hain.

Tumhare target ke liye **Arrays + Strings + Searching + Sorting + Hashing + Basic Math** sabse pehle strong karenge. Current service-company guidance also puts arrays/strings/basic sorting/searching at the center, with easy-to-medium difficulty being typical for fresher hiring. ([Apply](#))

🔥 **Tumhare liye priority order**

Array → String → Searching → Sorting → Hashing → Math → Linked List → Stack/Queue → Recursion → Patterns

Aur **har question** exactly tumhare format me karenge:

Question → Built-in allowed → Without built-in → Optimized (only if genuinely different) → Time Complexity → Space Complexity

Ab **list final hai**. Isme baar-baar naye random questions add nahi karenge.

Superb ek kaam karta hai string se shuru karte Hain teenon code do building method se bhi without building bhi aur optimise code bhi pura string ek bar mein karao lets go logic bekar mein confusing mat Karna jitne se kam ho sakta hai utna batao Jo interview impress bhi ho jaaye ha please ab easy dena buit in me tum bekaar kr dete ho aur output bhi do yrr

Bilkul yrr 🥺❤️ ab exactly tumhare format mein.

String ke **20 important questions** ek saath. Code **simple**, output bhi, aur jahan optimized genuinely same hai wahan repeat nahi karungi.

Note: “Built-in” ka matlab simple JS built-in use karna hai — koi faltu one-liner/complex combination nahi.

STRINGS — 20 MOST IMPORTANT INTERVIEW QUESTIONS

1. Reverse a String

Question: Reverse a given string.

Built-in

```
function reverse(str) {  
  return str.split('').reverse().join('');  
}
```

```
console.log(reverse("hello"));
```

Output: olleh

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function reverse(str) {  
  let result = '';  
  
  for (let i = str.length - 1; i >= 0; i--) {  
    result += str[i];  
  }  
  
  return result;  
}
```

```
console.log(reverse("hello"));
```

Output: olleh

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

2. Check Palindrome String

Question: Check whether a string is a palindrome.

Built-in

```
function isPalindrome(str) {  
    return str === str.split('').reverse().join('');  
}
```

```
console.log(isPalindrome("madam"));
```

Output: true

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function isPalindrome(str) {  
    for (let i = 0; i < str.length / 2; i++) {  
        if (str[i] !== str[str.length - 1 - i]) {  
            return false;  
        }  
    }  
}
```

```
    return true;  
}
```

```
console.log(isPalindrome("madam"));
```

Output: true

Time: $O(n)$

Space: $O(1)$

Optimized

```
function isPalindrome(str) {  
  let start = 0;  
  let end = str.length - 1;  
  
  while (start < end) {  
    if (str[start] !== str[end]) {  
      return false;  
    }  
  
    start++;  
    end--;  
  }  
  
  return true;  
}  
  
console.log(isPalindrome("madam"));
```

Output: true

Time: O(n)

Space: O(1)

3. Count Vowels and Consonants

Question: Count the vowels and consonants in a string.

Built-in

```
function count(str) {  
  let vowels = str.match(/[aeiou]/gi) || [];  
  let letters = str.match(/[a-z]/gi) || [];  
  
  return {  
    vowels: vowels.length,  
    consonants: letters.length - vowels.length  
  };  
}  
  
console.log(count("hello"));
```


Output:

```
{ vowels: 2, consonants: 3 }
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function count(str) {  
  let vowels = 0;  
  let consonants = 0;  
  
  for (let ch of str.toLowerCase()) {  
    if ("aeiou".includes(ch)) {  
      vowels++;  
    } else if (ch >= "a" && ch <= "z") {  
      consonants++;  
    }  
  }  
  
  return { vowels, consonants };  
}  
  
console.log(count("hello"));
```

Output:

```
{ vowels: 2, consonants: 3 }
```

Time: $O(n)$

Space: $O(n)$

Optimized

Same logic; no meaningful improvement needed.

4. Count Frequency of Characters

Question: Find the frequency of each character in a string.

Built-in

```
function frequency(str) {  
  let count = {};  
  
  for (let ch of str) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
  
  return count;  
}  
  
console.log(frequency("hello"));
```

Output:

```
{ h: 1, e: 1, l: 2, o: 1 }
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function frequency(str) {  
  let count = {};  
  
  for (let i = 0; i < str.length; i++) {  
    if (count[str[i]]) {  
      count[str[i]]++;  
    } else {  
      count[str[i]] = 1;  
    }  
  }  
  
  return count;  
}  
  
console.log(frequency("hello"));
```

Output:

```
{ h: 1, e: 1, l: 2, o: 1 }
```

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

5. Find First Non-Repeating Character

Question: Find the first character that occurs only once.

Built-in

```
function firstNonRepeating(str) {  
  let count = {};  
  
  for (let ch of str) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
  
  return [...str].find(ch => count[ch] === 1);  
}  
  
console.log(firstNonRepeating("swiss"));
```

Output: w

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function firstNonRepeating(str) {  
  let count = {};  
  
  for (let ch of str) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
  
  for (let ch of str) {  
    if (count[ch] === 1) {  
      return ch;  
    }  
  }  
}
```

```
console.log(firstNonRepeating("swiss"));
```

Output: w

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

6. Find First Repeating Character

Question: Find the first character that repeats.

Built-in

```
function firstRepeating(str) {  
  let count = {};  
  
  for (let ch of str) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
  
  return [...str].find(ch => count[ch] > 1);  
}
```

```
console.log(firstRepeating("hello"));
```

Output: l

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function firstRepeating(str) {  
  let count = {};  
  
  for (let ch of str) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
}
```

```
for (let ch of str) {  
  if (count[ch] > 1) {  
    return ch;  
  }  
}  
}  
  
console.log(firstRepeating("hello"));
```

Output: l

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

7. Remove Duplicate Characters

Question: Remove duplicate characters from a string.

Built-in

```
function removeDuplicates(str) {  
  return [...new Set(str)].join("");  
}  
  
console.log(removeDuplicates("programming"));
```

Output: progamin

Time: O(n)

Space: O(n)

Without Built-in

```
function removeDuplicates(str) {  
  let result = "";  
  
  for (let ch of str) {  
    if (!result.includes(ch)) {  
      result += ch;  
    }  
  }  
}
```

```
}

    return result;
}

console.log(removeDuplicates("programming"));
```

Output: progamin

Time: $O(n^2)$

Space: $O(n)$

Optimized

```
function removeDuplicates(str) {
    let seen = {};
    let result = "";

    for (let ch of str) {
        if (!seen[ch]) {
            seen[ch] = true;
            result += ch;
        }
    }

    return result;
}

console.log(removeDuplicates("programming"));
```

Output: progamin

Time: $O(n)$

Space: $O(n)$

8. Remove Spaces

Question: Remove all spaces from a string.

Built-in

```
function removeSpaces(str) {
    return str.replaceAll(" ", "");
}
```

```
console.log(removeSpaces("hello world"));
```

Output: helloworld

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function removeSpaces(str) {  
  let result = "";  
  
  for (let ch of str) {  
    if (ch !== " ") {  
      result += ch;  
    }  
  }  
  
  return result;  
}
```

```
console.log(removeSpaces("hello world"));
```

Output: helloworld

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

9. Count Words

Question: Count the number of words in a string.

Built-in

```
function countWords(str) {  
  return str.trim().split(/\s+/).length;  
}
```

```
console.log(countWords("I love JavaScript"));
```

Output: 3

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function countWords(str) {  
  let count = 0;  
  let inWord = false;  
  
  for (let ch of str) {  
    if (ch !== " " && !inWord) {  
      count++;  
      inWord = true;  
    }  
  
    if (ch === " ") {  
      inWord = false;  
    }  
  }  
  
  return count;  
}
```

```
console.log(countWords("I love JavaScript"));
```

Output: 3

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

10. Find Length Without `length`

Question: Find the length of a string without using `length`.

Built-in

```
function findLength(str) {  
  return [...str].length;  
}
```



```
}
```

```
console.log(findLength("hello"));
```

Output: 5

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function findLength(str) {
```

```
  let count = 0;
```

```
  for (let ch of str) {
```

```
    count++;
```

```
  }
```

```
  return count;
```

```
}
```

```
console.log(findLength("hello"));
```

Output: 5

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

11. Check Two Strings Are Anagrams

Question: Check whether two strings are anagrams.

Built-in

```
function isAnagram(a, b) {
```

```
  return a.split("").sort().join("") ===
```

```
    b.split("").sort().join("");
```

```
}
```

```
console.log(isAnagram("listen", "silent"));
```

Output: true

Time: $O(n \log n)$

Space: $O(n)$

Without Built-in

```
function isAnagram(a, b) {  
  if (a.length !== b.length) {  
    return false;  
  }  
  
  let count = {};  
  
  for (let ch of a) {  
    count[ch] = (count[ch] || 0) + 1;  
  }  
  
  for (let ch of b) {  
    if (!count[ch]) {  
      return false;  
    }  
  
    count[ch]--;  
  }  
  
  return true;  
}  
  
console.log(isAnagram("listen", "silent"));
```

Output: true

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in — $O(n)$.

12. Check Two Strings Are Equal

Question: Check whether two strings are equal.

Built-in

```
function isEqual(a, b) {  
  return a === b;  
}  
  
console.log(isEqual("hello", "hello"));
```

Output: true

Time: O(n)

Space: O(1)

Without Built-in

```
function isEqual(a, b) {  
  if (a.length !== b.length) {  
    return false;  
  }  
  
  for (let i = 0; i < a.length; i++) {  
    if (a[i] !== b[i]) {  
      return false;  
    }  
  }  
  
  return true;  
}  
  
console.log(isEqual("hello", "hello"));
```

Output: true

Time: O(n)

Space: O(1)

Optimized

Same as without built-in.

13. Convert Lowercase to Uppercase

Question: Convert a string to uppercase.

Built-in

```
function upper(str) {  
  return str.toUpperCase();  
}
```

```
console.log(upper("hello"));
```

Output: HELLO

Time: O(n)

Space: O(n)

Without Built-in

```
function upper(str) {  
  let result = "";  
  
  for (let ch of str) {  
    if (ch >= "a" && ch <= "z") {  
      result += String.fromCharCode(ch.charCodeAt(0) - 32);  
    } else {  
      result += ch;  
    }  
  }  
  
  return result;  
}
```

```
console.log(upper("hello"));
```

Output: HELLO

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

14. Convert Uppercase to Lowercase

Question: Convert a string to lowercase.

Built-in

```
function lower(str) {  
  return str.toLowerCase();  
}  
  
console.log(lower("HELLO"));
```

Output: hello

Time: O(n)

Space: O(n)

Without Built-in

```
function lower(str) {  
  let result = "";  
  
  for (let ch of str) {  
    if (ch >= "A" && ch <= "Z") {  
      result += String.fromCharCode(ch.charCodeAt(0) + 32);  
    } else {  
      result += ch;  
    }  
  }  
  
  return result;  
}  
  
console.log(lower("HELLO"));
```

Output: hello

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

15. Find ASCII Value

Question: Find the ASCII value of a character.

Built-in

```
function ascii(ch) {  
  return ch.charCodeAt(0);  
}
```

```
console.log(ascii("A"));
```

Output: 65

Time: O(1)

Space: O(1)

Without Built-in

ASCII conversion in JavaScript requires character-code functionality, so **there is no sensible manual version.**

Optimized

Same as built-in.

16. Replace a Character

Question: Replace all occurrences of a character with another character.

Built-in

```
function replaceChar(str) {  
  return str.replaceAll("a", "x");  
}
```

```
console.log(replaceChar("banana"));
```

Output: bxnxbnx

Time: O(n)

Space: O(n)

Without Built-in

```
function replaceChar(str) {  
  let result = "";  
  
  for (let ch of str) {
```

```
    if (ch === "a") {
        result += "x";
    } else {
        result += ch;
    }
}

return result;
}

console.log(replaceChar("banana"));
```

Output: bxnxbxn

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

17. Remove Special Characters

Question: Remove special characters from a string.

Built-in

```
function removeSpecial(str) {
    return str.replace(/[^a-zA-Z0-9]/g, "");
}

console.log(removeSpecial("hello@123!"));
```

Output: hello123

Time: O(n)

Space: O(n)

Without Built-in

```
function removeSpecial(str) {
    let result = "";

    for (let ch of str) {
```

```
    if (
      (ch >= "a" && ch <= "z") ||
      (ch >= "A" && ch <= "Z") ||
      (ch >= "0" && ch <= "9")
    ) {
      result += ch;
    }
  }

  return result;
}

console.log(removeSpecial("hello@123!"));
```

Output: hello123

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

18. Reverse Words in a String

Question: Reverse the order of words in a string.

Built-in

```
function reverseWords(str) {
  return str.split(" ").reverse().join(" ");
}
```

```
console.log(reverseWords("I love coding"));
```

Output: coding love I

Time: O(n)

Space: O(n)

Without Built-in

```
function reverseWords(str) {
  let words = str.split(" ");
```



```
let result = "";

for (let i = words.length - 1; i >= 0; i--) {
  result += words[i];

  if (i !== 0) {
    result += " ";
  }
}

return result;
}

console.log(reverseWords("I love coding"));
```

Output: coding love I

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

19. Find Longest Word

Question: Find the longest word in a string.

Built-in

```
function longestWord(str) {
  return str.split(" ").reduce((longest, word) =>
    word.length > longest.length ? word : longest
  );
}
```

```
console.log(longestWord("I love JavaScript"));
```

Output: JavaScript

Time: O(n)

Space: O(n)

Without Built-in

```
function longestWord(str) {  
  let words = str.split(" ");  
  let longest = "";  
  
  for (let word of words) {  
    if (word.length > longest.length) {  
      longest = word;  
    }  
  }  
  
  return longest;  
}  
  
console.log(longestWord("I love JavaScript"));
```

Output: JavaScript

Time: O(n)

Space: O(n)

Optimized

Same as without built-in.

20. Find Longest Common Prefix ★

Question: Find the longest common prefix among strings.

Example: ["flower", "flow", "flight"] → fl

Built-in

```
function commonPrefix(arr) {  
  let prefix = arr[0];  
  
  for (let word of arr) {  
    while (!word.startsWith(prefix)) {  
      prefix = prefix.slice(0, -1);  
    }  
  }  
}
```

```
    return prefix;
}

console.log(commonPrefix(["flower", "flow", "flight"]));
```

Output: fl

Time: $O(n \times m)$

Space: $O(1)$

Without Built-in

```
function commonPrefix(arr) {
    let prefix = "";

    for (let i = 0; i < arr[0].length; i++) {
        let ch = arr[0][i];

        for (let j = 1; j < arr.length; j++) {
            if (arr[j][i] !== ch) {
                return prefix;
            }
        }

        prefix += ch;
    }

    return prefix;
}

console.log(commonPrefix(["flower", "flow", "flight"]));
```

Output: fl

Time: $O(n \times m)$

Space: $O(1)$

Optimized

Same as without built-in.

STRING — बस ये याद रखो

20 questions complete. But placement ke liye sabki equal importance nahi hai.

★★★ MUST DO

1. Reverse String
2. Palindrome
3. Character Frequency
4. First Non-Repeating Character
5. First Repeating Character
6. Remove Duplicates
7. Anagram
8. Reverse Words
9. Longest Word
10. Longest Common Prefix

★★ Do after these

- Vowels & consonants
- Remove spaces
- Count words
- String equality
- Case conversion
- Replace character
- Remove special characters

★ Basic

- ASCII value
- Length without length

Aur ek correction: Q3, Q13–15 jaise questions mein “without built-in” technically pure built-in-free banana JavaScript mein awkward ho sakta hai. Interview mein **simple loop wala version** hi practical hai; tumhe unnecessary tricks yaad karne ki zarurat nahi.

Next topic: Searching → Sorting.

Jra last question ka without builtin explain krna line by line

Bilkul. **Q20 Longest Common Prefix** ka without-built-in code hi line by line samajhte hain.

Code:

```
function commonPrefix(arr) {  
    let prefix = "";  
  
    for (let i = 0; i < arr[0].length; i++) {  
        let ch = arr[0][i];  
  
        for (let j = 1; j < arr.length; j++) {  
            if (arr[j][i] !== ch) {  
                return prefix;  
            }  
        }  
  
        prefix += ch;  
    }  
  
    return prefix;  
}  
  
console.log(commonPrefix(["flower", "flow", "flight"]));
```

Step 1

```
let prefix = "";
```

Ek empty string banayi.

Isme hum **common characters** store karenge.

Initially:

```
prefix = ""
```

Step 2

```
for (let i = 0; i < arr[0].length; i++)
```

```
arr[0] = "flower"
```

Hum first word ke characters ko ek-ek karke check karenge.

```
flower
```

```
012345
```

So $i = 0 \rightarrow f$

Step 3

```
let ch = arr[0][i];
```

First word ka current character ch mein store kiya.

Ab:

```
ch = "f"
```

Step 4

```
for (let j = 1; j < arr.length; j++)
```

Ab baaki words ke **same position** wale characters check karenge.

j = 1:

```
flow
```

```
 ^
```

```
f
```

```
flow[0] = "f"
```

Step 5

```
if (arr[j][i] !== ch)
```

Check:

```
flow[0] !== "f"  
"f" !== "f"
```

False.

Matlab character same hai. 👍

Next:

```
j = 2
```

```
flight[0] = "f"
```

Again same.

Step 6

```
prefix += ch;
```

Sab words mein f same mila, toh f ko prefix mein add kar diya.

```
prefix = "f"
```

Next round: i = 1

First word:

```
flower  
↑
```

```
ch = "l"
```

Baaki:

```
flow[1]  = "l"  
flight[1] = "l"
```

Same → add l.

```
prefix = "fl"
```

Next round: i = 2

First word ka:

```
flower[2] = "o"
```

Check:

```
flow[2] = "o" ✓
```

```
flight[2] = "i" ✗
```

Ab:

```
if (arr[j][i] !== ch)
```

true ho gaya.

Isliye:

```
return prefix;
```

return karega:

```
"fl"
```

Final Output

```
fl
```

🧠 Bas logic itna hai:

First word ke character ko lo → same position par baaki words mein check karo → sab same hain toh prefix mein add karo → jahan difference mile, wahi stop.

flower, flow, flight

```
f l o
```

```
↑ ↑ ↑
```

```
✓ ✓ ✗
```

```
Common = "fl"
```

Yahi **Longest Common Prefix** hai.

Iska builtin bhi bta sirf main logic wala part

Haan, **sirf main logic:**

```
let prefix = arr[0];

for (let word of arr) {
  while (!word.startsWith(prefix)) {
    prefix = prefix.slice(0, -1);
  }
}
```

Isme main logic:

- `prefix = arr[0]` → pehle word ko prefix maan liya.
- `startsWith(prefix)` → check karta hai word given prefix se start ho raha hai ya nahi.
- Agar nahi:

```
prefix = prefix.slice(0, -1);
```

→ prefix ka **last character hata do.**

- Jab tak common prefix nahi milta, chhota karte jao.

Example:

flower
flow
flight

flower → flow → fl

Answer = fl.

Amazing now searching sorting in one lecture

Yes 🤓 **ab Searching + Sorting ko ek hi lecture mein finish karte hain.**

Total **16 important questions** — service-based fresher interviews ke liye. No random hard stuff.



1. Linear Search ★★

Question: Find an element in an array.

Built-in

```
function search(arr, target) {  
  return arr.indexOf(target);  
}
```

```
console.log(search([10, 20, 30, 40], 30));
```

Output: 2

Time: $O(n)$ | **Space:** $O(1)$

Without Built-in

```
function search(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
}
```

```
  return -1;  
}
```

```
console.log(search([10, 20, 30, 40], 30));
```

Output: 2

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in.

2. Binary Search ★★

Question: Find an element in a sorted array using Binary Search.

Built-in

JavaScript mein **direct built-in Binary Search method nahi hai**, so manual code use karenge.

Without Built-in / Optimized

```
function search(arr, target) {  
  let start = 0;  
  let end = arr.length - 1;  
  
  while (start <= end) {  
    let middle = Math.floor((start + end) / 2);  
  
    if (arr[middle] === target) {  
      return middle;  
    }  
  
    if (arr[middle] < target) {  
      start = middle + 1;  
    } else {  
      end = middle - 1;  
    }  
  }  
  
  return -1;  
}
```

```
console.log(search([10, 20, 30, 40, 50], 40));
```

Output: 3

Time: $O(\log n)$ | **Space:** $O(1)$

3. First Occurrence ★★

Question: Find the first occurrence of an element in a sorted array.

Built-in

```
console.log([1, 2, 2, 2, 3].indexOf(2));
```

Output: 1

Time: $O(n)$ | **Space:** $O(1)$

Without Built-in

```
function firstOccurrence(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
  
  return -1;  
}  
  
console.log(firstOccurrence([1, 2, 2, 2, 3], 2));
```

Output: 1

Time: $O(n)$ | **Space:** $O(1)$

Optimized — Binary Search

```
function firstOccurrence(arr, target) {  
  let start = 0;  
  let end = arr.length - 1;  
  let answer = -1;  
  
  while (start <= end) {  
    let middle = Math.floor((start + end) / 2);  
  
    if (arr[middle] === target) {  
      answer = middle;  
      end = middle - 1;  
    } else if (arr[middle] < target) {  
      start = middle + 1;  
    } else {  
      end = middle - 1;  
    }  
  }  
  
  return answer;  
}  
  
console.log(firstOccurrence([1, 2, 2, 2, 3], 2));
```

Output: 1

Time: $O(\log n)$ | **Space:** $O(1)$

4. Last Occurrence ★★

Question: Find the last occurrence of an element.

Built-in

```
console.log([1, 2, 2, 2, 3].lastIndexOf(2));
```

Output: 3

Time: $O(n)$ | **Space:** $O(1)$

Without Built-in

```
function lastOccurrence(arr, target) {  
  for (let i = arr.length - 1; i >= 0; i--) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
  
  return -1;  
}  
  
console.log(lastOccurrence([1, 2, 2, 2, 3], 2));
```

Output: 3

Time: $O(n)$ | **Space:** $O(1)$

Optimized — Binary Search

```
function lastOccurrence(arr, target) {  
  let start = 0;  
  let end = arr.length - 1;  
  let answer = -1;  
  
  while (start <= end) {  
    let middle = Math.floor((start + end) / 2);  
  
    if (arr[middle] === target) {  
      answer = middle;  
    }  
  }  
}
```

```

        start = middle + 1;
    } else if (arr[middle] < target) {
        start = middle + 1;
    } else {
        end = middle - 1;
    }
}

return answer;
}

console.log(lastOccurrence([1, 2, 2, 2, 3], 2));

```

Output: 3

Time: $O(\log n)$ | **Space:** $O(1)$

5. Count Occurrences ★★

Question: Count how many times an element occurs.

Built-in

```
console.log([1, 2, 2, 2, 3].filter(x => x === 2).length);
```

Output: 3

Time: $O(n)$ | **Space:** $O(n)$

Without Built-in

```

function countOccurrences(arr, target) {
    let count = 0;

    for (let num of arr) {
        if (num === target) {
            count++;
        }
    }

    return count;
}

console.log(countOccurrences([1, 2, 2, 2, 3], 2));

```

Output: 3

Time: O(n) | **Space:** O(1)

Optimized

Sorted array mein:

Count = Last Occurrence – First Occurrence + 1

Time: O(log n) | **Space:** O(1)

6. Search in Rotated Sorted Array ★★

Question: Search an element in a rotated sorted array.

Example:

[4, 5, 6, 7, 0, 1, 2]

Without Built-in

```
function search(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
}
```

```
  return -1;  
}
```

```
console.log(search([4, 5, 6, 7, 0, 1, 2], 0));
```

Output: 4

Time: O(n) | **Space:** O(1)

Optimized

```
function search(arr, target) {  
  let start = 0;  
  let end = arr.length - 1;  
  
  while (start <= end) {
```

```

    let middle = Math.floor((start + end) / 2);

    if (arr[middle] === target) {
        return middle;
    }

    if (arr[start] <= arr[middle]) {
        if (target >= arr[start] && target < arr[middle]) {
            end = middle - 1;
        } else {
            start = middle + 1;
        }
    } else {
        if (target > arr[middle] && target <= arr[end]) {
            start = middle + 1;
        } else {
            end = middle - 1;
        }
    }
}

return -1;
}

console.log(search([4, 5, 6, 7, 0, 1, 2], 0));

```

Output: 4

Time: $O(\log n)$ | **Space:** $O(1)$

SORTING

7. Bubble Sort ★★

Question: Sort an array using Bubble Sort.

Built-in

```
console.log([5, 3, 1, 4, 2].sort((a, b) => a - b));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n \log n)$ | **Space:** $O(n)$

Without Built-in

```
function bubbleSort(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = 0; j < arr.length - i - 1; j++) {  
      if (arr[j] > arr[j + 1]) {  
        let temp = arr[j];  
        arr[j] = arr[j + 1];  
        arr[j + 1] = temp;  
      }  
    }  
  }  
  
  return arr;  
}
```

```
console.log(bubbleSort([5, 3, 1, 4, 2]));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
function bubbleSort(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    let swapped = false;  
  
    for (let j = 0; j < arr.length - i - 1; j++) {  
      if (arr[j] > arr[j + 1]) {  
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];  
        swapped = true;  
      }  
    }  
  
    if (!swapped) {  
      break;  
    }  
  }  
  
  return arr;  
}
```

Best case: $O(n)$

Worst case: $O(n^2)$

Space: $O(1)$

8. Selection Sort ★★

Question: Sort an array using Selection Sort.

Built-in

```
console.log([5, 3, 1, 4, 2].sort((a, b) => a - b));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n \log n)$ | **Space:** $O(n)$

Without Built-in / Optimized

```
function selectionSort(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    let min = i;  
  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[j] < arr[min]) {  
        min = j;  
      }  
    }  
  
    let temp = arr[i];  
    arr[i] = arr[min];  
    arr[min] = temp;  
  }  
  
  return arr;  
}
```

```
console.log(selectionSort([5, 3, 1, 4, 2]));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n^2)$ | **Space:** $O(1)$

9. Insertion Sort ★★

Question: Sort an array using Insertion Sort.

Built-in

```
console.log([5, 3, 1, 4, 2].sort((a, b) => a - b));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n \log n)$ | **Space:** $O(n)$

Without Built-in / Optimized

```
function insertionSort(arr) {  
  for (let i = 1; i < arr.length; i++) {  
    let current = arr[i];  
    let j = i - 1;  
  
    while (j >= 0 && arr[j] > current) {  
      arr[j + 1] = arr[j];  
      j--;  
    }  
  
    arr[j + 1] = current;  
  }  
  
  return arr;  
}
```

```
console.log(insertionSort([5, 3, 1, 4, 2]));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n^2)$ | **Space:** $O(1)$

Best case: $O(n)$

10. Merge Sort ★★

Question: Sort an array using Merge Sort.

Built-in

```
console.log([5, 3, 1, 4, 2].sort((a, b) => a - b));
```

Output: [1, 2, 3, 4, 5]

Without Built-in / Optimized

```
function mergeSort(arr) {  
  if (arr.length <= 1) {  
    return arr;  
  }  
  
  let middle = Math.floor(arr.length / 2);  
  
  let left = mergeSort(arr.slice(0, middle));  
  let right = mergeSort(arr.slice(middle));  
  
  return merge(left, right);  
}  
  
function merge(left, right) {  
  let result = [];  
  let i = 0;  
  let j = 0;  
  
  while (i < left.length && j < right.length) {  
    if (left[i] < right[j]) {  
      result.push(left[i]);  
      i++;  
    } else {  
      result.push(right[j]);  
      j++;  
    }  
  }  
  
  return result.concat(left.slice(i), right.slice(j));  
}  
  
console.log(mergeSort([5, 3, 1, 4, 2]));
```

Output: [1, 2, 3, 4, 5]

Time: $O(n \log n)$ | **Space:** $O(n)$

11. Quick Sort ★★

Question: Sort an array using Quick Sort.

Built-in

```
console.log([5, 3, 1, 4, 2].sort((a, b) => a - b));
```

Output: [1, 2, 3, 4, 5]

Without Built-in / Optimized

```
function quickSort(arr) {  
  if (arr.length <= 1) {  
    return arr;  
  }  
  
  let pivot = arr[arr.length - 1];  
  let left = [];  
  let right = [];  
  
  for (let i = 0; i < arr.length - 1; i++) {  
    if (arr[i] < pivot) {  
      left.push(arr[i]);  
    } else {  
      right.push(arr[i]);  
    }  
  }  
  
  return [...quickSort(left), pivot, ...quickSort(right)];  
}  
  
console.log(quickSort([5, 3, 1, 4, 2]));
```

Output: [1, 2, 3, 4, 5]

Average Time: $O(n \log n)$

Worst Time: $O(n^2)$

Space: $O(n)$

12. Sort 0s, 1s and 2s ★★

Question: Sort an array containing only 0, 1 and 2.

Built-in

```
console.log([2, 0, 1, 2, 1, 0].sort((a, b) => a - b));
```

Output: [0, 0, 1, 1, 2, 2]

Without Built-in

```
function sort012(arr) {  
  let count0 = 0;  
  let count1 = 0;  
  let count2 = 0;  
  
  for (let num of arr) {  
    if (num === 0) count0++;  
    else if (num === 1) count1++;  
    else count2++;  
  }  
  
  let result = [];  
  
  while (count0--) result.push(0);  
  while (count1--) result.push(1);  
  while (count2--) result.push(2);  
  
  return result;  
}  
  
console.log(sort012([2, 0, 1, 2, 1, 0]));
```

Output: [0, 0, 1, 1, 2, 2]

Time: O(n) | **Space:** O(n)

Optimized — Dutch National Flag

Time: O(n) | **Space:** O(1)

13. Sort Ascending ★★

Question: Sort an array in ascending order.

Built-in

```
console.log([5, 2, 8, 1].sort((a, b) => a - b));
```

Output: [1, 2, 5, 8]

Without Built-in

```
function ascending(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] > arr[j]) {  
        let temp = arr[i];  
        arr[i] = arr[j];  
        arr[j] = temp;  
      }  
    }  
  }  
  
  return arr;  
}
```

```
console.log(ascending([5, 2, 8, 1]));
```

Output: [1, 2, 5, 8]

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

Built-in .sort() → generally **$O(n \log n)$** .

14. Sort Descending ★★

Question: Sort an array in descending order.

Built-in

```
console.log([5, 2, 8, 1].sort((a, b) => b - a));
```

Output: [8, 5, 2, 1]

Without Built-in

```
function descending(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] < arr[j]) {  
        let temp = arr[i];  
        arr[i] = arr[j];  
        arr[j] = temp;  
      }  
    }  
  }  
  
  return arr;  
}
```

```
console.log(descending([5, 2, 8, 1]));
```

Output: [8, 5, 2, 1]

Time: $O(n^2)$ | **Space:** $O(1)$

15. Kth Smallest Element

Question: Find the kth smallest element in an array.

Example: [7, 10, 4, 3, 20, 15], k = 3

Answer = 7

Built-in

```
let arr = [7, 10, 4, 3, 20, 15];
```

```
arr.sort((a, b) => a - b);
```

```
console.log(arr[2]);
```

Output: 7

Time: $O(n \log n)$ | **Space:** $O(n)$

Without Built-in

```
function kthSmallest(arr, k) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] > arr[j]) {  
        let temp = arr[i];  
        arr[i] = arr[j];  
        arr[j] = temp;  
      }  
    }  
  }  
  
  return arr[k - 1];  
}  
  
console.log(kthSmallest([7, 10, 4, 3, 20, 15], 3));
```

Output: 7

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

Quick Select → Average $O(n)$.

16. Kth Largest Element

Question: Find the kth largest element in an array.

Built-in

```
let arr = [7, 10, 4, 3, 20, 15];  
  
arr.sort((a, b) => b - a);  
  
console.log(arr[2]);
```

Output: 10

Time: $O(n \log n)$ | **Space:** $O(n)$

Without Built-in

```
function kthLargest(arr, k) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] < arr[j]) {  
        let temp = arr[i];  
        arr[i] = arr[j];  
        arr[j] = temp;  
      }  
    }  
  }  
  
  return arr[k - 1];  
}  
  
console.log(kthLargest([7, 10, 4, 3, 20, 15], 3));
```

Output: 10

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

Quick Select → Average $O(n)$.

SEARCHING + SORTING — FINAL REVISION

Searching

#	Question	Important
1	Linear Search	★★★★
2	Binary Search	★★★★
3	First Occurrence	★★★★
4	Last Occurrence	★★★★
5	Count Occurrences	★★★
6	Rotated Sorted Array	★★★

Sorting

#	Question	Important
7	Bubble Sort	★★★★
8	Selection Sort	★★★★
9	Insertion Sort	★★★★
10	Merge Sort	★★★★
11	Quick Sort	★★★★
12	Sort 0,1,2	★★★★
13	Ascending Sort	★★★
14	Descending Sort	★★★
15	Kth Smallest	★★★★
16	Kth Largest	★★★★

Bas ye 16 kar lo. Service-based fresher placement ke liye Searching + Sorting ka solid set ho jayega.

Wow now stack queue

Bilkul 🔥 **ab Stack + Queue bhi ek hi lecture mein finish.**

Service-based fresher placement ke liye **8 most important questions**, simple code + output + complexity. No unnecessary hard logic.



STACK — 4 Questions

1. Implement Stack Using Array ★★★★★

Question: Implement a stack using an array.

Built-in

```
let stack = [];
```

```
stack.push(10);
```

```
stack.push(20);
```

```
stack.push(30);
```

```
console.log(stack);
```

Output:

```
[10, 20, 30]
```

Time: $O(1)$ per push

Space: $O(n)$

Without Built-in

```
let stack = [];
```

```
let top = -1;
```

```
function push(value) {  
    top++;  
    stack[top] = value;  
}
```

```
push(10);
```

```
push(20);
```

```
push(30);
```

```
console.log(stack);
```

Output:

```
[10, 20, 30]
```

Time: $O(1)$

Space: $O(n)$

Optimized

Same as without built-in.

2. Push and Pop Operations ★★

Question: Perform push and pop operations on a stack.

Built-in

```
let stack = [];  
  
stack.push(10);  
stack.push(20);  
stack.push(30);  
  
console.log(stack.pop());  
console.log(stack);
```

Output:

```
30  
[10, 20]
```

Time: $O(1)$

Space: $O(n)$

Without Built-in

```
let stack = [];  
let top = -1;  
  
function push(value) {  
    top++;  
    stack[top] = value;  
}  
  
function pop() {  
    if (top === -1) {  
        return "Stack is empty";  
    }  
  
    let value = stack[top];  
    top--;  
  
    return value;  
}  
  
push(10);  
push(20);  
push(30);
```

```
console.log(pop());  
console.log(stack);
```

Output:

```
30  
[10, 20, 30]
```

Time: $O(1)$

Space: $O(n)$

Yahan 30 remove logically ho gaya because top ab 1 hai.

Optimized

Same as without built-in.

3. Check Balanced Parentheses ★★ ★

Question: Check whether the brackets in a string are balanced.

Example:

```
"{[()]}" → true  
"{[(())}" → false
```

Built-in

JavaScript mein iske liye koi direct built-in method nahi hai.

Without Built-in / Optimized

```
function isBalanced(str) {  
    let stack = [];  
  
    for (let ch of str) {  
        if (ch === "(" || ch === "[" || ch === "{") {  
            stack.push(ch);  
        } else {  
            let last = stack.pop();  
  
            if (  
                (ch === ")" && last !== "(") ||  
                (ch === "]" && last !== "[") ||  
                (ch === "}" && last !== "{")  
            ) {  
                return false;  
            }  
        }  
    }  
    return stack.length === 0;  
}
```

```

        (ch === "}" && last !== "{")
    ){
        return false;
    }
}
}

return stack.length === 0;
}

```

```
console.log(isBalanced("{[()]})");
```

Output:

```
true
```

Time: $O(n)$

Space: $O(n)$

Logic yaad rakhna:

Opening bracket → stack mein daalo

Closing bracket → stack se last opening bracket nikalo aur match karo.

4. Reverse a String Using Stack ★★

Question: Reverse a string using a stack.

Built-in

```
console.log("hello".split("").reverse().join(""));
```

Output:

```
olleh
```

Time: $O(n)$

Space: $O(n)$

Without Built-in

```
function reverse(str) {
    let stack = [];

```

```
for (let ch of str) {  
  stack.push(ch);  
}  
  
let result = "";  
  
while (stack.length > 0) {  
  result += stack.pop();  
}  
  
return result;  
}  
  
console.log(reverse("hello"));
```

Output:

olleh

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.



QUEUE — 4 Questions

5. Implement Queue Using Array ★★

Question: Implement a queue using an array.

Queue follows:

FIFO → First In, First Out

Built-in

```
let queue = [];
```



```
queue.push(10);  
queue.push(20);  
queue.push(30);  
  
console.log(queue);
```

Output:

[10, 20, 30]

Time: $O(1)$ for push

Space: $O(n)$

Without Built-in

```
let queue = [];  
let front = 0;  
let rear = -1;  
  
function enqueue(value) {  
  rear++;  
  queue[rear] = value;  
}  
  
enqueue(10);  
enqueue(20);  
enqueue(30);  
  
console.log(queue);
```

Output:

[10, 20, 30]

Time: $O(1)$

Space: $O(n)$

Optimized

Same as without built-in.

6. Enqueue and Dequeue

Question: Perform enqueue and dequeue operations on a queue.

Built-in

```
let queue = [];  
  
queue.push(10);  
queue.push(20);  
queue.push(30);  
  
console.log(queue.shift());  
console.log(queue);
```

Output:

```
10  
[20, 30]
```

Time: $\text{shift()} \rightarrow O(n)$

Space: $O(n)$

Without Built-in / Optimized

```
let queue = [];  
let front = 0;  
let rear = -1;  
  
function enqueue(value) {  
    rear++;  
    queue[rear] = value;  
}  
  
function dequeue() {  
    if (front > rear) {  
        return "Queue is empty";  
    }  
  
    let value = queue[front];  
    front++;  
  
    return value;  
}
```

```
enqueue(10);
enqueue(20);
enqueue(30);
```

```
console.log(dequeue());
console.log(dequeue());
```

Output:

```
10
20
```

Time: $O(1)$

Space: $O(n)$

🔥 **Ye actually optimized hai**, because $\text{shift}()$ $O(n)$ hota hai while our $\text{dequeue}()$ $O(1)$ hai.

7. Implement Circular Queue ★★

Question: Implement a circular queue.

Simple interview-level implementation:

```
class CircularQueue {
  constructor(size) {
    this.queue = new Array(size);
    this.size = size;
    this.front = -1;
    this.rear = -1;
  }

  enqueue(value) {
    if ((this.rear + 1) % this.size === this.front) {
      return "Queue is full";
    }

    if (this.front === -1) {
      this.front = 0;
    }
  }
}
```

```
    this.rear = (this.rear + 1) % this.size;
    this.queue[this.rear] = value;
}

dequeue() {
    if (this.front === -1) {
        return "Queue is empty";
    }

    let value = this.queue[this.front];

    if (this.front === this.rear) {
        this.front = -1;
        this.rear = -1;
    } else {
        this.front = (this.front + 1) % this.size;
    }

    return value;
}
}

let q = new CircularQueue(3);

q.enqueue(10);
q.enqueue(20);
q.enqueue(30);

console.log(q.dequeue());
```

Output:

10

Time: O(1)

Space: O(n)

Built-in: Direct built-in circular queue nahi hai.

8. Stack vs Queue ★★

Question: What is the difference between Stack and Queue?

Stack	Queue
LIFO	FIFO
Last In First Out	First In First Out
push()	enqueue()
pop()	dequeue()
Example: Undo	Example: Printer Queue

Easy trick 🧠

Stack:

```
10 → 20 → 30
      ↑
    POP
```

Queue:

```
10 → 20 → 30
↑
DEQUEUE
```

🔥 STACK + QUEUE — FINAL REVISION

STACK

1. Implement Stack ★★
2. Push & Pop ★★
3. Balanced Parentheses ★★
4. Reverse String Using Stack ★★

QUEUE

5. Implement Queue ★★
6. Enqueue & Dequeue ★★
7. Circular Queue ★★

8. Stack vs Queue ★★★★★

Total = 8 questions.

For your service-based placement prep, #1, #2, #3, #5, #6 and #8 **sabse important** hain.

Recursion acche se dena code smjha bhi dena sath sath main main logic

Bilkul 👍 **Recursion ko ekdum interview-level + easy way** me karte hain. Isme main **code ke saath main logic bhi** bataungi, taaki ratna na pade.

🔥 RECURSION — Most Important Questions

Recursion me bas ek core idea yaad rakho:

Function khud ko call karta hai, aur ek base condition par rukta hai.

Har recursion question me 2 cheezein:

1. **Base Case** → kab rukna hai
 2. **Recursive Call** → function ko dobara call karna
-

1. Print 1 to N ★★★★★

Interview Question:

“Write a recursive function to print numbers from 1 to N.”

```
function printNumbers(n) {  
  if (n === 0) {  
    return;  
  }  
  
  printNumbers(n - 1);  
  console.log(n);  
}
```

```
printNumbers(5);
```

Output:

```
1
2
3
4
5
```

Main Logic 🧠

printNumbers(5) → pehle 4 → 3 → 2 → 1 → 0

0 par recursion ruk jaata hai.

Phir return hote waqt:

```
1
2
3
4
5
```

Time: $O(n)$

Space: $O(n)$

2. Print N to 1 ★★

Interview Question:

“Write a recursive function to print numbers from N to 1.”

```
function printNumbers(n) {
  if (n === 0) {
    return;
  }

  console.log(n);
  printNumbers(n - 1);
}

printNumbers(5);
```

Output:

```
5
4
3
2
1
```

Main Logic 🧠

Isme console.log() **recursive call se pehle** hai.

```
print 5
→ print 4
→ print 3
→ print 2
→ print 1
→ stop
```

Time: $O(n)$

Space: $O(n)$

3. Factorial of a Number ★★

Interview Question:

“Write a recursive function to find the factorial of a number.”

```
function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  }

  return n * factorial(n - 1);
}
```

```
console.log(factorial(5));
```

Output:

```
120
```


Main Logic

Factorial:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Recursion:

```
factorial(5)
= 5 × factorial(4)
= 5 × 4 × factorial(3)
= 5 × 4 × 3 × factorial(2)
= 5 × 4 × 3 × 2 × factorial(1)
```

factorial(1) = 1

Then values multiply hote hue wapas aate hain.

Time: $O(n)$

Space: $O(n)$

4. Sum of First N Natural Numbers

Interview Question:

“Write a recursive function to find the sum of first N natural numbers.”

```
function sum(n) {
  if (n === 0) {
    return 0;
  }

  return n + sum(n - 1);
}
```

```
console.log(sum(5));
```

Output:

15

Main Logic

$$5 + 4 + 3 + 2 + 1$$

Recursion:

```
sum(5)
= 5 + sum(4)
= 5 + 4 + sum(3)
...
```

Finally:

```
5 + 4 + 3 + 2 + 1 = 15
```

Time: $O(n)$

Space: $O(n)$

5. Power of a Number ★★

Interview Question:

“Write a recursive function to calculate x raised to the power n.”

```
function power(x, n) {
  if (n === 0) {
    return 1;
  }

  return x * power(x, n - 1);
}
```

```
console.log(power(2, 5));
```

Output:

```
32
```

Main Logic 🧠

```
 $2^5 = 2 \times 2 \times 2 \times 2 \times 2$ 
```

Recursion:

```
power(2,5)
= 2 × power(2,4)
= 2 × 2 × power(2,3)
...
```

n === 0 par 1 return.

Time: O(n)

Space: O(n)

6. Reverse a String Using Recursion ★★ ★

Interview Question:

“Write a recursive function to reverse a string.”

```
function reverse(str) {  
  if (str === "") {  
    return "";  
  }  
  
  return reverse(str.slice(1)) + str[0];  
}
```

```
console.log(reverse("hello"));
```

Output:

olleh

Main Logic 🧠

String:

hello

Pehla character hatao:

```
hello  
ello  
llo  
lo  
o
```

Phir return hote waqt characters add hote hain:

```
o  
ol  
oll
```

olle
olleh

Time: $O(n^2)$

Space: $O(n^2)$ due to recursive calls/string slicing.

7. Check Palindrome Using Recursion ★★

Interview Question:

“Write a recursive function to check whether a string is a palindrome.”

```
function isPalindrome(str, start, end) {  
  if (start >= end) {  
    return true;  
  }  
  
  if (str[start] !== str[end]) {  
    return false;  
  }  
  
  return isPalindrome(str, start + 1, end - 1);  
}
```

```
console.log(isPalindrome("madam", 0, 4));
```

Output:

true

Main Logic 🧠

Compare:

m a d a m
↑ ↑

m === m ✓

Then:

a d a
↑ ↑

a === a 

Middle tak pahunch gaye → palindrome.

Time: O(n)

Space: O(n)

8. Fibonacci Series

Interview Question:

“Write a recursive function to find the nth Fibonacci number.”

```
function fibonacci(n) {  
  if (n === 0) {  
    return 0;  
  }  
  
  if (n === 1) {  
    return 1;  
  }  
  
  return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
console.log(fibonacci(6));
```

Output:

8

Main Logic 

Fibonacci:

0 1 1 2 3 5 8

Rule:

$F(n) = F(n-1) + F(n-2)$

So:

F(6)
= F(5) + F(4)

Time: $O(2^n)$

Space: $O(n)$

⚠ **Interview me important:** Recursive Fibonacci simple hai but inefficient hai. Optimized Fibonacci usually **iteration/DP** se kiya jata hai.

9. Sum of Array Elements ★★

Interview Question:

“Find the sum of all elements of an array using recursion.”

```
function sumArray(arr, index) {  
  if (index === arr.length) {  
    return 0;  
  }  
  
  return arr[index] + sumArray(arr, index + 1);  
}
```

```
console.log(sumArray([10, 20, 30, 40], 0));
```

Output:

100

Main Logic 🧠

10 + 20 + 30 + 40

Har recursive call next index par jaati hai.

10 + sum(1)
20 + sum(2)
30 + sum(3)
40 + sum(4)

index === arr.length → stop.

Time: $O(n)$

Space: $O(n)$

10. Find Maximum Element in Array ★★

Interview Question:

“Find the maximum element in an array using recursion.”

```
function findMax(arr, index) {  
  if (index === arr.length - 1) {  
    return arr[index];  
  }  
  
  let max = findMax(arr, index + 1);  
  
  if (arr[index] > max) {  
    return arr[index];  
  }  
  
  return max;  
}  
  
console.log(findMax([10, 5, 30, 20], 0));
```

Output:

30

Main Logic 🧠

Pehle last tak jao:

10 → 5 → 30 → 20

Last se maximum compare hota hua wapas aata hai.

20
30 > 20 → 30
5 < 30 → 30
10 < 30 → 30

Time: O(n)

Space: O(n)

Recursion ka Interview Cheat Sheet

Bas ye pattern yaad rakho:

```
function solve(n) {  
  
    // Base Case  
    if (condition) {  
        return;  
    }  
  
    // Recursive Call  
    solve(smallerProblem);  
}
```

Sabse important 7:

Question	Time	Space
Print 1 to N	$O(n)$	$O(n)$
Print N to 1	$O(n)$	$O(n)$
Factorial	$O(n)$	$O(n)$
Sum 1 to N	$O(n)$	$O(n)$
Power	$O(n)$	$O(n)$
Reverse String	$O(n^2)$	$O(n^2)$
Palindrome	$O(n)$	$O(n)$
Fibonacci	$O(2^n)$	$O(n)$
Sum Array	$O(n)$	$O(n)$
Maximum Array	$O(n)$	$O(n)$

Placement ke liye recursion me ye 10 enough hain. Iske baad unnecessarily 20–30 recursion questions karne ki zarurat nahi.

```
if (arr[index] > max) { return arr[index]; Ye smjh nhi aaya }
```

Haan, ye part thoda confusing lag sakta hai. Isko **sirf main logic** se samjho.

Code:


```
let max = findMax(arr, index + 1);
```

```
if (arr[index] > max) {  
  return arr[index];  
}
```

```
return max;
```

Maan lo array hai:

```
[10, 5, 30, 20]
```

Step 1

Sabse pehle recursion last element tak jayegi:

```
10 → 5 → 30 → 20
```

Last par:

```
return 20;
```

Ab max = 20 hai.

Step 2

Ab hum 30 par wapas aaye:

```
if (30 > 20) {  
  return 30;  
}
```

True 

Isliye:

```
return 30;
```

Ab maximum **30** ho gaya.

Step 3

Ab 5 par:

```
if (5 > 30)
```

False ❌

Toh:

```
return max;
```

Matlab **30 return karo.**

Step 4

Ab 10 par:

```
if (10 > 30)
```

False ❌

Toh again:

```
return max;
```

30 return hoga.

Bas ek line yaad rakho 🧠

```
if (arr[index] > max)
```

ka matlab:

“Kya current element, abhi tak mile maximum se bada hai?”

Agar **haan** → current element maximum hai.

Agar **nahi** → purana max hi maximum hai.

So:

```
if (arr[index] > max) {  
    return arr[index];  
}
```

```
return max;
```

= Jo bada hai, usko return karo.

Par hum 30 par vapas aa kaise rhe h

Haan! **Ye recursion ka sabse important concept hai.** ❤️

Tumhara question exactly wahi hai jo recursion samajhne ke liye zaroori hai.

Code ko dekho:

```
function findMax(arr, index) {  
  if (index === arr.length - 1) {  
    return arr[index];  
  }  
  
  let max = findMax(arr, index + 1);  
  
  if (arr[index] > max) {  
    return arr[index];  
  }  
  
  return max;  
}
```

Array:

[10, 5, 30, 20]

Pehle function andar hi andar jaata hai

```
findMax(0) → 10  
↓  
findMax(1) → 5  
↓  
findMax(2) → 30  
↓  
findMax(3) → 20
```

index = 3 par condition true:

```
if (index === arr.length - 1)
```

Toh:

```
return 20;
```

🔥 Ab “wapas” kaise aate hain?

Yahi main point hai.

Jab ye hua:

```
let max = findMax(arr, index + 1);
```

toh **current function wait kar raha tha** ki andar wala function answer de.

Example:

```
findMax(2)
```

ke andar ye tha:

```
let max = findMax(arr, 3);
```

findMax(3) ne 20 return kiya.

Toh ab:

```
let max = 20;
```

Aur hum **findMax(2)** wale function mein wapas aa gaye.

Isliye ab:

```
if (arr[2] > max)
```

Matlab:

```
if (30 > 20)
```

True ✅

Aur:

```
return 30;
```

Ek dum simple visualization 🧠

```
findMax(0) [10] WAITING...
```

↓

```
findMax(1) [5] WAITING...
```

↓

```
findMax(2) [30] WAITING...
```

```

↓
findMax(3) [20] → return 20
↑
|
findMax(2): 30 > 20 → return 30
↑
|
findMax(1): 5 > 30 ❌ → return 30
↑
|
findMax(0): 10 > 30 ❌ → return 30

```

Bas ye line pakad lo:

```
let max = findMax(arr, index + 1);
```

“Pehle aage jao, wahan se answer lekar wapas aao, phir current element se compare karo.”

Isi wajah se hum **30 par wapas aate hain** — actually hum findMax(2) wale pending function call par return hote hain, jahan arr[index] **30** hai.

Nhi aaya smjh ki hum 30 par phuche kaise simple 2 line answer do

findMax(3) se **20 return hua**, isliye recursion automatically **pichle pending call findMax(2) par wapas aayi**.

findMax(2) me index = 2 tha, aur arr[2] = 30 — isliye hum **30 par aaye**.

Implement circular queue wala optimised h kya jo tumne Diya h ?

Haan, **jo Circular Queue wala code maine diya tha, woh optimized hai**.

- enqueue() → **O(1)**
- dequeue() → **O(1)**
- Space → **O(n)**
- Empty cells ko dobara **reuse** karta hai, isi wajah se circular queue useful/optimized hai.

Ab hashmap kre

Bilkul. **HashMap/Hashing** ko bhi same style me karte hain — **sirf most important interview questions**, simple code + output + Time/Space. Isme recursion jaisi detailed explanation nahi, jab tak tum na bolo.

HASHMAP / HASHING — 8 Important Questions

1. Count Frequency of Elements ★★★★★

Interview Question:

“Find the frequency of each element in an array.”

Built-in / HashMap

```
function frequency(arr) {  
  let map = {};  
  
  for (let num of arr) {  
    map[num] = (map[num] || 0) + 1;  
  }  
  
  return map;  
}  
  
console.log(frequency([1, 2, 2, 3, 1, 2]));
```

Output:

```
{ 1: 2, 2: 3, 3: 1 }
```

Time: $O(n)$

Space: $O(n)$

2. Find Duplicate Element ★★★★★

Interview Question:

“Find the first duplicate element in an array.”

```
function findDuplicate(arr) {  
  let map = {};
```

```
for (let num of arr) {
  map[num] = (map[num] || 0) + 1;
}

for (let num of arr) {
  if (map[num] > 1) {
    return num;
  }
}

console.log(findDuplicate([1, 3, 4, 2, 2]));
```

Output:

2

Time: O(n)

Space: O(n)

3. Find First Non-Repeating Element ★★★★★

Interview Question:

“Find the first non-repeating element in an array.”

```
function firstNonRepeating(arr) {
  let map = {};

  for (let num of arr) {
    map[num] = (map[num] || 0) + 1;
  }

  for (let num of arr) {
    if (map[num] === 1) {
      return num;
    }
  }
}

console.log(firstNonRepeating([4, 5, 1, 2, 1, 4]));
```

Output:

Time: $O(n)$

Space: $O(n)$

4. Two Sum ★★

Interview Question:

“Find two elements whose sum is equal to the given target.”

```
function twoSum(arr, target) {  
  let map = {};  
  
  for (let i = 0; i < arr.length; i++) {  
    let needed = target - arr[i];  
  
    if (map[needed] !== undefined) {  
      return [map[needed], i];  
    }  
  
    map[arr[i]] = i;  
  }  
  
  return [];  
}
```

```
console.log(twoSum([2, 7, 11, 15], 9));
```

Output:

[0, 1]

Time: $O(n)$

Space: $O(n)$

5. Check if Two Arrays Have Same Elements ★★

Interview Question:

“Check whether two arrays contain the same elements with the same frequency.”


```
function sameElements(arr1, arr2) {
  let map = {};

  for (let num of arr1) {
    map[num] = (map[num] || 0) + 1;
  }

  for (let num of arr2) {
    if (!map[num]) {
      return false;
    }

    map[num]--;
  }

  return true;
}

console.log(sameElements([1, 2, 2, 3], [2, 3, 2, 1]));
```

Output:

true

Time: O(n)

Space: O(n)

6. Find Majority Element ★★

Interview Question:

“Find the element that appears more than $n/2$ times.”

```
function majorityElement(arr) {
  let map = {};

  for (let num of arr) {
    map[num] = (map[num] || 0) + 1;

    if (map[num] > arr.length / 2) {
      return num;
    }
  }
}
```

```
    return -1;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Output:

2

Time: $O(n)$

Space: $O(n)$

7. Find Intersection Using HashMap ★★

Interview Question:

“Find the common elements between two arrays.”

```
function intersection(arr1, arr2) {
    let map = {};
    let result = [];

    for (let num of arr1) {
        map[num] = true;
    }

    for (let num of arr2) {
        if (map[num]) {
            result.push(num);
            map[num] = false;
        }
    }

    return result;
}

console.log(intersection([1, 2, 3, 4], [2, 4, 6]));
```

Output:

[2, 4]

Time: $O(n + m)$

Space: $O(n)$

8. Count Pairs With Given Sum ★★★★★

Interview Question:

“Count the number of pairs whose sum is equal to the given target.”

```
function countPairs(arr, target) {  
  let map = {};  
  let count = 0;  
  
  for (let num of arr) {  
    let needed = target - num;  
  
    if (map[needed]) {  
      count += map[needed];  
    }  
  
    map[num] = (map[num] || 0) + 1;  
  }  
  
  return count;  
}  
  
console.log(countPairs([1, 5, 7, -1, 5], 6));
```

Output:

3

Time: $O(n)$

Space: $O(n)$

★ **HashMap me sabse important:**

Frequency → Duplicate → First Non-Repeating → Two Sum → Majority → Pair Sum

Ye 8 kar logi to **fresher placement ke HashMap/Hashing portion ke liye kaafi strong coverage** hai.

Hashmat mein building use hota hai kya without Nahin hote

Haan, **HashMap/Hashing** mein usually **built-in Map / object** use hota hai.

Agar interviewer bole “**without HashMap**”, tab simple nested loops se kar sakte ho, but woh generally **$O(n^2)$** ho jayega.

👉 **Placement ke liye HashMap wale questions mein Map/object use karna hi important hai.**

Tumhe abhi separate “without HashMap” versions karne ki zarurat nahi hai.

Ye sb optimised h kya

Haan, **jo 8 HashMap questions diye hain, unke solutions already optimized hain** — mostly **$O(n)$** time.

Bas ek clarification:

- Frequency → **$O(n)$** ✓
- Duplicate → **$O(n)$** ✓
- First non-repeating → **$O(n)$** ✓
- Two Sum → **$O(n)$** ✓
- Same elements → **$O(n)$** ✓
- Majority → **$O(n)$** ✓
- Intersection → **$O(n + m)$** ✓
- Count pairs → **$O(n)$** ✓

Lekin Majority Element ka aur bhi optimized space version (Boyer-Moore) hota hai: $O(n)$ time, $O(1)$ space. Interview me optimization specifically poochhe tab woh karna.

Btade bhai tu vo bhi

Haan bhai 😭 **Majority Element ka actual optimized version ye hai — Boyer-Moore Voting Algorithm.**

Interview Question:

“Find the majority element in an array in $O(n)$ time and $O(1)$ space.”

```
function majorityElement(arr) {
  let candidate = 0;
  let count = 0;

  for (let num of arr) {
    if (count === 0) {
      candidate = num;
    }

    if (num === candidate) {
      count++;
    } else {
      count--;
    }
  }

  return candidate;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
```

Output:

2

Main logic 🧠

- candidate = possible majority element
- Same element mile → count++
- Different element mile → count--
- count === 0 → new candidate

Time: $O(n)$ ✓

Space: $O(1)$ ✓

👉 **HashMap wala:** $O(n)$ time, $O(n)$ space

👉 **Boyer-Moore:** $O(n)$ time, **$O(1)$ space — optimized** ★

Hashmap h kya vaise ek simple definition bta means iska code kyu alag h

Haan, bilkul simple:

HashMap ek data structure hai jo data ko key : value pair me store karta hai, taaki kisi value ko jaldi find/count kar sakein.

Example:

```
let map = {};  
  
map["apple"] = 3;  
map["banana"] = 2;  
  
console.log(map);
```

Output:

```
{ apple: 3, banana: 2 }
```

Code alag kyu lagta hai?

Kyunki normal array me hum **index** se data rakhte hain:

```
arr[0] = 10  
arr[1] = 20
```

HashMap me hum **key** se data rakhte hain:

```
map["apple"] = 3  
map["banana"] = 2
```

Isliye frequency wale questions me:

```
map[num] = (map[num] || 0) + 1;
```

ka simple meaning hai:

“Is number ko map me count karo; pehli baar hai to 1, warna uska count +1.”

Bas **HashMap = key ke through data store/find karna** yaad rakho.

But ye jo optimised majority element wala code h usme to ab map nhi h

Exactly! Us **optimized majority-element code** mein **HashMap nahi hai**.

Kyunki **Boyer-Moore HashMap** ki jagah ek **different technique** use karta hai.

- **HashMap approach:** number → count store karta hai → $O(n)$ space

- **Boyer-Moore:** sirf candidate aur count rakhta hai → $O(1)$ space

Matlab **HashMap** ek technique hai, aur **Boyer-Moore** doosri optimized technique.

Interview me agar bole “**use HashMap**” → HashMap wala code.

Agar bole “ **$O(1)$ extra space me optimize karo**” → Boyer-Moore wala.

Now ab kiske questions kre

Ab **HashMap** ke baad **Basic Math** karte hain.

Tumhari placement priority ke hisaab se sequence:

1. ☒ Arrays
2. ☒ Strings
3. ☒ Searching
4. ☒ Sorting
5. ☒ HashMap
6.  **Basic Math**
7. Linked List
8. Stack & Queue ☒
9. Recursion
10. Patterns

Basic Math me 15 important questions karenge — reverse number, palindrome number, prime, factorial, Armstrong, GCD/LCM, Fibonacci etc.

Chalo **Basic Math** start karte hain.

LinkedIn list krte h start saare ek hi lecture me

Haan bhai, **Linked List** ke saare important questions ek hi lecture mein karte hain. Simple code, interview-level, aur har question ke **Time** + **Space** ke saath.

LINKED LIST — 10 Most Important Questions

Pehle ek basic Node structure:

```
class Node {  
  constructor(data) {  
    this.data = data;  
    this.next = null;  
  }  
}
```

1. Create / Traverse a Linked List ★★ ★

Interview Question:

“Create a linked list and print all its elements.”

```
let head = new Node(10);  
  
head.next = new Node(20);  
head.next.next = new Node(30);  
  
let current = head;  
  
while (current !== null) {  
  console.log(current.data);  
  current = current.next;  
}
```

Output:

```
10  
20  
30
```

Time: $O(n)$

Space: $O(n)$

2. Insert at Beginning ★★ ★

Interview Question:

“Insert a node at the beginning of a linked list.”

```
function insertAtBeginning(head, value) {  
  let newNode = new Node(value);
```



```
    newNode.next = head;
    head = newNode;

    return head;
}

let head = new Node(20);
head.next = new Node(30);

head = insertAtBeginning(head, 10);

let current = head;

while (current !== null) {
    console.log(current.data);
    current = current.next;
}
```

Output:

```
10
20
30
```

Time: $O(1)$

Space: $O(1)$

3. Insert at End ★★

Interview Question:

“Insert a node at the end of a linked list.”

```
function insertAtEnd(head, value) {
    let newNode = new Node(value);

    if (head === null) {
        return newNode;
    }

    let current = head;

    while (current.next !== null) {
```

```

    current = current.next;
}

current.next = newNode;

return head;
}

let head = new Node(10);
head.next = new Node(20);

head = insertAtEnd(head, 30);

let current = head;

while (current !== null) {
    console.log(current.data);
    current = current.next;
}

```

Output:

```

10
20
30

```

Time: $O(n)$

Space: $O(1)$

4. Delete a Node ★★

Interview Question:

“Delete a node with a given value from a linked list.”

```

function deleteNode(head, value) {
    if (head === null) {
        return head;
    }

    if (head.data === value) {
        return head.next;
    }
}

```

```
let current = head;

while (current.next !== null) {
  if (current.next.data === value) {
    current.next = current.next.next;
    break;
  }

  current = current.next;
}

return head;
}
```

Example:

10 → 20 → 30

Delete 20:

10 → 30

Time: O(n)

Space: O(1)

5. Search an Element ★★

Interview Question:

“Search for an element in a linked list.”

```
function search(head, target) {
  let current = head;

  while (current !== null) {
    if (current.data === target) {
      return true;
    }

    current = current.next;
  }
}
```

```
    return false;
}

console.log(search(head, 20));
```

Output:

true

Time: $O(n)$

Space: $O(1)$

6. Reverse a Linked List ★★

Interview Question:

“Reverse a singly linked list.”

```
function reverse(head) {
    let previous = null;
    let current = head;

    while (current !== null) {
        let next = current.next;

        current.next = previous;

        previous = current;
        current = next;
    }

    return previous;
}
```

Example:

10 → 20 → 30

becomes:

30 → 20 → 10

Time: $O(n)$

Space: $O(1)$ ★

7. Find Middle of Linked List ★★☆☆

Interview Question:

“Find the middle element of a linked list.”

Optimized — Slow & Fast Pointer

```
function findMiddle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {  
    slow = slow.next;  
    fast = fast.next.next;  
  }  
  
  return slow.data;  
}
```

For:

10 → 20 → 30 → 40 → 50

Output:

30

Time: O(n)

Space: O(1) ★

8. Detect Cycle / Loop ★★☆☆

Interview Question:

“Detect whether a linked list contains a cycle.”

Optimized — Floyd’s Algorithm

```
function hasCycle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {
```

```
    slow = slow.next;
    fast = fast.next.next;

    if (slow === fast) {
        return true;
    }
}

return false;
}
```

Output:

true / false

Time: $O(n)$

Space: $O(1)$ ★

9. Count Number of Nodes ★★

Interview Question:

“Count the number of nodes in a linked list.”

```
function countNodes(head) {
    let count = 0;
    let current = head;

    while (current !== null) {
        count++;
        current = current.next;
    }

    return count;
}

console.log(countNodes(head));
```

Output:

3

Time: $O(n)$

Space: $O(1)$

10. Remove Duplicates ★★

Interview Question:

“Remove duplicate elements from a sorted linked list.”

```
function removeDuplicates(head) {  
    let current = head;  
  
    while (current !== null && current.next !== null) {  
        if (current.data === current.next.data) {  
            current.next = current.next.next;  
        } else {  
            current = current.next;  
        }  
    }  
  
    return head;  
}
```

Example:

10 → 20 → 20 → 30 → 30

Output:

10 → 20 → 30

Time: $O(n)$

Space: $O(1)$ ★

Linked List — Placement Priority

Question	Importance
Traverse	★★★★
Insert Beginning	★★★★
Insert End	★★★★
Delete Node	★★★★
Search	★★★★

Question	Importance
Reverse Linked List	★★★★🔥
Find Middle	★★★★🔥
Detect Cycle	★★★★🔥
Count Nodes	★★★
Remove Duplicates	★★★

🔥 **Sabse zyada pakke karna:**

Reverse → Middle → Cycle → Insert/Delete → Search/Traversal

Ye **10 questions Linked List ke liye enough hain** tumhari fresher placement prep ke liye.

Iska kuch built in vgera optimize vgera nhi h kya

Haan, but **Linked List me arrays/strings ki tarah built-in methods ka direct use nahi hota.**

- **Built-in:** JavaScript me linked list ke liye direct built-in insert, delete, reverse, middle methods nahi hain. ❌
- **Without built-in:** Jo code maine diya wahi standard manual solution hai. ✅
- **Optimized:** Jahan genuinely optimization possible hai, woh already diya hai:
 - o Insert beginning → $O(1)$
 - o Reverse ★(n), $O(1)$ □
 - o Middle ★(n), $O(1)$ □
 - o Cycle ★(n), $O(1)$ □
 - o Remove duplicates → $O(n)$, $O(1)$

So **Linked List ke liye separate built-in versions karne ki zarurat nahi hai.** Jo codes diye hain wahi interview ke liye optimized/standard hain.

Without optimize kyu nhi diye

Haan, meri galti 🤔 Tumne start se bola tha ki **Built-in → Without built-in → Optimized** chahiye.

Linked List me **built-in direct method nahi hota**, isliye:

- **Built-in:** ❌ Direct built-in available nahi
- **Without optimized:** Basic/manual solution
- **Optimized:** Better standard solution, **sirf jahan actual improvement ho**

Jo maine directly optimized de diye, isliye distinction nahi dikhaya.

Aage se har topic me ye format strictly follow karunga.

Aur Linked List ke 10 questions ko bhi isi format me chaho to main **without-optimized + optimized** versions de sakta hoon.

Ha de do

Bilkul. **Linked List ke 10 questions ko proper format me** de raha hoon:

Built-in → Without Optimized → Optimized

Jahan built-in available nahi hai, clearly bata raha hoon.

LINKED LIST — Proper Interview Format

Basic Node:

```
class Node {  
    constructor(data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

1. Traverse Linked List ★★★★★

Question: “Print all elements of a linked list.”

Built-in

❌ Linked List ke liye direct built-in traversal method nahi hai.

Without Optimized

```
function printList(head) {  
  let current = head;  
  
  while (current !== null) {  
    console.log(current.data);  
    current = current.next;  
  }  
}
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without optimized — $O(n)$ already best possible hai.

2. Insert at Beginning

Question: “Insert a node at the beginning.”

Built-in

 Direct built-in method nahi hai.

Without Optimized

```
function insertAtBeginning(head, value) {  
  let newNode = new Node(value);  
  
  newNode.next = head;  
  head = newNode;  
  
  return head;  
}
```

Time: $O(1)$

Space: $O(1)$

Optimized

Same as without optimized 

Because beginning par insertion already **O(1)** hai.

3. Insert at End ★★

Question: “Insert a node at the end.”

Built-in

✗ Direct built-in method nahi hai.

Without Optimized

```
function insertAtEnd(head, value) {  
  let newNode = new Node(value);  
  
  if (head === null) {  
    return newNode;  
  }  
  
  let current = head;  
  
  while (current.next !== null) {  
    current = current.next;  
  }  
  
  current.next = newNode;  
  
  return head;  
}
```

Time: O(n)

Space: O(1)

Optimized ★

Agar **tail pointer** maintain karte hain:

```
function insertAtEnd(tail, value) {  
  let newNode = new Node(value);  
  
  tail.next = newNode;
```

```
    return newNode;
}
```

Time: $O(1)$ ★

Space: $O(1)$

👉 Actual optimization yahan **tail pointer maintain** karna hai.

4. Delete a Node ★★

Question: “Delete a node with a given value.”

Built-in

✗ Direct built-in method nahi hai.

Without Optimized

```
function deleteNode(head, value) {
    if (head === null) {
        return head;
    }

    if (head.data === value) {
        return head.next;
    }

    let current = head;

    while (current.next !== null) {
        if (current.next.data === value) {
            current.next = current.next.next;
            break;
        }

        current = current.next;
    }

    return head;
}
```

Time: $O(n)$
Space: $O(1)$

Optimized

Same as without optimized for deleting **by value**.

$O(n)$ because node search karna padta hai.

5. Search Element ★★

Question: “Search an element in a linked list.”

Built-in

✗ Direct built-in search method nahi hai.

Without Optimized

```
function search(head, target) {  
  let current = head;  
  
  while (current !== null) {  
    if (current.data === target) {  
      return true;  
    }  
  
    current = current.next;  
  }  
  
  return false;  
}
```

Time: $O(n)$
Space: $O(1)$

Optimized

Same as without optimized.

Linked List me generally search **$O(n)$** se better nahi ho sakta.

6. Reverse Linked List

Question: “Reverse a singly linked list.”

Built-in

✗ Direct built-in reverse() for Linked List nahi hai.

Without Optimized

Extra array use karke:

```
function reverse(head) {  
  let values = [];  
  let current = head;  
  
  while (current !== null) {  
    values.push(current.data);  
    current = current.next;  
  }  
  
  current = head;  
  let i = values.length - 1;  
  
  while (current !== null) {  
    current.data = values[i];  
    i--;  
    current = current.next;  
  }  
  
  return head;  
}
```

Time: $O(n)$

Space: $O(n)$

Optimized 

Pointers ko reverse karo:

```
function reverse(head) {  
  let previous = null;  
  let current = head;
```

```
while (current !== null) {  
  let next = current.next;  
  
  current.next = previous;  
  
  previous = current;  
  current = next;  
}  
  
return previous;  
}
```

Time: $O(n)$

Space: $O(1)$ ★

👉 Ye interview me preferred solution hai.

7. Find Middle Element ★★☆☆🔥

Question: “Find the middle element of a linked list.”

Built-in

❌ Direct built-in method nahi hai.

Without Optimized

Pehle length count karo:

```
function findMiddle(head) {  
  let length = 0;  
  let current = head;  
  
  while (current !== null) {  
    length++;  
    current = current.next;  
  }  
  
  current = head;  
  
  for (let i = 0; i < Math.floor(length / 2); i++) {  
    current = current.next;  
  }  
}
```

```
}  
  
return current.data;  
}
```

Time: $O(n)$

Space: $O(1)$

Optimized ★

Slow + Fast pointer:

```
function findMiddle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {  
    slow = slow.next;  
    fast = fast.next.next;  
  }  
  
  return slow.data;  
}
```

Time: $O(n)$

Space: $O(1)$

👉 Time complexity same hai, but **one traversal** me middle mil jaata hai. ★

8. Detect Cycle ★★☆☆🔥

Question: “Check whether a linked list contains a cycle.”

Built-in

❌ Direct built-in method nahi hai.

Without Optimized — HashMap/Set

```
function hasCycle(head) {  
  let visited = new Set();  
  let current = head;
```



```
while (current !== null) {  
  if (visited.has(current)) {  
    return true;  
  }  
  
  visited.add(current);  
  current = current.next;  
}  
  
return false;  
}
```

Time: $O(n)$

Space: $O(n)$

Optimized ★

Floyd's slow-fast pointer:

```
function hasCycle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {  
    slow = slow.next;  
    fast = fast.next.next;  
  
    if (slow === fast) {  
      return true;  
    }  
  }  
  
  return false;  
}
```

Time: $O(n)$

Space: $O(1)$ ★

9. Count Nodes ★★

Question: “Count the number of nodes.”

Built-in

✗ Direct built-in method nahi hai.

Without Optimized

```
function countNodes(head) {  
  let count = 0;  
  let current = head;  
  
  while (current !== null) {  
    count++;  
    current = current.next;  
  }  
  
  return count;  
}
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without optimized ✓

$O(n)$ se better possible nahi hai because har node count karni padegi.

10. Remove Duplicates ★★

Question: “Remove duplicates from a sorted linked list.”

Built-in

✗ Direct built-in method nahi hai.

Without Optimized

```
function removeDuplicates(head) {  
  let current = head;  
  
  while (current !== null && current.next !== null) {  
    if (current.data === current.next.data) {  
      current.next = current.next.next;  
    }  
  }  
}
```

```
    } else {  
        current = current.next;  
    }  
}  
  
return head;  
}
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without optimized ✓

Already $O(n)$ time and $O(1)$ space.

🔥 Final: Kya Actually Yaad Karna Hai?

★ Must-do optimized questions:

1. Insert at End → Tail Pointer → $O(1)$
2. Reverse → Pointer method → $O(n)$, $O(1)$
3. Middle → Slow/Fast → $O(n)$, $O(1)$
4. Cycle → Floyd → $O(n)$, $O(1)$

Baaki questions ke **without-optimized solutions hi already best hain.**

So interview me agar interviewer bole “Can you optimize this?”, in 4 concepts ko identify karna important hai.

Searching ke 5th question count occurrence me tumne ye kiya h Optimized Sorted array mein: Count = Last Occurrence – First Occurrence + 1 Time: $O(\log n)$ | Space: $O(1)$ code kyu nhi diya

Haan, **code dena chahiye tha.** Meri mistake. 😞

5. Count Occurrences — Optimized ★★★★★

Question:

“Find the number of occurrences of a target element in a sorted array.”

Example:

[1, 2, 2, 2, 3, 4]

Target = 2

First occurrence = 1

Last occurrence = 3

So:

Count = 3 - 1 + 1 = 3

Optimized Code — Binary Search

```
function firstOccurrence(arr, target) {
  let start = 0;
  let end = arr.length - 1;
  let answer = -1;

  while (start <= end) {
    let middle = Math.floor((start + end) / 2);

    if (arr[middle] === target) {
      answer = middle;
      end = middle - 1;
    } else if (arr[middle] < target) {
      start = middle + 1;
    } else {
      end = middle - 1;
    }
  }

  return answer;
}

function lastOccurrence(arr, target) {
  let start = 0;
  let end = arr.length - 1;
  let answer = -1;

  while (start <= end) {
    let middle = Math.floor((start + end) / 2);
```

```

    if (arr[middle] === target) {
        answer = middle;
        start = middle + 1;
    } else if (arr[middle] < target) {
        start = middle + 1;
    } else {
        end = middle - 1;
    }
}

return answer;
}

function countOccurrences(arr, target) {
    let first = firstOccurrence(arr, target);
    let last = lastOccurrence(arr, target);

    if (first === -1) {
        return 0;
    }

    return last - first + 1;
}

console.log(countOccurrences([1, 2, 2, 2, 3, 4], 2));

```

Output

3

Time: $O(\log n)$

Space: $O(1)$ ★

Main logic: Pehle **first position**, phir **last position** find karo → $\text{last} - \text{first} + 1$.

Aage se jo bhi **optimized** bolungi, uska **actual code bhi dung**i.

But sorting me tumne sort 0 1 2 wale ka optimize nhi diya aur nahi kth largest
smallest ka

Haan bhai, **ye dono ke optimized codes bhi dene chahiye the**. Correct kar dete hain.

12. Sort 0s, 1s and 2s — Optimized ★★ ★

Question:

“Sort an array containing only 0, 1 and 2 in $O(n)$ time and $O(1)$ space.”

Optimized — Dutch National Flag

```
function sort012(arr) {  
  let low = 0;  
  let mid = 0;  
  let high = arr.length - 1;  
  
  while (mid <= high) {  
    if (arr[mid] === 0) {  
      [arr[low], arr[mid]] = [arr[mid], arr[low]];  
      low++;  
      mid++;  
    }  
    else if (arr[mid] === 1) {  
      mid++;  
    }  
    else {  
      [arr[mid], arr[high]] = [arr[high], arr[mid]];  
      high--;  
    }  
  }  
  
  return arr;  
}  
  
console.log(sort012([2, 0, 2, 1, 1, 0]));
```

Output:

[0, 0, 1, 1, 2, 2]

Time: $O(n)$

Space: $O(1)$ ★

15. Kth Smallest Element ★★

Question:

“Find the kth smallest element in an array.”

Optimized — Quick Select

```
function kthSmallest(arr, k) {  
  let left = 0;  
  let right = arr.length - 1;  
  
  while (left <= right) {  
    let pivot = arr[right];  
    let index = left;  
  
    for (let i = left; i < right; i++) {  
      if (arr[i] < pivot) {  
        [arr[i], arr[index]] = [arr[index], arr[i]];  
        index++;  
      }  
    }  
  
    [arr[index], arr[right]] = [arr[right], arr[index]];  
  
    if (index === k - 1) {  
      return arr[index];  
    }  
  
    if (index < k - 1) {  
      left = index + 1;  
    } else {  
      right = index - 1;  
    }  
  }  
}  
  
console.log(kthSmallest([7, 10, 4, 3, 20, 15], 3));
```

Output:

7

Average Time: $O(n)$

Worst Time: $O(n^2)$

Space: $O(1)$

16. Kth Largest Element ★★ ★

Question:

“Find the kth largest element in an array.”

Same Quick Select logic, bas **largest ke according partition:**

```
function kthLargest(arr, k) {
  let left = 0;
  let right = arr.length - 1;

  while (left <= right) {
    let pivot = arr[right];
    let index = left;

    for (let i = left; i < right; i++) {
      if (arr[i] > pivot) {
        [arr[i], arr[index]] = [arr[index], arr[i]];
        index++;
      }
    }

    [arr[index], arr[right]] = [arr[right], arr[index]];

    if (index === k - 1) {
      return arr[index];
    }

    if (index < k - 1) {
      left = index + 1;
    } else {
      right = index - 1;
    }
  }
}
```



```
console.log(kthLargest([7, 10, 4, 3, 20, 15], 3));
```

Output:

10

Average Time: $O(n)$

Worst Time: $O(n^2)$

Space: $O(1)$

🔥 Placement ke liye yaad rakho

- 0,1,2 → Dutch National Flag → $O(n)$, $O(1)$
- Kth Smallest → Quick Select → Average $O(n)$
- Kth Largest → Quick Select → Average $O(n)$

Aur haan, **Kth wale Quick Select thode advanced hain**. Agar interviewer optimization specifically na pooche, to `sort()` wala solution fresher interview me kaafi hai.

Array dedo ab

Haan, **Arrays ko ek baar properly interview-level revise** kar lete hain. Tumhare liye **25 most important questions** enough hain.

🔥 ARRAY — 25 Important Interview Questions

🟢 Level 1 — Must Do

1. Find the largest element in an array. ★★★★★
2. Find the smallest element in an array. ★★★★★
3. Find the second largest element in an array. ★★★★★
4. Find the second smallest element in an array. ★★★★★
5. Reverse an array. ★★★★★
6. Check if an array is sorted. ★★★★★

7. Remove duplicates from a sorted array. ★★★★★

8. Count the frequency of each element. ★★★★★

● Level 2 — Very Important

9. Find the missing number in an array. ★★★★★

10. Find the duplicate element in an array. ★★★★★

11. Find all duplicate elements in an array. ★★★★★

12. Find the intersection of two arrays. ★★★★★

13. Find the union of two arrays. ★★★★★

14. Move all zeros to the end of an array. ★★★★★

15. Move all negative numbers to one side of an array. ★★

16. Find a pair whose sum is equal to a given number. ★★★★★

17. Find the majority element in an array. ★★★★★

18. Find a pair with a given difference. ★★

19. Find the maximum number of consecutive 1s. ★★

● Level 3 — Important Interview Problems

20. Find the maximum subarray sum. ★★★★★

21. Find the maximum profit from buying and selling a stock. ★★★★★

22. Find the subarray with a given sum. ★★★★★

23. Find the longest subarray with a given sum. ★★★★★

24. Rotate an array by K positions. ★★★★★

25. Find the kth largest/smallest element in an array. ★★★★★

📌 Hum har question mein ye format rakhenge:

Question → Built-in allowed → Without built-in → Optimized (agar genuinely different ho) → Output → Time + Space Complexity

Aur **unnecessarily difficult code nahi**. Simple interview-level JavaScript hi rakhenge.

Array ke 1–19 hum already kar chuke hain; ab 20 se continue kar sakte hain.

Shuru se shuru kro saare dedo ek hi lecture me

Bilkul. **Array ka complete lecture — starting se 25 questions**, exactly interview style mein.

Har question mein **Built-in → Without Built-in → Optimized (jab genuinely different ho) → Output → Complexity**.

Code simple JavaScript mein rakha hai.

🔥 ARRAY — COMPLETE INTERVIEW LECTURE

1. Find the Largest Element ★★★★★

Built-in allowed

```
function largest(arr) {  
  return Math.max(...arr);  
}
```

```
console.log(largest([10, 25, 7, 40, 15]));  
// 40
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function largest(arr) {  
  let max = arr[0];
```

```
for (let i = 1; i < arr.length; i++) {  
  if (arr[i] > max) {  
    max = arr[i];  
  }  
}  
  
return max;  
}  
  
console.log(largest([10, 25, 7, 40, 15]));  
// 40
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in — $O(n)$, $O(1)$.

2. Find the Smallest Element ★★

Built-in

```
function smallest(arr) {  
  return Math.min(...arr);  
}  
  
console.log(smallest([10, 25, 7, 40, 15]));  
// 7
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function smallest(arr) {  
  let min = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] < min) {  
      min = arr[i];  
    }  
  }  
}
```

```

    }

    return min;
}

console.log(smallest([10, 25, 7, 40, 15]));
// 7

```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

3. Find the Second Largest Element ★★

Built-in

```

function secondLargest(arr) {
    let sorted = [...arr].sort((a, b) => b - a);
    return sorted[1];
}

console.log(secondLargest([10, 25, 7, 40, 15]));
// 25

```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in

```

function secondLargest(arr) {
    let largest = -Infinity;
    let second = -Infinity;

    for (let num of arr) {
        if (num > largest) {
            second = largest;
            largest = num;
        } else if (num > second && num !== largest) {
            second = num;
        }
    }
}

```

```

    }
  }

  return second;
}

console.log(secondLargest([10, 25, 7, 40, 15]));
// 25

```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in — already $O(n)$.

4. Find the Second Smallest Element ★★

Built-in

```

function secondSmallest(arr) {
  let sorted = [...arr].sort((a, b) => a - b);
  return sorted[1];
}

console.log(secondSmallest([10, 25, 7, 40, 15]));
// 10

```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in

```

function secondSmallest(arr) {
  let smallest = Infinity;
  let second = Infinity;

  for (let num of arr) {
    if (num < smallest) {
      second = smallest;
      smallest = num;
    } else if (num < second && num !== smallest) {

```

```
        second = num;
    }
}

return second;
}

console.log(secondSmallest([10, 25, 7, 40, 15]));
// 10
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

5. Reverse an Array ★★

Built-in

```
function reverse(arr) {
    return arr.reverse();
}

console.log(reverse([1, 2, 3, 4, 5]));
// [5, 4, 3, 2, 1]
```

Time: $O(n)$

Space: $O(1)$

Without built-in

```
function reverse(arr) {
    let result = [];

    for (let i = arr.length - 1; i >= 0; i--) {
        result.push(arr[i]);
    }

    return result;
}
```

```
console.log(reverse([1, 2, 3, 4, 5]));  
// [5, 4, 3, 2, 1]
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function reverse(arr) {  
  let start = 0;  
  let end = arr.length - 1;  
  
  while (start < end) {  
    [arr[start], arr[end]] = [arr[end], arr[start]];  
    start++;  
    end--;  
  }  
  
  return arr;  
}
```

```
console.log(reverse([1, 2, 3, 4, 5]));  
// [5, 4, 3, 2, 1]
```

Time: $O(n)$

Space: $O(1)$

6. Check if Array is Sorted ★★

Built-in

```
function isSorted(arr) {  
  return arr.every((num, i) => i === 0 || arr[i - 1] <= num);  
}
```

```
console.log(isSorted([1, 2, 3, 4, 5]));  
// true
```

Time: $O(n)$

Space: $O(1)$

Without built-in

```
function isSorted(arr) {  
  for (let i = 1; i < arr.length; i++) {  
    if (arr[i] < arr[i - 1]) {  
      return false;  
    }  
  }  
  
  return true;  
}
```

```
console.log(isSorted([1, 2, 3, 4, 5]));  
// true
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in.

7. Remove Duplicates from Sorted Array ★★

Built-in

```
function removeDuplicates(arr) {  
  return [...new Set(arr)];  
}
```

```
console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));  
// [1, 2, 3, 4]
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function removeDuplicates(arr) {  
  let result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    if (i === 0 || arr[i] !== arr[i - 1]) {
```

```
    result.push(arr[i]);
  }
}

return result;
}

console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
// [1, 2, 3, 4]
```

Time: $O(n)$

Space: $O(n)$

Optimized

```
function removeDuplicates(arr) {
  if (arr.length === 0) return 0;

  let index = 1;

  for (let i = 1; i < arr.length; i++) {
    if (arr[i] !== arr[i - 1]) {
      arr[index] = arr[i];
      index++;
    }
  }

  return arr.slice(0, index);
}

console.log(removeDuplicates([1, 1, 2, 2, 3, 4, 4]));
// [1, 2, 3, 4]
```

Time: $O(n)$

Space: $O(1)$ extra

8. Count Frequency of Elements

Built-in

```
function frequency(arr) {
  return arr.reduce((map, num) => {
```

```
    map[num] = (map[num] || 0) + 1;
    return map;
  }, {});
}

console.log(frequency([1, 2, 2, 3, 1, 2]));
// {1: 2, 2: 3, 3: 1}
```

Time: $O(n)$

Space: $O(n)$

Without built-in

```
function frequency(arr) {
  let map = {};

  for (let num of arr) {
    if (map[num]) {
      map[num]++;
    } else {
      map[num] = 1;
    }
  }

  return map;
}

console.log(frequency([1, 2, 2, 3, 1, 2]));
// {1: 2, 2: 3, 3: 1}
```

Time: $O(n)$

Space: $O(n)$

Optimized

Same as without built-in.

9. Find Missing Number

Array contains numbers from 1 to n, with one number missing.

Built-in

```
function missingNumber(arr, n) {  
  let expected = Array.from({ length: n }, (_, i) => i + 1);  
  
  return expected.find(num => !arr.includes(num));  
}  
  
console.log(missingNumber([1, 2, 4, 5], 5));  
// 3
```

Time: $O(n^2)$

Space: $O(n)$

Without built-in

```
function missingNumber(arr, n) {  
  for (let num = 1; num <= n; num++) {  
    let found = false;  
  
    for (let i = 0; i < arr.length; i++) {  
      if (arr[i] === num) {  
        found = true;  
        break;  
      }  
    }  
  
    if (!found) {  
      return num;  
    }  
  }  
}  
  
console.log(missingNumber([1, 2, 4, 5], 5));  
// 3
```

Time: $O(n^2)$

Space: $O(1)$

Optimized — Sum Formula

```
function missingNumber(arr, n) {  
  let total = n * (n + 1) / 2;  
  let sum = 0;
```

```
    for (let num of arr) {
        sum += num;
    }

    return total - sum;
}

console.log(missingNumber([1, 2, 4, 5], 5));
// 3
```

Time: $O(n)$
Space: $O(1)$

10. Find Duplicate Element ★★

Built-in

```
function findDuplicate(arr) {
    return arr.find((num, i) => arr.indexOf(num) !== i);
}

console.log(findDuplicate([1, 3, 4, 2, 2]));
// 2
```

Time: $O(n^2)$
Space: $O(1)$

Without built-in

```
function findDuplicate(arr) {
    let count = {};

    for (let num of arr) {
        count[num] = (count[num] || 0) + 1;
    }

    for (let num of arr) {
        if (count[num] > 1) {
            return num;
        }
    }
}
```

```
console.log(findDuplicate([1, 3, 4, 2, 2]));  
// 2
```

Time: $O(n)$

Space: $O(n)$

Optimized

For fresher interview, **HashMap solution is enough**. No need for complicated cycle detection.

11. Find All Duplicate Elements ★★

```
function findDuplicates(arr) {  
  let count = {};  
  let result = [];  
  
  for (let num of arr) {  
    count[num] = (count[num] || 0) + 1;  
  }  
  
  for (let num in count) {  
    if (count[num] > 1) {  
      result.push(Number(num));  
    }  
  }  
  
  return result;  
}  
  
console.log(findDuplicates([1, 2, 3, 2, 4, 1, 5]));  
// [1, 2]
```

Time: $O(n)$

Space: $O(n)$

Optimized: Same.

12. Find Intersection of Two Arrays ★★

Built-in

```
function intersection(arr1, arr2) {  
  return [...new Set(arr1.filter(num => arr2.includes(num)))];  
}
```

```
console.log(intersection([1, 2, 3, 4], [2, 4, 6]));  
// [2, 4]
```

Time: $O(n \times m)$

Space: $O(n)$

Without built-in / HashMap

```
function intersection(arr1, arr2) {  
  let map = {};  
  let result = [];  
  
  for (let num of arr1) {  
    map[num] = true;  
  }  
  
  for (let num of arr2) {  
    if (map[num]) {  
      result.push(num);  
      map[num] = false;  
    }  
  }  
  
  return result;  
}
```

```
console.log(intersection([1, 2, 3, 4], [2, 4, 6]));  
// [2, 4]
```

Time: $O(n + m)$

Space: $O(n)$

Optimized

Same as HashMap solution.

13. Find Union of Two Arrays ★★

Built-in

```
function union(arr1, arr2) {  
  return [...new Set([...arr1, ...arr2])];  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));  
// [1, 2, 3, 4]
```

Time: $O(n + m)$

Space: $O(n + m)$

Without built-in

```
function union(arr1, arr2) {  
  let result = [];  
  
  for (let num of arr1) {  
    if (!result.includes(num)) {  
      result.push(num);  
    }  
  }  
}
```

```
  for (let num of arr2) {  
    if (!result.includes(num)) {  
      result.push(num);  
    }  
  }  
}
```

```
  return result;  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4]));  
// [1, 2, 3, 4]
```

Time: $O((n + m)^2)$

Space: $O(n + m)$

Optimized

```
function union(arr1, arr2) {  
  let map = {};  
  let result = [];  
  
  for (let num of arr1) {  
    if (!map[num]) {  
      map[num] = true;  
      result.push(num);  
    }  
  }  
  
  for (let num of arr2) {  
    if (!map[num]) {  
      map[num] = true;  
      result.push(num);  
    }  
  }  
  
  return result;  
}  
  
console.log(union([1, 2, 3], [2, 3, 4]));  
// [1, 2, 3, 4]
```

Time: $O(n + m)$

Space: $O(n + m)$

14. Move All Zeros to End ★★

Built-in

```
function moveZeros(arr) {  
  let nonZero = arr.filter(num => num !== 0);  
  let zeros = arr.filter(num => num === 0);  
  
  return [...nonZero, ...zeros];  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));  
// [1, 3, 12, 0, 0]
```

Time: O(n)

Space: O(n)

Without built-in

```
function moveZeros(arr) {  
  let result = [];  
  
  for (let num of arr) {  
    if (num !== 0) {  
      result.push(num);  
    }  
  }  
  
  for (let num of arr) {  
    if (num === 0) {  
      result.push(num);  
    }  
  }  
  
  return result;  
}
```

```
console.log(moveZeros([0, 1, 0, 3, 12]));  
// [1, 3, 12, 0, 0]
```

Time: O(n)

Space: O(n)

Optimized

```
function moveZeros(arr) {  
  let index = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] !== 0) {  
      [arr[index], arr[i]] = [arr[i], arr[index]];  
      index++;  
    }  
  }  
}
```

```
    return arr;
}

console.log(moveZeros([0, 1, 0, 3, 12]));
// [1, 3, 12, 0, 0]
```

Time: O(n)
Space: O(1)

15. Move Negative Numbers to One Side ★★

Built-in

```
function moveNegatives(arr) {
  return [
    ...arr.filter(num => num < 0),
    ...arr.filter(num => num >= 0)
  ];
}

console.log(moveNegatives([1, -2, 3, -4, 5]));
// [-2, -4, 1, 3, 5]
```

Time: O(n)
Space: O(n)

Without built-in

```
function moveNegatives(arr) {
  let result = [];

  for (let num of arr) {
    if (num < 0) {
      result.push(num);
    }
  }

  for (let num of arr) {
    if (num >= 0) {
      result.push(num);
    }
  }
}
```

```
    return result;
}

console.log(moveNegatives([1, -2, 3, -4, 5]));
// [-2, -4, 1, 3, 5]
```

Time: O(n)

Space: O(n)

Optimized

```
function moveNegatives(arr) {
    let index = 0;

    for (let i = 0; i < arr.length; i++) {
        if (arr[i] < 0) {
            [arr[index], arr[i]] = [arr[i], arr[index]];
            index++;
        }
    }

    return arr;
}

console.log(moveNegatives([1, -2, 3, -4, 5]));
// [-2, -4, 3, 1, 5]
```

Time: O(n)

Space: O(1)

Note: optimized version negative elements ko ek side karta hai; exact original order preserve karna required nahi hai.

16. Find Pair with Given Sum ★★

Example: target = 9 → 2 + 7

Built-in

```
function pairSum(arr, target) {
    for (let num of arr) {
```

```

    let needed = target - num;

    if (arr.includes(needed) && needed !== num) {
        return [num, needed];
    }
}

return [];
}

console.log(pairSum([2, 7, 11, 15], 9));
// [2, 7]

```

Time: $O(n^2)$

Space: $O(1)$

Without built-in

```

function pairSum(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }
        }
    }
}

return [];
}

console.log(pairSum([2, 7, 11, 15], 9));
// [2, 7]

```

Time: $O(n^2)$

Space: $O(1)$

Optimized — HashMap

```

function pairSum(arr, target) {
    let map = {};

    for (let num of arr) {
        let needed = target - num;
    }
}

```

```
    if (map[needed]) {  
        return [needed, num];  
    }  
  
    map[num] = true;  
}  
  
return [];  
}  
  
console.log(pairSum([2, 7, 11, 15], 9));  
// [2, 7]
```

Time: $O(n)$

Space: $O(n)$

17. Find Majority Element ★★

Majority = element appearing **more than $n/2$ times**.

Built-in / HashMap

```
function majorityElement(arr) {  
    let map = {};  
  
    for (let num of arr) {  
        map[num] = (map[num] || 0) + 1;  
  
        if (map[num] > arr.length / 2) {  
            return num;  
        }  
    }  
  
    return -1;  
}  
  
console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));  
// 2
```

Time: $O(n)$

Space: $O(n)$

Without HashMap

```
function majorityElement(arr) {
  for (let i = 0; i < arr.length; i++) {
    let count = 0;

    for (let j = 0; j < arr.length; j++) {
      if (arr[i] === arr[j]) {
        count++;
      }
    }

    if (count > arr.length / 2) {
      return arr[i];
    }
  }

  return -1;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
// 2
```

Time: $O(n^2)$

Space: $O(1)$

Optimized — Boyer-Moore

```
function majorityElement(arr) {
  let candidate = 0;
  let count = 0;

  for (let num of arr) {
    if (count === 0) {
      candidate = num;
    }

    if (num === candidate) {
      count++;
    } else {
      count--;
    }
  }
}
```

```
    return candidate;
}

console.log(majorityElement([2, 2, 1, 2, 3, 2, 2]));
// 2
```

Time: $O(n)$
Space: $O(1)$

18. Find Pair with Given Difference ★★

Without built-in

```
function pairDifference(arr, difference) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (Math.abs(arr[i] - arr[j]) === difference) {
                return [arr[i], arr[j]];
            }
        }
    }

    return [];
}
```

```
console.log(pairDifference([1, 5, 3, 7], 2));
// [1, 3]
```

Time: $O(n^2)$
Space: $O(1)$

Optimized — HashMap

```
function pairDifference(arr, difference) {
    let map = {};

    for (let num of arr) {
        if (map[num - difference] || map[num + difference]) {
            return true;
        }

        map[num] = true;
    }
}
```



```
}

return false;
}

console.log(pairDifference([1, 5, 3, 7], 2));
// true
```

Time: $O(n)$

Space: $O(n)$

19. Maximum Consecutive 1s ★★

Without built-in

```
function maxOnes(arr) {
  let count = 0;
  let max = 0;

  for (let num of arr) {
    if (num === 1) {
      count++;

      if (count > max) {
        max = count;
      }
    } else {
      count = 0;
    }
  }

  return max;
}

console.log(maxOnes([1, 1, 0, 1, 1, 1]));
// 3
```

Time: $O(n)$

Space: $O(1)$

Optimized

Same as without built-in — already optimal.

20. Maximum Subarray Sum ★★

Example: $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow 6$

Without built-in

```
function maxSubarraySum(arr) {
  let max = arr[0];

  for (let i = 0; i < arr.length; i++) {
    let sum = 0;

    for (let j = i; j < arr.length; j++) {
      sum += arr[j];

      if (sum > max) {
        max = sum;
      }
    }
  }

  return max;
}

console.log(maxSubarraySum([-2, 1, -3, 4, -1, 2, 1, -5, 4]));
// 6
```

Time: $O(n^2)$

Space: $O(1)$

Optimized — Kadane's Algorithm

```
function maxSubarraySum(arr) {
  let current = arr[0];
  let max = arr[0];

  for (let i = 1; i < arr.length; i++) {
    current = Math.max(arr[i], current + arr[i]);
  }
}
```

```
    if (current > max) {  
        max = current;  
    }  
}  
  
return max;  
}  
  
console.log(maxSubarraySum([-2, 1, -3, 4, -1, 2, 1, -5, 4]));  
// 6
```

Time: $O(n)$

Space: $O(1)$

21. Maximum Profit from Buying and Selling Stock ★★

Without built-in

```
function maxProfit(prices) {  
    let maxProfit = 0;  
  
    for (let i = 0; i < prices.length; i++) {  
        for (let j = i + 1; j < prices.length; j++) {  
            let profit = prices[j] - prices[i];  
  
            if (profit > maxProfit) {  
                maxProfit = profit;  
            }  
        }  
    }  
  
    return maxProfit;  
}  
  
console.log(maxProfit([7, 1, 5, 3, 6, 4]));  
// 5
```

Time: $O(n^2)$

Space: $O(1)$

Optimized

```
function maxProfit(prices) {  
  let minPrice = prices[0];  
  let maxProfit = 0;  
  
  for (let i = 1; i < prices.length; i++) {  
    if (prices[i] < minPrice) {  
      minPrice = prices[i];  
    }  
  
    let profit = prices[i] - minPrice;  
  
    if (profit > maxProfit) {  
      maxProfit = profit;  
    }  
  }  
  
  return maxProfit;  
}  
  
console.log(maxProfit([7, 1, 5, 3, 6, 4]));  
// 5
```

Time: $O(n)$

Space: $O(1)$

22. Find Subarray with Given Sum ★★

Without built-in

```
function subarraySum(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    let sum = 0;  
  
    for (let j = i; j < arr.length; j++) {  
      sum += arr[j];  
    }  
  }  
}
```

```

        if (sum === target) {
            return arr.slice(i, j + 1);
        }
    }
}

return [];

console.log(subarraySum([1, 2, 3, 7, 5], 12));
// [2, 3, 7]

```

Time: $O(n^2)$

Space: $O(n)$

Optimized — Sliding Window

For **positive numbers**:

```

function subarraySum(arr, target) {
    let start = 0;
    let sum = 0;

    for (let end = 0; end < arr.length; end++) {
        sum += arr[end];

        while (sum > target) {
            sum -= arr[start];
            start++;
        }

        if (sum === target) {
            return arr.slice(start, end + 1);
        }
    }

    return [];
}

console.log(subarraySum([1, 2, 3, 7, 5], 12));
// [2, 3, 7]

```

Time: $O(n)$

Space: $O(n)$ because of returned slice.

23. Longest Subarray with Given Sum ★★ ★

For **positive numbers**:

```
function longestSubarray(arr, target) {
  let start = 0;
  let sum = 0;
  let maxLength = 0;

  for (let end = 0; end < arr.length; end++) {
    sum += arr[end];

    while (sum > target) {
      sum -= arr[start];
      start++;
    }

    if (sum === target) {
      let length = end - start + 1;

      if (length > maxLength) {
        maxLength = length;
      }
    }
  }

  return maxLength;
}

console.log(longestSubarray([1, 2, 1, 1, 1, 3, 2], 5));
// 4
```

Time: $O(n)$

Space: $O(1)$

Agar array mein negative numbers bhi ho sakte hain, to prefix-sum + HashMap approach use hoti hai.

24. Rotate Array by K Positions ★★

Example:

[1,2,3,4,5], K = 2

Output:

[4,5,1,2,3]

Built-in

```
function rotate(arr, k) {  
  k = k % arr.length;  
  
  return arr.slice(-k).concat(arr.slice(0, -k));  
}
```

```
console.log(rotate([1, 2, 3, 4, 5], 2));  
// [4, 5, 1, 2, 3]
```

Time: O(n)

Space: O(n)

Without built-in

```
function rotate(arr, k) {  
  let result = [];  
  
  k = k % arr.length;  
  
  for (let i = arr.length - k; i < arr.length; i++) {  
    result.push(arr[i]);  
  }  
  
  for (let i = 0; i < arr.length - k; i++) {  
    result.push(arr[i]);  
  }  
  
  return result;  
}
```

```
console.log(rotate([1, 2, 3, 4, 5], 2));  
// [4, 5, 1, 2, 3]
```

Time: $O(n)$

Space: $O(n)$

Optimized — In-place

```
function rotate(arr, k) {  
  k = k % arr.length;  
  
  reversePart(arr, 0, arr.length - 1);  
  reversePart(arr, 0, k - 1);  
  reversePart(arr, k, arr.length - 1);  
  
  return arr;  
}  
  
function reversePart(arr, start, end) {  
  while (start < end) {  
    [arr[start], arr[end]] = [arr[end], arr[start]];  
    start++;  
    end--;  
  }  
}
```

```
console.log(rotate([1, 2, 3, 4, 5], 2));  
// [4, 5, 1, 2, 3]
```

Time: $O(n)$

Space: $O(1)$

25. Find Kth Largest / Smallest Element ★★ ★

Example:

[7, 10, 4, 3, 20, 15], $k = 3$

Kth smallest = 7

Kth largest = 10

Built-in

```
function kthSmallest(arr, k) {  
  let sorted = [...arr].sort((a, b) => a - b);  
  return sorted[k - 1];  
}  
  
function kthLargest(arr, k) {  
  let sorted = [...arr].sort((a, b) => b - a);  
  return sorted[k - 1];  
}  
  
console.log(kthSmallest([7, 10, 4, 3, 20, 15], 3));  
// 7  
  
console.log(kthLargest([7, 10, 4, 3, 20, 15], 3));  
// 10
```

Time: $O(n \log n)$

Space: $O(n)$

Without built-in

```
function kthSmallest(arr, k) {  
  let copy = [];  
  
  for (let num of arr) {  
    copy.push(num);  
  }  
  
  for (let i = 0; i < copy.length; i++) {  
    for (let j = i + 1; j < copy.length; j++) {  
      if (copy[i] > copy[j]) {  
        [copy[i], copy[j]] = [copy[j], copy[i]];  
      }  
    }  
  }  
  
  return copy[k - 1];  
}  
  
console.log(kthSmallest([7, 10, 4, 3, 20, 15], 3));  
// 7
```

Time: $O(n^2)$

Space: $O(n)$

Optimized — Quick Select

```
function kthSmallest(arr, k) {  
  let left = 0;  
  let right = arr.length - 1;  
  
  while (left <= right) {  
    let pivot = arr[right];  
    let index = left;  
  
    for (let i = left; i < right; i++) {  
      if (arr[i] < pivot) {  
        [arr[i], arr[index]] = [arr[index], arr[i]];  
        index++;  
      }  
    }  
  
    [arr[index], arr[right]] = [arr[right], arr[index]];  
  
    if (index === k - 1) {  
      return arr[index];  
    }  
  
    if (index < k - 1) {  
      left = index + 1;  
    } else {  
      right = index - 1;  
    }  
  }  
}  
  
console.log(kthSmallest([7, 10, 4, 3, 20, 15], 3));  
// 7
```

Average Time: $O(n)$

Worst Time: $O(n^2)$

Space: $O(1)$

ARRAY QUICK REVISION

#	Question	Best Complexity
1	Largest	$O(n)$
2	Smallest	$O(n)$
3	Second Largest	$O(n)$
4	Second Smallest	$O(n)$
5	Reverse	$O(n)$, $O(1)$
6	Sorted Check	$O(n)$
7	Remove Duplicates	$O(n)$, $O(1)$
8	Frequency	$O(n)$
9	Missing Number	$O(n)$, $O(1)$
10	Duplicate	$O(n)$, $O(n)$
11	All Duplicates	$O(n)$, $O(n)$
12	Intersection	$O(n+m)$
13	Union	$O(n+m)$
14	Move Zeros	$O(n)$, $O(1)$
15	Move Negatives	$O(n)$, $O(1)$
16	Pair Sum	$O(n)$, $O(n)$
17	Majority Element	$O(n)$, $O(1)$
18	Pair Difference	$O(n)$, $O(n)$
19	Consecutive 1s	$O(n)$, $O(1)$
20	Maximum Subarray Sum	$O(n)$, $O(1)$
21	Stock Profit	$O(n)$, $O(1)$
22	Subarray Given Sum	$O(n)^*$
23	Longest Subarray Sum	$O(n)$
24	Rotate Array	$O(n)$, $O(1)$
25	Kth Smallest/Largest	Avg $O(n)$

***Q22 ka $O(n)$ sliding-window solution positive numbers ke case mein hai.**

🎯 Placement ke liye sabse zyada priority

Must-do:

Largest → Second Largest → Reverse → Sorted → Frequency → Missing → Duplicate → Move Zeros → Pair Sum → Majority → Maximum Subarray → Stock Profit → Rotate → Kth Largest/Smallest

Ye **25 Arrays ka complete interview set** hai. Iske baad Array ke random 50–100 questions karne ki zarurat nahi hai; in 25 ko code karke practice karna zyada useful rahega.

Now react code dedo jo interviewe puchh skte h likhne ke liye kya kya ho skta h
vaise

Haan. **React interview mein theory se zyada kabhi-kabhi “ye component/code likho”** bol dete hain. Fresher/service-based placement ke hisaab se ye coding tasks sabse important hain.

🔥 React Interview — Code Writing Questions

🟢 Level 1 — Must Do

1. Create a simple React component

Interviewer: “Create a functional component that displays Hello World.”

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

```
export default App;
```

2. Create a component using Props ★★

Interviewer: “Create a component that receives a name through props and displays it.”

```
function User({ name }) {  
  return <h2>Hello, {name}</h2>;  
}
```

```
function App() {  
  return <User name="Pranjal" />;  
}
```

```
export default App;
```

3. useState — Counter ★★★★★

Interviewer: “Create a counter with increment and decrement buttons.”

```
import { useState } from "react";
```

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>{count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
  
      <button onClick={() => setCount(count - 1)}>  
        Decrement  
      </button>  
    </div>  
  );  
}
```

```
export default App;
```

4. Toggle Button ★★★★★

Interviewer: “Create a button that toggles between Show and Hide.”

```
import { useState } from "react";
```

```
function App() {
```

```
const [show, setShow] = useState(true);

return (
  <div>
    <button onClick={() => setShow(!show)}>
      {show ? "Hide" : "Show"}
    </button>

    {show && <p>Hello React</p>}
  </div>
);
}

export default App;
```

5. Input Field ★★

Interviewer: “Create an input field and display its value.”

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <p>{name}</p>
    </div>
  );
}

export default App;
```

6. Controlled Form ★★

Interviewer: “Create a form with name and email fields and handle its submission.”

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    console.log(name);
    console.log(email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Name"
      />

      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />

      <button type="submit">Submit</button>
    </form>
  );
}

export default App;
```

🟡 Level 2 — Very Important

7. Render an Array using map() ★★☆☆

Interviewer: “Display a list of users using map().”

```
function App() {  
  const users = ["Aman", "Riya", "Rahul"];  
  
  return (  
    <div>  
      {users.map((user, index) => (  
        <p key={index}>{user}</p>  
      ))}  
    </div>  
  );  
}  
  
export default App;
```

8. Conditional Rendering ★★☆☆

Interviewer: “Show different content based on login status.”

```
function App() {  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h2>Welcome User</h2>  
      ) : (  
        <h2>Please Login</h2>  
      )}  
    </div>  
  );  
}  
  
export default App;
```

9. Search Filter ★★

Interviewer: “Create a search box that filters a list of users.”

```
import { useState } from "react";

function App() {
  const users = ["Aman", "Riya", "Rahul", "Priya"];
  const [search, setSearch] = useState("");

  const filteredUsers = users.filter((user) =>
    user.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <input
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        placeholder="Search"
      />

      {filteredUsers.map((user, index) => (
        <p key={index}>{user}</p>
      ))}
    </div>
  );
}

export default App;
```

10. Add Item to List ★★

Interviewer: “Create an input and add new items to a list.”

```
import { useState } from "react";

function App() {
  const [item, setItem] = useState("");
  const [items, setItems] = useState([]);

  function addItem() {
```

```

    if (item.trim() === "") return;

    setItems([...items, item]);
    setItem("");
  }

  return (
    <div>
      <input
        value={item}
        onChange={(e) => setItem(e.target.value)}
      />

      <button onClick={addItem}>Add</button>

      {items.map((item, index) => (
        <p key={index}>{item}</p>
      ))}
    </div>
  );
}

export default App;

```

11. Delete Item from List ★★☆☆

Interviewer: “Create a list where each item can be deleted.”

```

import { useState } from "react";

function App() {
  const [items, setItems] = useState(["Apple", "Banana", "Mango"]);

  function deleteItem(index) {
    setItems(items.filter((_, i) => i !== index));
  }

  return (
    <div>
      {items.map((item, index) => (
        <div key={index}>

```

```

        <span>{item}</span>

        <button onClick={() => deleteItem(index)}>
          Delete
        </button>
      </div>
    )}
  </div>
);
}

export default App;

```

12. useEffect — Fetch API ★★★★★

Interviewer: “Fetch users from an API when the component mounts.”

```

import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => setUsers(data));
  }, []);

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;

```

13. Loading State ★★★★★

Interviewer: “Show Loading while API data is being fetched.”

```
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <h2>Loading...</h2>;
  }

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

14. useEffect Dependency ★★★★★

Interviewer: “Run an effect whenever count changes.”

```
import { useEffect, useState } from "react";

function App() {
  const [count, setCount] = useState(0);
```

```
useEffect(() => {
  console.log("Count changed:", count);
}, [count]);

return (
  <button onClick={() => setCount(count + 1)}>
    {count}
  </button>
);
}

export default App;
```

15. Parent to Child Data ★★★★★

Interviewer: “Pass data from parent component to child component.”

```
function Child({ name }) {
  return <h2>Hello {name}</h2>;
}

function App() {
  return <Child name="Pranjal" />;
}

export default App;
```

16. Child to Parent Data ★★★★★

Interviewer: “Send data from child component to parent.”

```
import { useState } from "react";

function Child({ sendData }) {
  return (
    <button onClick={() => sendData("Hello Parent")}>
      Send Data
    </button>
  );
}
```

```

}

function App() {
  const [message, setMessage] = useState("");

  return (
    <div>
      <Child sendData={setMessage} />
      <p>{message}</p>
    </div>
  );
}

export default App;

```

Level 3 — Interview mein kaafi useful

17. React Router — Two Pages ★★☆☆

Interviewer: “Create Home and About pages using React Router.”

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

```

```
    </Routes>
  </BrowserRouter>
);
}
```

```
export default App;
```

18. Fetch API with Axios ★★★★★

Interviewer: “Fetch data from an API using Axios.”

```
import axios from "axios";
import { useEffect, useState } from "react";
```

```
function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      });
  }, []);

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

```
export default App;
```

19. Context API ★★★★★

Interviewer: “Create a context and access its value in a child component.”

```
import { createContext, useContext } from "react";

const UserContext = createContext();

function Child() {
  const name = useContext(UserContext);

  return <h2>{name}</h2>;
}

function App() {
  return (
    <UserContext.Provider value="Pranjal">
      <Child />
    </UserContext.Provider>
  );
}

export default App;
```

20. useRef ★★

Interviewer: “Focus an input using useRef.”

```
import { useRef } from "react";

function App() {
  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>
    </div>
  );
}
```



```
);  
}
```

```
export default App;
```

★ Sabse Important 10

Agar interviewer tumhe **live React coding** de, to sabse pehle inhe perfect karo:

1. **Counter using useState**
2. **Toggle Show/Hide**
3. **Controlled Input**
4. **Form Handling**
5. **Array rendering using map**
6. **Search/Filter**
7. **Add & Delete items**
8. **API fetching using useEffect**
9. **Props — Parent → Child**
10. **Child → Parent using callback**

Aur tumhare **MERN fresher interview** ke liye uske baad **React Router + Axios + Context API + useRef** kar lena.

Ye list theory questions nahi hai — ye specifically woh type ke code-writing tasks hain jo interviewer tumhe editor/screen par likhne ko de sakta hai.

Node vgera ke bhi puchte h kya

Haan, **Node.js aur Express.js ka code bhi interview mein likhne ko de sakte hain**, especially MERN/full-stack fresher role mein.

React ke baad tumhe ye **most important code-writing tasks** karne chahiye:

🔥 Node.js

1. Simple HTTP server create karna
2. fs module se file read/write
3. File create/update/delete

4. path module use karna
5. Environment variables use karna
6. Async/await ka example
7. Promise handling
8. Error handling with try/catch

Express.js ★★

1. Basic Express server
2. GET API banana
3. POST API banana
4. PUT API banana
5. DELETE API banana
6. Route parameters — /users/:id
7. Query parameters — /users?name=...
8. Middleware banana
9. Error-handling middleware
10. JSON request body handle karna
11. CORS setup
12. Router banana
13. Controller + Routes structure
14. Authentication middleware
15. JWT generate/verify karna

MongoDB + Mongoose

1. Connect MongoDB
2. Create Schema/Model
3. Insert document
4. Find documents
5. Find by ID
6. Update document
7. Delete document
8. Search/filter
9. MongoDB queries
10. Mongoose relationships/populate

🎯 Tumhare placement ke liye order

React → Node → Express → MongoDB → Authentication/JWT → REST API

Aur haan, Node/Express mein bhi main tumhe wahi format dunga:

Interviewer question → simple code → output/response → Time/Space complexity (jahan meaningful ho).

React ke baad **Node.js ke 15 most important code-writing questions** se start karna best rahega.

Ha yrr dedo

Haan yrr 😊 **Node.js ke 15 most important code-writing questions** karte hain. Ye fresher/MERN interview ke liye kaafi relevant hain.

🔥 NODE.JS — 15 CODE WRITING QUESTIONS

1. Create a Basic HTTP Server ★★★★★

Interviewer: “Create a simple HTTP server using Node.js.”

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.write("Hello Node.js");
  res.end();
});

server.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Output:

Server running on port 5000

Browser:

Hello Node.js

Time: $O(1)$ per request

Space: $O(1)$

2. Handle Different Routes ★★

Interviewer: “Create different responses for different URLs.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  if (req.url === "/") {
    res.end("Home Page");
  }

  else if (req.url === "/about") {
    res.end("About Page");
  }

  else {
    res.end("Page Not Found");
  }
});

server.listen(5000);
```

Output:

/ → Home Page

/about → About Page

Time: $O(1)$

Space: $O(1)$

3. Read a File using fs ★★

Interviewer: “Read a file using Node.js.”

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log(data);
});
```

Time: $O(n)$

Space: $O(n)$

4. Write Data to a File ★★

Interviewer: “Write data into a file using Node.js.”

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello Node.js", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("File written successfully");
});
```

Output:

File written successfully

Time: $O(n)$

Space: $O(n)$

5. Append Data to a File ★★

Interviewer: “Add new data to an existing file.”

```
const fs = require("fs");

fs.appendFile("data.txt", "\nHello Again", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("Data added");
});
```

Time: $O(n)$

Space: $O(n)$

6. Delete a File ★★

Interviewer: “Delete a file using Node.js.”

```
const fs = require("fs");

fs.unlink("data.txt", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("File deleted");
});
```

Time: $O(n)$

Space: $O(1)$

7. Use Async/Await

Interviewer: “Read a file using async/await.”

```
const fs = require("fs").promises;

async function readFile() {
  try {
    const data = await fs.readFile("data.txt", "utf8");

    console.log(data);
  } catch (error) {
    console.log(error);
  }
}

readFile();
```

Time: $O(n)$

Space: $O(n)$

8. Handle Errors using try/catch

Interviewer: “Handle errors in an asynchronous function.”

```
async function getData() {

  try {
    let data = await fetchData();

    console.log(data);
  } catch (error) {
    console.log("Something went wrong");
  }
}

getData();
```

Main point: try mein successful code, catch mein error handling.

9. Create a Simple REST API ★★

Interviewer: “Create a GET API using Node.js.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.setHeader("Content-Type", "application/json");

  if (req.url === "/users") {

    const users = [
      { id: 1, name: "Aman" },
      { id: 2, name: "Riya" }
    ];

    res.end(JSON.stringify(users));
  }
});

server.listen(5000);
```

Response:

```
[
  {
    "id": 1,
    "name": "Aman"
  },
  {
    "id": 2,
    "name": "Riya"
  }
]
```

10. Use Environment Variables ★★

Interviewer: “How do you use environment variables in Node.js?”

.env


```
PORT=5000
DB_URL=mongodb://localhost:27017/mydb
```

server.js

```
require("dotenv").config();
```

```
const port = process.env.PORT;
```

```
console.log(port);
```

Output:

5000

👉 Real projects mein **database URL**, **API keys**, **JWT secret** etc. environment variables mein rakhte hain.

11. Create a Promise ★★

Interviewer: “Create and handle a Promise.”

```
function getData() {
  return new Promise((resolve, reject) => {

    let success = true;

    if (success) {
      resolve("Data received");
    } else {
      reject("Something went wrong");
    }
  });
}
```

```
getData()
  .then((data) => console.log(data))
  .catch((error) => console.log(error));
```

Output:

Data received

12. Use EventEmitter ★★

Interviewer: “Create a custom event using EventEmitter.”

```
const EventEmitter = require("events");
```

```
const event = new EventEmitter();
```

```
event.on("message", () => {  
  console.log("Message received");  
});
```

```
event.emit("message");
```

Output:

Message received

Important: on() listener register karta hai, emit() event trigger karta hai.

13. Create a Custom Module ★★

Interviewer: “Create and import a custom Node.js module.”

math.js

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

app.js

```
const add = require("./math");
```

```
console.log(add(10, 20));
```

Output:

30

14. Get Data from URL ★★

Interviewer: “Get query parameters from a URL.”

```
const http = require("http");
const url = require("url");

const server = http.createServer((req, res) => {

  const query = url.parse(req.url, true).query;

  res.end(`Hello ${query.name}`);
});

server.listen(5000);
```

URL:

http://localhost:5000/?name=Pranjal

Output:

Hello Pranjal

15. Create a Server with JSON Response ★★

Interviewer: “Return JSON data from a Node.js server.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.setHeader("Content-Type", "application/json");

  const user = {
    name: "Pranjal",
    role: "Developer"
  };
});
```

```
res.end(JSON.stringify(user));  
});
```

```
server.listen(5000);
```

Response:

```
{  
  "name": "Pranjal",  
  "role": "Developer"  
}
```

Time: O(1)

Space: O(1)

Node.js mein sabse important

Agar **15 mein se priority** nikalni ho:

★ ★ ★ MUST DO

1. Basic HTTP Server
2. Routes
3. fs read/write
4. Async/Await
5. Error Handling
6. REST API
7. Environment Variables
8. Promise
9. JSON Response
10. URL/Query Parameters

★ ★ Secondary

11. Append/Delete file
 12. EventEmitter
 13. Custom Module
-

But ek important point

MERN interview mein Node.js se zyada Express.js ke code-writing questions aane ke chances hote hain, kyunki actual backend APIs generally Express se banate ho.

Isliye next **Express.js ke 15 most important live-coding questions** karna sabse useful rahega:

GET → POST → PUT → DELETE → params → query → middleware → error middleware → router → controller → JWT auth → protected route → MongoDB API.

Ok start aur ha mai esmodules use krti hu naki common ks

Haan yrr, **tum ES Modules use karti ho**, so ab se Node/Express ka code **import/export syntax** mein hi dunga — `require()` / `module.exports` nahi.

Chalo **Express.js ke 15 most important code-writing questions** start karte hain, same interview format mein.

EXPRESS.JS — COMPLETE CODE-WRITING LECTURE

1. Create a Basic Express Server ★★ ★

Interviewer: “Create a basic Express server.”

```
import express from "express";

const app = express();

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Output:

Server running on port 5000

Time: O(1)
Space: O(1)

2. Create a GET API ★★

Interviewer: “Create a GET API that returns a message.”

```
import express from "express";

const app = express();

app.get("/api/users", (req, res) => {
  res.json({
    message: "Users fetched successfully"
  });
});

app.listen(5000);
```

Response:

```
{
  "message": "Users fetched successfully"
}
```

3. Create a POST API ★★

Interviewer: “Create a POST API to receive user data.”

```
import express from "express";

const app = express();

app.use(express.json());

app.post("/api/users", (req, res) => {
  const { name, email } = req.body;

  res.json({
    message: "User created",
```

```
    name,  
    email  
  });  
};  
  
app.listen(5000);
```

Request Body:

```
{  
  "name": "Pranjal",  
  "email": "pranjal@gmail.com"  
}
```

Response:

```
{  
  "message": "User created",  
  "name": "Pranjal",  
  "email": "pranjal@gmail.com"  
}
```

4. Create PUT API ★★

Interviewer: “Create an API to update a user.”

```
app.put("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  const { name } = req.body;  
  
  res.json({  
    message: "User updated",  
    id,  
    name  
  });  
});
```

URL:

PUT /api/users/10

5. Create DELETE API ★★

Interviewer: “Create an API to delete a user.”

```
app.delete("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  
  res.json({  
    message: "User deleted",  
    id  
  });  
});
```

URL:

DELETE /api/users/10

6. Route Parameters ★★

Interviewer: “Get an ID from the URL.”

```
app.get("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  
  res.json({  
    userId: id  
  });  
});
```

URL:

/api/users/25

Response:

```
{  
  "userId": "25"  
}
```

7. Query Parameters

Interviewer: “Get search data from query parameters.”

```
app.get("/api/users", (req, res) => {  
  const name = req.query.name;  
  
  res.json({  
    search: name  
  });  
});
```

URL:

/api/users?name=Pranjal

Response:

```
{  
  "search": "Pranjal"  
}
```

Yaad rakho:

```
/users/:id    → req.params.id  
/users?name=x → req.query.name
```

8. Middleware

Interviewer: “Create a middleware that logs every request.”

```
function logger(req, res, next) {  
  console.log(req.method, req.url);  
  next();  
}
```

```
app.use(logger);
```

next() ka kaam hai **request ko next middleware/route par bhejna.**

9. Authentication Middleware ★★

Interviewer: “Create middleware to check whether a user is authenticated.”

```
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization;  
  
  if (!token) {  
    return res.status(401).json({  
      message: "Unauthorized"  
    });  
  }  
  
  next();  
}
```

Protected route:

```
app.get("/api/profile", authMiddleware, (req, res) => {  
  res.json({  
    message: "Profile data"  
  });  
});
```

10. Error Handling Middleware ★★

Interviewer: “Create an error-handling middleware.”

```
app.use((err, req, res, next) => {  
  console.log(err);  
  
  res.status(500).json({  
    message: "Something went wrong"  
  });  
});
```

⚠ Express error middleware mein **4 parameters** hote hain:

```
(err, req, res, next)
```

11. Router ★★

Interviewer: “Create separate routes using Express Router.”

routes/userRoutes.js

```
import express from "express";
```

```
const router = express.Router();
```

```
router.get("/", (req, res) => {  
  res.json({  
    message: "All users"  
  });  
});
```

```
router.post("/", (req, res) => {  
  res.json({  
    message: "User created"  
  });  
});
```

```
export default router;
```

server.js

```
import express from "express";
```

```
import userRoutes from "../routes/userRoutes.js";
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use("/api/users", userRoutes);
```

```
app.listen(5000);
```

12. Controller + Route ★★

Interviewer: “Separate controller logic from routes.”

controllers/userController.js

```
export function getUsers(req, res) {  
  res.json({  
    message: "All users"  
  });  
}
```

routes/userRoutes.js

```
import express from "express";  
import { getUsers } from "../controllers/userController.js";
```

```
const router = express.Router();
```

```
router.get("/", getUsers);
```

```
export default router;
```

server.js

```
import express from "express";  
import userRoutes from "../routes/userRoutes.js";
```

```
const app = express();
```

```
app.use("/api/users", userRoutes);
```

```
app.listen(5000);
```

13. Async Controller with Error Handling



Interviewer: “Create an asynchronous controller.”

```
export async function getUsers(req, res) {  
  try {  
    const users = await User.find();  
  
    res.json(users);  
  } catch (error) {  
    res.status(500).json({
```

```
    message: "Failed to fetch users"
  });
}
```

Ye **MongoDB + Mongoose** ke saath bahut common hai.

14. JWT Authentication ★★

Interviewer: “Generate a JWT token after login.”

```
import jwt from "jsonwebtoken";
```

```
const token = jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);
```

```
res.json({
  token
});
```

Verify token middleware:

```
import jwt from "jsonwebtoken";
```

```
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
```

```
  if (!token) {
    return res.status(401).json({
      message: "Unauthorized"
    });
  }
```

```
  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );
```

```
    req.user = decoded;

    next();
  } catch (error) {
    res.status(401).json({
      message: "Invalid token"
    });
  }
}

export default authMiddleware;
```

15. Complete CRUD API ★★

Interviewer: “Create CRUD APIs for users.”

```
import express from "express";

const app = express();

app.use(express.json());

let users = [];

app.get("/users", (req, res) => {
  res.json(users);
});

app.post("/users", (req, res) => {
  const user = {
    id: users.length + 1,
    name: req.body.name
  };

  users.push(user);

  res.status(201).json(user);
});
```

```
app.put("/users/:id", (req, res) => {
  const user = users.find(
    user => user.id === Number(req.params.id)
  );

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  user.name = req.body.name;

  res.json(user);
});

app.delete("/users/:id", (req, res) => {
  users = users.filter(
    user => user.id !== Number(req.params.id)
  );

  res.json({
    message: "User deleted"
  });
});

app.listen(5000);
```

EXPRESS — Placement Priority

★★★★ Must Practice

1. Basic Express Server
2. GET API
3. POST API
4. PUT API
5. DELETE API
6. req.body
7. req.params

8. req.query
9. Middleware
10. Authentication Middleware
11. Router
12. Controller
13. Error Handling
14. JWT Authentication
15. CRUD API

Tumhare MERN interviews ke liye ye Express coding set kaafi strong hai. Iske baad **MongoDB + Mongoose ke code-writing questions** karna logical next step hai.

Ha start

Bilkul yrr 🤪 **MongoDB + Mongoose code-writing interview prep** start karte hain.

Same pattern rakhenge: **Question → Built-in/simple → Without built-in → Optimized → Complexity.**

Aur Node/Express ki tarah **ES Modules** use karenge.

MongoDB + Mongoose — Most Important Code Questions

1. Connect MongoDB using Mongoose ★★★★★

Interviewer: “Write code to connect a Node.js application with MongoDB using Mongoose.”

```
import mongoose from "mongoose";

mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((error) => {
    console.log(error);
  });
```

Time: O(1) application-side

Space: O(1)

2. Create a Mongoose Schema ★★★★★

Interviewer: “Create a User schema with name, email and password.”

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});

export default mongoose.model("User", userSchema);
```

Time: O(1)

Space: O(1)

3. Insert/Create a Document ★★★★★

Interviewer: “Write an API to create a new user.”

```
app.post("/users", async (req, res) => {
  const { name, email, password } = req.body;

  const user = await User.create({
    name,
    email,
    password
  });

  res.json(user);
});
```

Time: O(1) average for one document

Space: O(1)

4. Find All Users ★★★★★

Interviewer: “Write a query to fetch all users.”

```
const users = await User.find();
```

```
console.log(users);
```

Time: $O(n)$

Space: $O(n)$ for returned documents

5. Find User by ID ★★

Interviewer: “Find a user using their ID.”

```
const user = await User.findById(req.params.id);
```

```
console.log(user);
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

6. Find User by Email ★★

Interviewer: “Find a user by email.”

```
const user = await User.findOne({  
  email: "abc@gmail.com"  
});
```

```
console.log(user);
```

Time: $O(n)$ without index, approximately $O(1)$ with an index

Space: $O(1)$

7. Update User ★★

Interviewer: “Update a user’s name using their ID.”

```
const user = await User.findByIdAndUpdate(  
  req.params.id,  
  { name: "Pranjal" },  
  { new: true }  
);
```

```
console.log(user);
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

8. Delete User ★★★★★

Interviewer: “Delete a user using their ID.”

```
await User.findByIdAndDelete(req.params.id);
```

```
console.log("User deleted");
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

9. Find Users with a Condition ★★★★★

Interviewer: “Find users whose age is greater than 18.”

```
const users = await User.find({  
  age: { $gt: 18 }  
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable index

Space: $O(n)$

10. Projection ★★★★★

Interviewer: “Fetch only name and email, not password.”

```
const users = await User.find()  
  .select("name email");
```

```
console.log(users);
```

Time: $O(n)$ without suitable index
Space: $O(n)$

11. Sort Documents ★★

Interviewer: “Fetch users sorted by name.”

```
const users = await User.find()  
  .sort({ name: 1 });
```

```
console.log(users);
```

1 → ascending

-1 → descending

Time: depends on index; without suitable index, sorting can be $O(n \log n)$

Space: depends on query execution

12. Limit Documents ★★

Interviewer: “Fetch only 5 users.”

```
const users = await User.find()  
  .limit(5);
```

```
console.log(users);
```

Time: depends on query/index

Space: $O(5)$ returned result

13. Skip Documents ★★

Interviewer: “Skip the first 5 users.”

```
const users = await User.find()  
  .skip(5)  
  .limit(5);
```

```
console.log(users);
```

Time: depends on database/index
Space: $O(5)$

14. Count Documents ★★

Interviewer: “Count the number of users.”

```
const count = await User.countDocuments();
```

```
console.log(count);
```

Time: depends on query/index
Space: $O(1)$

15. Check Whether User Exists ★★★

Interviewer: “Check whether a user with a particular email exists.”

```
const user = await User.findOne({  
  email: "abc@gmail.com"  
});
```

```
if (user) {  
  console.log("User exists");  
} else {  
  console.log("User does not exist");  
}
```

Time: $O(n)$ without index, approximately $O(1)$ with email index
Space: $O(1)$

🔥 Placement ke liye sabse important

In MongoDB/Mongoose me **ye pakka strong karo:**

1. find()
2. findOne()
3. findById()
4. create()

5. findByIdAndUpdate()
6. findByIdAndDelete()
7. \$gt, \$lt, \$gte, \$lte
8. \$in, \$ne
9. Projection — .select()
- 10..sort()
- 11..limit() / .skip()
- 12.countDocuments()
- 13.Schema + Model
- 14.MongoDB connection

Next: \$in, \$ne, \$and, \$or, pagination, populate(), aggregation aur proper CRUD API — ye interview ke liye next important set hai.

Ha dedo

Chalo yrr 🔥 **MongoDB/Mongoose ka next important set** — simple interview-style code ke saath.

16. \$in Operator ★★ ★

Interviewer: “Find users whose age is either 18, 20 or 22.”

```
const users = await User.find({  
  age: { $in: [18, 20, 22] }  
});
```

```
console.log(users);
```

Time: O(n) without suitable index

Space: O(n)

17. \$ne Operator ★★

Interviewer: “Find users whose role is not admin.”

```
const users = await User.find({  
  role: { $ne: "admin" }  
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable index

Space: $O(n)$

18. \$and Operator ★★★★★

Interviewer: “Find users whose age is greater than 18 and role is user.”

```
const users = await User.find({
  $and: [
    { age: { $gt: 18 } },
    { role: "user" }
  ]
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable indexes

Space: $O(n)$

19. \$or Operator ★★★★★

Interviewer: “Find users whose role is admin or manager.”

```
const users = await User.find({
  $or: [
    { role: "admin" },
    { role: "manager" }
  ]
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable indexes

Space: $O(n)$

20. Pagination ★★☆☆

Interviewer: “Implement pagination and return 5 users per page.”

```
const page = 2;
const limit = 5;

const users = await User.find()
  .skip((page - 1) * limit)
  .limit(limit);

console.log(users);
```

For page 2:

$\text{skip} = (2 - 1) \times 5 = 5$

So first 5 users skip honge.

Time: depends on database/index and skip size

Space: $O(\text{limit})$

21. populate() ★★☆☆

Interviewer: “Fetch bookings along with the user details.”

Suppose Booking schema:

```
const bookingSchema = new mongoose.Schema({
  car: String,
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});
```

Query:

```
const bookings = await Booking.find()
  .populate("user");

console.log(bookings);
```


Time: depends on number of documents and related lookups
Space: O(n)

22. Populate Specific Fields ★★ ★

Interviewer: “Populate only the user’s name and email.”

```
const bookings = await Booking.find()  
  .populate("user", "name email");
```

```
console.log(bookings);
```

Password jaise unnecessary fields nahi aayenge.

Time: depends on query

Space: O(n)

23. Aggregation — \$match ★★ ★

Interviewer: “Find users whose age is greater than 18 using aggregation.”

```
const users = await User.aggregate([  
  {  
    $match: {  
      age: { $gt: 18 }  
    }  
  }  
]);
```

```
console.log(users);
```

Time: depends on pipeline/index

Space: depends on result/pipeline

24. Aggregation — \$group ★★ ★

Interviewer: “Count users according to their role.”

```
const result = await User.aggregate([
  {
    $group: {
      _id: "$role",
      count: { $sum: 1 }
    }
  }
]);
```

```
console.log(result);
```

Example output:

```
[
  { _id: "user", count: 10 },
  { _id: "admin", count: 2 }
]
```

Time: $O(n)$ without useful optimization

Space: depends on number of groups

25. Aggregation — \$sort ★★

Interviewer: “Sort users by age in descending order.”

```
const users = await User.aggregate([
  {
    $sort: {
      age: -1
    }
  }
]);
```

```
console.log(users);
```

1 → ascending

-1 → descending

Time: generally $O(n \log n)$ without a suitable index

Space: depends on execution

26. Complete CRUD API ★★

Ye **bahut important** hai because interviewer bol sakta hai:

“Create a basic CRUD API using Express and MongoDB.”

Create

```
app.post("/users", async (req, res) => {  
  const user = await User.create(req.body);  
  
  res.json(user);  
});
```

Read

```
app.get("/users", async (req, res) => {  
  const users = await User.find();  
  
  res.json(users);  
});
```

Update

```
app.put("/users/:id", async (req, res) => {  
  const user = await User.findByIdAndUpdate(  
    req.params.id,  
    req.body,  
    { new: true }  
  );  
  
  res.json(user);  
});
```

Delete

```
app.delete("/users/:id", async (req, res) => {  
  await User.findByIdAndDelete(req.params.id);  
  
  res.json({  
    message: "User deleted"  
  });  
});
```

Time: CRUD operation ke according vary karega

Space: returned data ke according

MongoDB/Mongoose Placement Priority

Agar time kam ho to **is order mein karo**:

★★★ Must Do

- Schema + Model
- create()
- find()
- findOne()
- findById()
- Update
- Delete
- \$gt, \$lt, \$gte, \$lte
- \$in, \$ne
- \$and, \$or
- Projection
- populate()
- Pagination
- Complete CRUD

★★ Good to Know

- Aggregation
- \$match
- \$group
- \$sort
- Indexing

Ab MongoDB ka **basic + interview code portion kaafi strong** ho gaya. Next logically **Authentication + JWT code-writing** karna chahiye — register, login, bcrypt password hashing, JWT generate, auth middleware aur protected route.

Ha krade

Chalo yrr  ab **Authentication + JWT** karte hain. Ye MERN fresher interview mein **bahut important code-writing topic** hai.

Same pattern: **interviewer question** → **simple code** → **complexity**.
Aur haan, **ES Modules (import/export)** hi use karenge.

Authentication + JWT — Interview Code

1. Hash Password using bcrypt ★★★★★

Interviewer: “Hash the user’s password before storing it.”

```
import bcrypt from "bcrypt";

const hashedPassword = await bcrypt.hash(password, 10);

console.log(hashedPassword);
```

Time: $O(1)$ practically depends on bcrypt cost

Space: $O(1)$

2. Compare Password ★★★★★

Interviewer: “Check whether the entered password matches the hashed password.”

```
import bcrypt from "bcrypt";

const isMatch = await bcrypt.compare(
  password,
  user.password
);

console.log(isMatch);
```

Output:

true

or

false

Time: Depends on bcrypt cost

Space: $O(1)$

3. Register User ★★

Interviewer: “Create a registration API with password hashing.”

```
import bcrypt from "bcrypt";

app.post("/register", async (req, res) => {
  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = await User.create({
    name,
    email,
    password: hashedPassword
  });

  res.status(201).json({
    message: "User registered",
    user
  });
});
```

Time: Depends on bcrypt + database operation

Space: O(1)

4. Login User ★★

Interviewer: “Create a login API that verifies email and password.”

```
import bcrypt from "bcrypt";

app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
```

```

    return res.status(404).json({
      message: "User not found"
    });
  }

  const isMatch = await bcrypt.compare(
    password,
    user.password
  );

  if (!isMatch) {
    return res.status(401).json({
      message: "Invalid password"
    });
  }

  res.json({
    message: "Login successful"
  });
});

```

Time: Depends on DB + bcrypt

Space: O(1)

5. Generate JWT Token ★★

Interviewer: “Generate a JWT token after successful login.”

```
import jwt from "jsonwebtoken";
```

```

const token = jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);

```

```
console.log(token);
```

Time: O(1) for practical purposes

Space: O(1)

6. Login + JWT ★★

Ye **bahut important combined question** hai.

Interviewer: “Create a login API that returns a JWT token.”

```
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";

app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  const isMatch = await bcrypt.compare(
    password,
    user.password
  );

  if (!isMatch) {
    return res.status(401).json({
      message: "Invalid password"
    });
  }

  const token = jwt.sign(
    { userId: user._id },
    process.env.JWT_SECRET,
    { expiresIn: "1d" }
  );

  res.json({
    message: "Login successful",
    token
  });
});
```



```
});  
});
```

Time: DB + bcrypt + JWT generation

Space: O(1)

7. Verify JWT Token ★★ ★

Interviewer: “Verify a JWT token.”

```
import jwt from "jsonwebtoken";
```

```
const decoded = jwt.verify(  
  token,  
  process.env.JWT_SECRET  
);
```

```
console.log(decoded);
```

Time: O(1)

Space: O(1)

8. Authentication Middleware ★★ ★

Interviewer: “Create middleware to protect routes using JWT.”

```
import jwt from "jsonwebtoken";
```

```
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization?.split(" ")[1];  
  
  if (!token) {  
    return res.status(401).json({  
      message: "Unauthorized"  
    });  
  }  
}
```

```
try {  
  const decoded = jwt.verify(  
    token,  
    process.env.JWT_SECRET  
  );  
}
```

```

    token,
    process.env.JWT_SECRET
  );

  req.user = decoded;

  next();
} catch (error) {
  res.status(401).json({
    message: "Invalid token"
  });
}
}

export default authMiddleware;

```

Time: O(1)

Space: O(1)

9. Protected Route ★★

Interviewer: “Create a protected profile route.”

```

app.get(
  "/profile",
  authMiddleware,
  async (req, res) => {

    const user = await User.findById(
      req.user.userId
    );

    res.json(user);
  }
);

```

Yahan:

authMiddleware

pehle token verify karega.

Then:

```
req.user.userId
```

se user fetch hoga.

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

10. Check Duplicate Email ★★

Interviewer: “Prevent registration if email already exists.”

```
const existingUser = await User.findOne({
  email
});
```

```
if (existingUser) {
  return res.status(400).json({
    message: "Email already exists"
  });
}
```

Then user create karo.

Time: $O(n)$ without index, approximately $O(1)$ with email index

Space: $O(1)$

11. Logout ★★

JWT ka simple logout usually client-side token removal hota hai.

Interviewer: “Implement logout.”

```
app.post("/logout", (req, res) => {
  res.json({
    message: "Logout successful"
  });
});
```

Agar JWT **httpOnly cookie** mein stored hai, then cookie clear kar sakte ho:

```
res.clearCookie("token");

res.json({
  message: "Logout successful"
});
```

12. Role-Based Authorization ★★

Interviewer: “Allow only admins to access this route.”

```
function adminOnly(req, res, next) {
  if (req.user.role !== "admin") {
    return res.status(403).json({
      message: "Access denied"
    });
  }

  next();
}
```

Route:

```
app.get(
  "/admin",
  authMiddleware,
  adminOnly,
  (req, res) => {
    res.json({
      message: "Welcome Admin"
    });
  }
);
```

Time: O(1)

Space: O(1)

Authentication mein ye MUST DO

Placement ke liye sabse pehle ye **8 codes** perfectly kar lo:

1. ★★★★★ Register API
2. ★★★★★ Login API
3. ★★★★★ bcrypt hash
4. ★★★★★ bcrypt compare
5. ★★★★★ JWT generate
6. ★★★★★ JWT verify
7. ★★★★★ Auth middleware
8. ★★★★★ Protected route

Uske baad:

9. Duplicate email
10. Role-based authorization
11. Logout
12. Cookies

Next logical step: Complete MERN CRUD API — Model → Controller → Routes → Express server → MongoDB → JWT protected CRUD. Ye ekdum **real interview coding task** jaisa hoga.

Ha bta jldi

Haan yrr, **ab Complete MERN CRUD API** karte hain — ye interview mein sabse useful practical task hai.

Complete MERN CRUD — Interview Task

Interviewer:

> “Create a CRUD API using Node.js, Express, MongoDB and Mongoose.”

1. User Model

```
import mongoose from "mongoose";
```

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

export default mongoose.model("User", userSchema);
```

2. Create User — POST ★★★★★

```
app.post("/users", async (req, res) => {
  try {
    const user = await User.create(req.body);

    res.status(201).json(user);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: $O(1)$ average

Space: $O(1)$

3. Get All Users — GET ★★★★★

```
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();

    res.json(users);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: $O(n)$

Space: $O(n)$

4. Get Single User — GET ★★★★★

```
app.get("/users/:id", async (req, res) => {  
  try {  
    const user = await User.findById(req.params.id);  
  
    res.json(user);  
  } catch (error) {  
    res.status(500).json({  
      message: "Server error"  
    });  
  }  
});
```

Time: O(1) average with `_id` index

Space: O(1)

5. Update User — PUT ★★★★★

```
app.put("/users/:id", async (req, res) => {  
  try {  
    const user = await User.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true }  
    );  
  
    res.json(user);  
  } catch (error) {  
    res.status(500).json({  
      message: "Server error"  
    });  
  }  
});
```

Time: O(1) average with `_id` index

Space: O(1)

6. Delete User — DELETE ★★

```
app.delete("/users/:id", async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);

    res.json({
      message: "User deleted"
    });
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: O(1) average with `_id` index

Space: O(1)

★ Protected CRUD

Interviewer agar bole:

“Make the CRUD API accessible only to logged-in users.”

Bas middleware add:

```
app.get("/users", authMiddleware, async (req, res) => {
  const users = await User.find();

  res.json(users);
});
```

Same way POST, PUT, DELETE mein bhi:

```
app.post("/users", authMiddleware, ...);
app.put("/users/:id", authMiddleware, ...);
app.delete("/users/:id", authMiddleware, ...);
```

Ab MERN interview coding mein tumhare main topics

Must know:

- React basic coding
- React Hooks
- API calling
- Node.js basics
- Express server
- Middleware
- REST APIs
- MongoDB queries
- Mongoose
- CRUD
- bcrypt
- JWT
- Authentication middleware
- Protected routes
- Role-based authorization

Next: main tumhe **10 scenario-based MERN interview coding questions** dunga — jaise interviewer actual mein bolega: *“Login API banao”, “User ko admin-only delete karne do”, “Search API banao”*. Ye placement ke liye kaafi useful honge.

Nhi aur nhi mai fresher hu

Haan yrr 🥲 **fresher ho to itna hi kaafi hai**. Ab aur advanced MERN coding karne ki zarurat nahi.

Placement ke liye tumhari coding prep ko yahin tak rakho:

- **DSA:** Arrays, Strings, Searching, Sorting, HashMap, Stack/Queue, Recursion, Linked List, Basic Math, Patterns
- **React:** Counter, Toggle, Forms, Props, map, Search/Filter, Add/Delete, useEffect, API calling, Router, Context
- **Node/Express:** Server, routes, middleware, REST API, CRUD, error handling

- **MongoDB:** Basic queries, Mongoose, CRUD, operators, projection, populate, pagination
- **Authentication:** bcrypt, JWT, login/register, middleware, protected route

Bas inke codes khud se likhne ki practice karo. Fresher interview ke liye ab advanced aggregation, Redis, WebSockets, microservices etc. mein jaane ki zarurat nahi.

Ab tumhara focus **code yaad karna nahi, bina dekhe likhna** hona chahiye. 100

Yaare Jo tumhe upar array ke question kar rahe ho khaskar built in mein kafi jyada Tough kar rahe ho to acche se do yaar Jo ekadam badhiya Ho samajh mein bhi Aaye ya nahin Ho jaaye samajh Hi Ho jaaye sab kuchh

Haan yrr, **bilkul sahi bol rahi ho** 🤔 Pehle maine built-in wale kuch questions unnecessarily complicated bana diye the. Tumhe interview prep ke liye **simple, natural JavaScript** chahiye — aisa code jo dekhte hi samajh aaye aur interview mein khud likh pao.

Ab se Arrays ko **freshly, clean format** mein karenge:

Har question ka format

Interviewer: Find the largest element in an array.

1. Built-in allowed

```
let arr = [10, 5, 20, 8];

let largest = Math.max(...arr);

console.log(largest); // 20
```

Time: O(n)

Space: O(n) due to spread

2. Without built-in

```
let arr = [10, 5, 20, 8];

let largest = arr[0];

for (let i = 1; i < arr.length; i++) {
```

```
if (arr[i] > largest) {  
    largest = arr[i];  
}  
}  
  
console.log(largest); // 20
```

Time: $O(n)$

Space: $O(1)$

3. Optimized:

Same as without built-in — already **$O(n)$** time and **$O(1)$** space.

Aur built-in mein aise hi simple methods use honge:

- Reverse → `arr.reverse()`
- Sort → `arr.sort()`
- Search → `arr.indexOf()`
- Last occurrence → `arr.lastIndexOf()`
- Duplicate removal → `new Set(arr)`
- Maximum → `Math.max(...arr)`
- Minimum → `Math.min(...arr)`

Koi faltu reduce + filter + Set + map ka combination nahi. 😭

Aur agar built-in se question naturally solve nahi hota, to main zabardasti built-in approach **nahi banaunga**.

Sabse important

Main Arrays ke **25 questions dobara start se** isi easy format mein dunga:

Question → Built-in → Without built-in → Optimized → Output → Time/Space

Aur jahan optimized same hai, sirf likhunga:

Optimized: Same as without built-in. Already optimal.

Isse tumhe **samajh bhi aayega + khud se code bhi likhna aa jayega.** 100

Start sb ek sath built in bhi without builtin bhi and optimize bhi easy dena yrr
hard nhi yaad nhi hota itna sb kuch

Bilkul yrr ❤️ Ab Arrays ko ekdum fresh aur easy level par karte hain. Hard logic nahi. Tumhara goal hai **interview mein question dekhkar khud code likh paana**, na ki 25 algorithms ratna.

Rule: Built-in → simple built-in, Without built-in → simple loop, Optimized → sirf jab genuinely better ho.

● ARRAYS — 25 Most Important Questions

1. Find Largest Element ★★★★★

Question: Find the largest element in an array.

Built-in

```
let arr = [10, 5, 20, 8];

let largest = Math.max(...arr);

console.log(largest); // 20
```

Time: $O(n)$ | **Space:** $O(n)$

Without built-in

```
let arr = [10, 5, 20, 8];

let largest = arr[0];

for (let i = 1; i < arr.length; i++) {
  if (arr[i] > largest) {
    largest = arr[i];
  }
}

console.log(largest); // 20
```

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in — $O(n)$, $O(1)$.

2. Find Smallest Element ★★

Question: Find the smallest element in an array.

Built-in

```
let arr = [10, 5, 20, 8];

let smallest = Math.min(...arr);

console.log(smallest); // 5
```

Without built-in

```
let arr = [10, 5, 20, 8];

let smallest = arr[0];

for (let i = 1; i < arr.length; i++) {
  if (arr[i] < smallest) {
    smallest = arr[i];
  }
}

console.log(smallest); // 5
```

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in.

3. Find Second Largest ★★

Question: Find the second largest element.

Built-in

```
let arr = [10, 5, 20, 8];

arr.sort((a, b) => b - a);

console.log(arr[1]); // 10
```

Time: $O(n \log n)$

Without built-in

```
let arr = [10, 5, 20, 8];

let largest = -Infinity;
let second = -Infinity;

for (let num of arr) {
  if (num > largest) {
    second = largest;
    largest = num;
  } else if (num > second && num !== largest) {
    second = num;
  }
}

console.log(second); // 10
```

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in. Already $O(n)$.

4. Find Second Smallest ★★

Question: Find the second smallest element.

Built-in

```
let arr = [10, 5, 20, 8];

arr.sort((a, b) => a - b);
```

```
console.log(arr[1]); // 8
```

Without built-in

```
let arr = [10, 5, 20, 8];
```

```
let smallest = Infinity;
```

```
let second = Infinity;
```

```
for (let num of arr) {  
  if (num < smallest) {  
    second = smallest;  
    smallest = num;  
  } else if (num < second && num !== smallest) {  
    second = num;  
  }  
}
```

```
console.log(second); // 8
```

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in.

5. Reverse an Array ★★ ★

Question: Reverse an array.

Built-in

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.reverse();
```

```
console.log(arr);
```

```
// [5, 4, 3, 2, 1]
```

Time: $O(n)$ | **Space:** $O(1)$ in typical JS implementations

Without built-in

```
let arr = [1, 2, 3, 4, 5];
let result = [];

for (let i = arr.length - 1; i >= 0; i--) {
  result.push(arr[i]);
}

console.log(result);
// [5, 4, 3, 2, 1]
```

Time: $O(n)$ | **Space:** $O(n)$

Optimized

```
let arr = [1, 2, 3, 4, 5];

let start = 0;
let end = arr.length - 1;

while (start < end) {
  [arr[start], arr[end]] = [arr[end], arr[start]];

  start++;
  end--;
}

console.log(arr);
// [5, 4, 3, 2, 1]
```

Time: $O(n)$ | **Space:** $O(1)$

6. Check if Array is Sorted

Question: Check whether an array is sorted in ascending order.

Built-in

JavaScript mein iske liye koi simple direct built-in method nahi hai.

Without built-in

```
let arr = [1, 2, 3, 4, 5];

let sorted = true;

for (let i = 0; i < arr.length - 1; i++) {
  if (arr[i] > arr[i + 1]) {
    sorted = false;
    break;
  }
}

console.log(sorted); // true
```

Time: O(n) | **Space:** O(1)

Optimized

Same as without built-in.

7. Remove Duplicates ★★

Question: Remove duplicate elements from an array.

Built-in

```
let arr = [1, 2, 2, 3, 3, 4];

let result = [...new Set(arr)];

console.log(result);
// [1, 2, 3, 4]
```

Time: O(n) | **Space:** O(n)

Without built-in

```
let arr = [1, 2, 2, 3, 3, 4];
let result = [];

for (let num of arr) {
  if (!result.includes(num)) {
```

```
    result.push(num);  
  }  
}
```

```
console.log(result);  
// [1, 2, 3, 4]
```

Time: $O(n^2)$ | **Space:** $O(n)$

Optimized

```
let arr = [1, 2, 2, 3, 3, 4];
```

```
let result = [];  
let seen = {};
```

```
for (let num of arr) {  
  if (!seen[num]) {  
    result.push(num);  
    seen[num] = true;  
  }  
}
```

```
console.log(result);  
// [1, 2, 3, 4]
```

Time: $O(n)$ | **Space:** $O(n)$

8. Count Frequency

Question: Count how many times each element occurs.

Built-in

```
let arr = [1, 2, 2, 3, 1, 2];
```

```
let frequency = {};
```

```
arr.forEach(num => {  
  frequency[num] = (frequency[num] || 0) + 1;  
});
```

```
console.log(frequency);  
// { 1: 2, 2: 3, 3: 1 }
```

Time: $O(n)$ | **Space:** $O(n)$

Without built-in

```
let arr = [1, 2, 2, 3, 1, 2];  
let frequency = {};  
  
for (let num of arr) {  
  if (frequency[num]) {  
    frequency[num]++;  
  } else {  
    frequency[num] = 1;  
  }  
}
```

```
console.log(frequency);
```

Time: $O(n)$ | **Space:** $O(n)$

Optimized

Same as without built-in.

9. Find Missing Number ★★

Array contains numbers from 1 to n, one number missing.

Question: Find the missing number.

Built-in

```
let arr = [1, 2, 4, 5];  
  
for (let i = 1; i <= 5; i++) {  
  if (!arr.includes(i)) {  
    console.log(i); // 3  
    break;  
  }  
}
```

Time: $O(n^2)$

Without built-in

```
let arr = [1, 2, 4, 5];

for (let i = 1; i <= 5; i++) {
  let found = false;

  for (let j = 0; j < arr.length; j++) {
    if (arr[j] === i) {
      found = true;
      break;
    }
  }

  if (!found) {
    console.log(i); // 3
    break;
  }
}
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized ★

```
let arr = [1, 2, 4, 5];

let n = 5;

let total = n * (n + 1) / 2;

let sum = 0;

for (let num of arr) {
  sum += num;
}

console.log(total - sum); // 3
```

Time: $O(n)$ | **Space:** $O(1)$

10. Find Duplicate Number ★★

Question: Find a duplicate element.

Built-in

```
let arr = [1, 3, 4, 2, 2];

let duplicate = arr.find(
  (num, index) => arr.indexOf(num) !== index
);

console.log(duplicate); // 2
```

Without built-in

```
let arr = [1, 3, 4, 2, 2];

for (let i = 0; i < arr.length; i++) {
  for (let j = i + 1; j < arr.length; j++) {
    if (arr[i] === arr[j]) {
      console.log(arr[i]); // 2
      break;
    }
  }
}
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [1, 3, 4, 2, 2];

let count = {};

for (let num of arr) {
  count[num] = (count[num] || 0) + 1;
}

for (let num of arr) {
  if (count[num] > 1) {
    console.log(num); // 2
    break;
  }
}
```

```
}  
}
```

Time: $O(n)$ | **Space:** $O(n)$

11. Find All Duplicate Elements ★★

Question: Find all duplicate elements.

Built-in

```
let arr = [1, 2, 2, 3, 4, 4];  
  
let result = arr.filter(  
  (num, index) => arr.indexOf(num) !== index  
);  
  
console.log([...new Set(result)]);  
// [2, 4]
```

Without built-in

```
let arr = [1, 2, 2, 3, 4, 4];  
  
let result = [];  
  
for (let i = 0; i < arr.length; i++) {  
  for (let j = i + 1; j < arr.length; j++) {  
    if (arr[i] === arr[j]) {  
      result.push(arr[i]);  
      break;  
    }  
  }  
}  
  
console.log(result);  
// [2, 4]
```

Time: $O(n^2)$ | **Space:** $O(n)$

Optimized

```
let arr = [1, 2, 2, 3, 4, 4];

let count = {};
let result = [];

for (let num of arr) {
  count[num] = (count[num] || 0) + 1;
}

for (let num in count) {
  if (count[num] > 1) {
    result.push(Number(num));
  }
}

console.log(result);
// [2, 4]
```

Time: $O(n)$ | **Space:** $O(n)$

12. Intersection of Two Arrays ★★

Question: Find common elements of two arrays.

Built-in

```
let arr1 = [1, 2, 3, 4];
let arr2 = [2, 4, 6];

let result = arr1.filter(num => arr2.includes(num));

console.log(result);
// [2, 4]
```

Time: $O(n \times m)$

Without built-in

```
let arr1 = [1, 2, 3, 4];
let arr2 = [2, 4, 6];
```

```
let result = [];

for (let i = 0; i < arr1.length; i++) {
  for (let j = 0; j < arr2.length; j++) {
    if (arr1[i] === arr2[j]) {
      result.push(arr1[i]);
      break;
    }
  }
}

console.log(result);
// [2, 4]
```

Time: $O(n \times m)$ | **Space:** $O(n)$

Optimized

```
let arr1 = [1, 2, 3, 4];
let arr2 = [2, 4, 6];

let map = {};
let result = [];

for (let num of arr1) {
  map[num] = true;
}

for (let num of arr2) {
  if (map[num]) {
    result.push(num);
    map[num] = false;
  }
}

console.log(result);
// [2, 4]
```

Time: $O(n + m)$ | **Space:** $O(n)$

13. Union of Two Arrays ★★

Question: Combine two arrays without duplicates.

Built-in

```
let arr1 = [1, 2, 3];
let arr2 = [2, 3, 4];

let result = [...new Set([...arr1, ...arr2])];

console.log(result);
// [1, 2, 3, 4]
```

Time: $O(n + m)$ | **Space:** $O(n + m)$

Without built-in

```
let arr1 = [1, 2, 3];
let arr2 = [2, 3, 4];

let result = [];

for (let num of arr1) {
  if (!result.includes(num)) {
    result.push(num);
  }
}

for (let num of arr2) {
  if (!result.includes(num)) {
    result.push(num);
  }
}

console.log(result);
// [1, 2, 3, 4]
```

Time: $O((n + m)^2)$ | **Space:** $O(n + m)$

Optimized

```
let arr1 = [1, 2, 3];
let arr2 = [2, 3, 4];
```

```
let map = {};  
let result = [];  
  
for (let num of [...arr1, ...arr2]) {  
  if (!map[num]) {  
    result.push(num);  
    map[num] = true;  
  }  
}
```

```
console.log(result);  
// [1, 2, 3, 4]
```

Time: $O(n + m)$ | **Space:** $O(n + m)$

14. Move Zeros to End

Question: Move all zeros to the end.

Built-in

```
let arr = [0, 1, 0, 3, 12];  
  
let result = arr.filter(num => num !== 0);  
  
while (result.length < arr.length) {  
  result.push(0);  
}
```

```
console.log(result);  
// [1, 3, 12, 0, 0]
```

Time: $O(n)$ | **Space:** $O(n)$

Without built-in

```
let arr = [0, 1, 0, 3, 12];  
let result = [];  
  
for (let num of arr) {  
  if (num !== 0) {  
    result.push(num);  
  }  
}
```

```
    }  
  }  
  
  while (result.length < arr.length) {  
    result.push(0);  
  }  
  
  console.log(result);
```

Time: $O(n)$ | **Space:** $O(n)$

Optimized ★

```
let arr = [0, 1, 0, 3, 12];  
  
let index = 0;  
  
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] !== 0) {  
    [arr[index], arr[i]] = [arr[i], arr[index]];  
    index++;  
  }  
}  
  
console.log(arr);  
// [1, 3, 12, 0, 0]
```

Time: $O(n)$ | **Space:** $O(1)$

15. Move Negative Numbers to One Side ★★

Question: Move all negative numbers to the beginning.

Built-in

```
let arr = [1, -2, 3, -4, 5];  
  
let result = [  
  ...arr.filter(num => num < 0),  
  ...arr.filter(num => num >= 0)  
];
```

```
console.log(result);  
// [-2, -4, 1, 3, 5]
```

Time: $O(n)$ | **Space:** $O(n)$

Without built-in

```
let arr = [1, -2, 3, -4, 5];
```

```
let result = [];
```

```
for (let num of arr) {  
  if (num < 0) {  
    result.push(num);  
  }  
}
```

```
for (let num of arr) {  
  if (num >= 0) {  
    result.push(num);  
  }  
}
```

```
console.log(result);  
// [-2, -4, 1, 3, 5]
```

Time: $O(n)$ | **Space:** $O(n)$

Optimized

```
let arr = [1, -2, 3, -4, 5];
```

```
let index = 0;
```

```
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] < 0) {  
    [arr[index], arr[i]] = [arr[i], arr[index]];  
    index++;  
  }  
}
```

```
console.log(arr);
```

Time: $O(n)$ | **Space:** $O(1)$

16. Find Pair with Given Sum ★★

Question: Find two numbers whose sum is equal to the target.

Built-in

```
let arr = [2, 7, 11, 15];
let target = 9;

let result = arr.find(
  (num, i) => arr.includes(target - num, i + 1)
);

console.log(result);
```

Ye built-in version thoda confusing ho sakta hai, so **interview mein simple loop better hai.**

Without built-in

```
let arr = [2, 7, 11, 15];
let target = 9;

for (let i = 0; i < arr.length; i++) {
  for (let j = i + 1; j < arr.length; j++) {
    if (arr[i] + arr[j] === target) {
      console.log(arr[i], arr[j]);
    }
  }
}
```

// 2 7

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [2, 7, 11, 15];
let target = 9;

let seen = {};
```

```

for (let num of arr) {
  let needed = target - num;

  if (seen[needed]) {
    console.log(needed, num);
    break;
  }

  seen[num] = true;
}

// 2 7

```

Time: $O(n)$ | **Space:** $O(n)$

17. Majority Element ★★

Question: Find the element that appears more than $n/2$ times.

Built-in

```

let arr = [2, 2, 1, 2, 3, 2, 2];

let result = arr.find(
  num => arr.filter(x => x === num).length > arr.length / 2
);

console.log(result); // 2

```

Without built-in

```

let arr = [2, 2, 1, 2, 3, 2, 2];

for (let i = 0; i < arr.length; i++) {
  let count = 0;

  for (let j = 0; j < arr.length; j++) {
    if (arr[i] === arr[j]) {
      count++;
    }
  }
}

```

```
    if (count > arr.length / 2) {  
        console.log(arr[i]);  
        break;  
    }  
}
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [2, 2, 1, 2, 3, 2, 2];
```

```
let candidate = arr[0];  
let count = 1;
```

```
for (let i = 1; i < arr.length; i++) {  
    if (arr[i] === candidate) {  
        count++;  
    } else {  
        count--;  
    }  
}
```

```
if (count === 0) {  
    candidate = arr[i];  
    count = 1;  
}  
}
```

```
console.log(candidate); // 2
```

Time: $O(n)$ | **Space:** $O(1)$

18. Pair with Given Difference ★★

Question: Find two numbers whose difference is equal to the target.

Built-in

```
let arr = [5, 20, 3, 2, 50, 80];  
let target = 78;
```

```
let found = arr.some(num =>
  arr.includes(num + target)
);
```

```
console.log(found); // true
```

Time: $O(n^2)$

Without built-in

```
let arr = [5, 20, 3, 2, 50, 80];
```

```
let target = 78;
```

```
for (let i = 0; i < arr.length; i++) {
  for (let j = i + 1; j < arr.length; j++) {
    if (Math.abs(arr[i] - arr[j]) === target) {
      console.log(arr[i], arr[j]);
    }
  }
}
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [5, 20, 3, 2, 50, 80];
```

```
let target = 78;
```

```
let seen = {};
```

```
for (let num of arr) {
  if (seen[num - target] || seen[num + target]) {
    console.log("Pair found");
    break;
  }

  seen[num] = true;
}
```

Time: $O(n)$ | **Space:** $O(n)$

19. Maximum Consecutive 1s ★★

Question: Find the maximum number of consecutive 1s.

Built-in

```
let arr = [1, 1, 0, 1, 1, 1];

let count = 0;
let max = 0;

arr.forEach(num => {
  if (num === 1) {
    count++;
    max = Math.max(max, count);
  } else {
    count = 0;
  }
});

console.log(max); // 3
```

Without built-in

```
let arr = [1, 1, 0, 1, 1, 1];

let count = 0;
let max = 0;

for (let num of arr) {
  if (num === 1) {
    count++;

    if (count > max) {
      max = count;
    }
  } else {
    count = 0;
  }
}

console.log(max); // 3
```

Time: $O(n)$ | **Space:** $O(1)$

Optimized

Same as without built-in.

20. Maximum Subarray Sum ★★

Question: Find the maximum sum of a continuous subarray.

Built-in

```
let arr = [-2, 1, -3, 4, -1, 2, 1];

let max = -Infinity;

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];
    max = Math.max(max, sum);
  }
}

console.log(max); // 6
```

Without built-in

```
let arr = [-2, 1, -3, 4, -1, 2, 1];

let max = -Infinity;

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];

    if (sum > max) {
      max = sum;
    }
  }
}
```

```
}  
}
```

```
console.log(max); // 6
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized — Kadane

```
let arr = [-2, 1, -3, 4, -1, 2, 1];
```

```
let current = arr[0];
```

```
let max = arr[0];
```

```
for (let i = 1; i < arr.length; i++) {  
  current = Math.max(arr[i], current + arr[i]);
```

```
  if (current > max) {  
    max = current;  
  }  
}
```

```
console.log(max); // 6
```

Time: $O(n)$ | **Space:** $O(1)$

21. Best Time to Buy and Sell Stock ★★

Question: Find maximum profit from buying and selling a stock once.

Built-in

```
let prices = [7, 1, 5, 3, 6, 4];
```

```
let profit = 0;
```

```
for (let i = 0; i < prices.length; i++) {  
  for (let j = i + 1; j < prices.length; j++) {  
    profit = Math.max(  
      profit,  
      prices[j] - prices[i]  
    );  
  }  
}
```

```
    }  
  }  
  
  console.log(profit); // 5
```

Without built-in

```
let prices = [7, 1, 5, 3, 6, 4];  
  
let profit = 0;  
  
for (let i = 0; i < prices.length; i++) {  
  for (let j = i + 1; j < prices.length; j++) {  
    let currentProfit = prices[j] - prices[i];  
  
    if (currentProfit > profit) {  
      profit = currentProfit;  
    }  
  }  
}  
  
console.log(profit); // 5
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let prices = [7, 1, 5, 3, 6, 4];  
  
let minPrice = prices[0];  
let profit = 0;  
  
for (let i = 1; i < prices.length; i++) {  
  if (prices[i] < minPrice) {  
    minPrice = prices[i];  
  }  
  
  let currentProfit = prices[i] - minPrice;  
  
  if (currentProfit > profit) {  
    profit = currentProfit;  
  }  
}
```

```
console.log(profit); // 5
```

Time: $O(n)$ | **Space:** $O(1)$

22. Find Subarray with Given Sum ★★

Question: Find a continuous subarray whose sum is equal to the target.

Positive numbers ke case mein.

Without built-in

```
let arr = [1, 4, 20, 3, 10, 5];
let target = 33;

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];

    if (sum === target) {
      console.log("Subarray found");
    }
  }
}
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [1, 4, 20, 3, 10, 5];
let target = 33;

let start = 0;
let sum = 0;

for (let end = 0; end < arr.length; end++) {
  sum += arr[end];

  while (sum > target) {
```

```

    sum -= arr[start];
    start++;
}

if (sum === target) {
    console.log("Subarray found");
    break;
}
}

```

Time: $O(n)$ | **Space:** $O(1)$

23. Longest Subarray with Given Sum ★★

Question: Find the length of the longest subarray whose sum is equal to target.

Positive numbers ke case mein.

Without built-in

```

let arr = [1, 2, 3, 1, 1, 1, 1];
let target = 3;

let maxLength = 0;

for (let i = 0; i < arr.length; i++) {
    let sum = 0;

    for (let j = i; j < arr.length; j++) {
        sum += arr[j];

        if (sum === target) {
            let length = j - i + 1;

            if (length > maxLength) {
                maxLength = length;
            }
        }
    }
}

```

```
console.log(maxLength); // 3
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

```
let arr = [1, 2, 3, 1, 1, 1, 1];  
let target = 3;
```

```
let start = 0;  
let sum = 0;  
let maxLength = 0;
```

```
for (let end = 0; end < arr.length; end++) {  
    sum += arr[end];
```

```
    while (sum > target) {  
        sum -= arr[start];  
        start++;  
    }
```

```
    if (sum === target) {  
        let length = end - start + 1;  
  
        if (length > maxLength) {  
            maxLength = length;  
        }  
    }  
}
```

```
console.log(maxLength); // 3
```

Time: $O(n)$ | **Space:** $O(1)$

24. Rotate Array by K Positions

Question: Rotate an array to the right by k positions.

Example:

[1,2,3,4,5], k = 2

Output:

[4,5,1,2,3]

Built-in

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

let result = arr.slice(-k).concat(arr.slice(0, -k));

console.log(result);
// [4, 5, 1, 2, 3]
```

Time: $O(n)$ | **Space:** $O(n)$

Without built-in

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

let result = [];

for (let i = arr.length - k; i < arr.length; i++) {
  result.push(arr[i]);
}

for (let i = 0; i < arr.length - k; i++) {
  result.push(arr[i]);
}

console.log(result);
// [4, 5, 1, 2, 3]
```

Time: $O(n)$ | **Space:** $O(n)$

Optimized

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

k = k % arr.length;

arr.reverse();
```



```

let start = 0;
let end = k - 1;

while (start < end) {
  [arr[start], arr[end]] = [arr[end], arr[start]];
  start++;
  end--;
}

start = k;
end = arr.length - 1;

while (start < end) {
  [arr[start], arr[end]] = [arr[end], arr[start]];
  start++;
  end--;
}

console.log(arr);
// [4, 5, 1, 2, 3]

```

Time: $O(n)$ | **Space:** $O(1)$

25. Find Kth Smallest / Largest ★★

Question: Find the kth smallest element.

Example:

[7, 10, 4, 3, 20, 15], k = 3

Answer = 7

Built-in

```

let arr = [7, 10, 4, 3, 20, 15];
let k = 3;

arr.sort((a, b) => a - b);

console.log(arr[k - 1]); // 7

```

Time: $O(n \log n)$ | **Space:** depends on JS sort implementation

Without built-in

```
let arr = [7, 10, 4, 3, 20, 15];
let k = 3;

for (let i = 0; i < arr.length; i++) {
  for (let j = i + 1; j < arr.length; j++) {
    if (arr[i] > arr[j]) {
      [arr[i], arr[j]] = [arr[j], arr[i]];
    }
  }
}

console.log(arr[k - 1]); // 7
```

Time: $O(n^2)$ | **Space:** $O(1)$

Optimized

Yahan **Quick Select** aata hai, but honestly fresher/service-based placement ke liye ye comparatively advanced hai.

👉 **Abhi ise ratne ki zarurat nahi.**

Interview mein agar kth smallest aaye to pehle **sort + arr[k-1]** wala solution confidently likhna enough hai.

❤️ Ab tumhe kya yaad karna hai?

Sab 25 ke 3 versions ratne ki bilkul zarurat nahi hai.

🔥 **Pehle ye pakke karo:**

1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 14, 16, 19, 20, 21, 24

Aur optimization ke liye sirf ye concepts samajh lo:

- Missing Number → **Sum formula**
- Move Zeros → **Two pointer**
- Pair Sum → **HashMap**
- Majority → **Boyer-Moore**

- Maximum Subarray → **Kadane**
- Stock Profit → **Minimum price track**
- Subarray Sum → **Sliding Window**
- Rotate Array → **Reverse**

Baaki basic solution se bhi interview clear ho sakta hai, especially fresher/service-based roles mein. Tumhe code ko *samajh ke likhna* hai, 75 different tricks yaad nahi karni. 100

Yrr mujhe to sbse hard aaray hi lgrr ab kya kru itna tough

Arre yrr 🤔 **tension mat lo**. Actually problem Arrays nahi hai — problem ye hai ki humne **ek saath 25 questions + 3 approaches** de diye. Naturally dimaag overload hoga.

Tum **abhi 25 arrays mat karo**. Placement ke liye pehle sirf **10 super-easy array questions** karo.

🟢 Phase 1 — Bas ye 10

1. Largest element
2. Smallest element
3. Reverse array
4. Check sorted
5. Sum of array
6. Count even/odd
7. Search an element
8. Count frequency
9. Remove duplicates
10. Move zeros to end

In 10 ko pehle bina dekhe likhna seekho.

Phir Phase 2

11. Second largest
12. Missing number

13. Duplicate number

14. Pair sum

15. Intersection

Phir hi Phase 3

16. Majority element

17. Maximum subarray

18. Stock profit

19. Subarray sum

20. Rotate array

Baaki 5 questions **optional/advanced** rakhenge.

Aur ek important change

Ab main tumhe **har question ke 3 versions ek saath nahi ratwaunga.**

Pehle:

Question → easiest code → output → complexity

Phir agar question samajh aa gaya, tab:

“Built-in kya hoga?”

“Without built-in kya hoga?”

“Optimized kya hoga?”

Isse tumhara dimaag confuse nahi hoga.

Aur honestly, **fresher interview ke liye har question ke 3 codes yaad karna zaroori nahi hai.** Tumhe ek basic solution confidently likhna aana chahiye, aur common optimized solutions ka idea hona chahiye.

Hum Arrays ko zero se, bahut easy level se dobara karenge. Pehle 10 easy questions hi. 🍷

Dekho Sach batao na problem mein yah hai ki sem question humne Kisi aur method se bhi kiye Hain like Jo frequency count Hai vah To humne string mein bhi dekha hai theek hai job intersection Hai vah humne hash map mein bhi dekha hai majority element bhi aisa lag raha hai same question ke bahut sare solutions ho gaye hain yahi problem hai to mujhe aise Hi karke batao na ki mere sare question answer cover Ho jaaye bahut acche se matlab samajh rahe ho na kyunki ek hi question Ham तरह-तरह se kar rahe hain sem question mujhe string mein bhi dekhne ka mil raha hai question hash map mein bhi sem chij Ko Tum optimise bhi Bata rahe ho Ajeeb Sa ho raha hai

Haan yrr, **ab tumhari actual problem samajh aa gayi**. Aur honestly, tum bilkul sahi bol rahi ho. 😭

Problem ye nahi hai ki questions zyada hain. Problem ye hai ki **same logic ko alag-alag topics mein baar-baar ghuma rahe hain**:

- Frequency → Array mein
- Frequency → String mein
- Frequency → HashMap mein
- Intersection → Array mein
- Intersection → HashMap mein
- Majority → Array mein
- Majority → HashMap mein
- Phir same question ka built-in
- Phir without built-in
- Phir optimized

Isse **questions cover nahi ho rahe, bas same concepts repeat ho rahe hain**.

Ab hum strategy change karenge ✅

Hum **question ko nahi, concept ko ek baar master karenge**.

Example:

Frequency Count ko ek hi baar properly karenge:

```
let arr = [1, 2, 2, 3, 3, 3];
```

```
let freq = {};
```

```
for (let num of arr) {
  if (freq[num]) {
    freq[num]++;
  } else {
    freq[num] = 1;
  }
}
```

```
console.log(freq);
```

Iska concept samajh gaya →
baad mein **String Frequency** aaye to bas:

```
for (let char of str)
```

karna hai.

Naya question nahi maana jayega. ✓

Similarly:

Concept	Ek baar seekhenge	Baad mein
Frequency	HashMap/Counting	Array + String dono cover
Duplicate	Hashing	Array/String variations
Intersection	Hashing	Array variation
Two Sum	Hashing	Array variation
Majority	Hashing	Array variation
Search	Linear/Binary	Array
Reverse	Loop	String + Array
Sorting	Sorting algorithms	Array
Palindrome	Two pointer	String
Max/Min	Traversal	Array
Sum/Count	Traversal	Array/String
Subarray	Sliding/Kadane	Sirf jab zarurat ho

Aur **optimization ka bhi rule change** 🚨

Har question ke 3 solutions **nahi** karenge.

Hum sirf:

1. Interview mein yaad rehne wala easiest solution

↓

2. Uski complexity

↓

3. Agar genuinely important optimized approach hai, tabhi optimized

Bas.

Tumhare placement ke liye final approach

Main tumhe **duplicate questions hata kar ek consolidated DSA list** banaunga.

Matlab agar:

Frequency Count Array mein aa gaya

to String mein dobara usko **full question** nahi karenge.

Bas bolenge:

“Ye same frequency concept hai jo hum Array mein kar chuke hain;
yahan array ki jagah string characters hain.”

Isi tarah HashMap ko bhi **alag 20 questions ka burden nahi banayenge**.

Goal: tumhare paas 100–150 naam ke questions nahi, balki **50–60 actual reusable patterns** hone chahiye jinse interview ke bahut saare questions solve ho jayein.

Aur haan — **Arrays ko bhi isi consolidated approach se restart karenge**.

Tumhe same cheez 3 jagah yaad nahi karni padegi.

Yes aise hi jldi hojaayega kaam bhi now start per mujhe teenon method chahie built in bhi without built in bhi aur optimised bhi lekin han कुछ-कुछ question mein ho jata hai na aise ki man Lo ki Ham sem question ko hashmat se bhi kar rahe hain aur vahi agar hamara optimised Hai To Tum mujhe Bata Dena ki vah jo humne pahle aise wala question kiya hai uske andar Jo solution Hai vahi yahan per bhi sem solution use karna hai theek hai ya Jo without hai agar uska code

pahle wale Se Kisi Se match Karta Hai To vah Hai agar Mera solution pahle Se Hi banaa hua Hai To Main usko use karna chahungi lekin chahie teeno kyuki interviewer kuch bhi puch skta h

Bilkul. Ab exactly isi strategy se karenge. Ye tumhare liye best rahega.

Har **unique question/concept** mein main 3 methods dunga:

1 Built-in

Sabse easy JS method jo interview mein allowed ho.

2 Without built-in

Simple loop/logic wala code — **ye tumhara main backup solution** rahega.

3 Optimized

Sirf jab **actual improvement** ho. Aur agar optimized solution **pehle kisi question mein already kar chuke hain**, to main dobara code nahi likhwaunga.

Example:

Frequency Count

- Array mein HashMap wala solution kar liya.
- String mein same concept aaya → **“same HashMap/frequency code, bas num ki jagah char.”**
- Dobara pura code ratne ki zarurat nahi.

Intersection

- HashMap/Set wala solution pehle kar liya.
- Kisi aur topic mein intersection aaye → **same solution reuse karna hai.**
- Main clearly likhunga: **“PAHLE WALA SOLUTION USE KARO.”**

★ Main har question ke saath ye bhi bataunga:

Reuse: Ye solution Question #X mein already kar chuke ho.

New: Isme naya concept hai, isliye ye code yaad karo.

Optimized: Ye genuinely better hai; isko alag se yaad rakhna useful hai.

Isse tumhare **3 methods bhi cover ho jayenge**, interviewer agar koi bhi approach maange to tum ready rahogi, **lekin same code 4 baar nahi padhna padega**.

Final rule 🔥

Question repeat ho sakta hai → solution repeat nahi karna.

Aur agar **same code/logic** pehle se tumhare paas hai → wahi reuse karna hai.

Ab Arrays ko bhi isi system se karenge: **easy → built-in → without built-in → optimized → reuse/reference**, bina unnecessary duplication ke.

Start

Chalo yrr **Arrays ko proper consolidated way mein start karte hain.**

Pehle **10 basic questions** karenge. Har question mein **Built-in + Without Built-in + Optimized** dunga, aur jahan previous solution reuse ho sakta hai, clearly bataunga.

● ARRAY — PART 1

1. Find Largest Element

Array: [10, 5, 20, 8]

Output: 20

✓ Built-in

```
let arr = [10, 5, 20, 8];
```

```
console.log(Math.max(...arr));
```

✓ Without built-in

```
let arr = [10, 5, 20, 8];
```

```
let largest = arr[0];
```

```
for (let i = 1; i < arr.length; i++) {  
  if (arr[i] > largest) {
```

```
    largest = arr[i];  
  }  
}
```

```
console.log(largest);
```

Optimized

Without built-in wala hi optimal hai.

- Time: $O(n)$
 - Space: $O(1)$
-

2. Find Smallest Element

Array: [10, 5, 20, 8]

Output: 5

Built-in

```
let arr = [10, 5, 20, 8];
```

```
console.log(Math.min(...arr));
```

Without built-in

```
let arr = [10, 5, 20, 8];
```

```
let smallest = arr[0];
```

```
for (let i = 1; i < arr.length; i++) {  
  if (arr[i] < smallest) {  
    smallest = arr[i];  
  }  
}
```

```
console.log(smallest);
```

Optimized

Same as without built-in — already optimal.

$O(n)$ time, $O(1)$ space.

 **Reuse:** Largest wala exact same logic hai. Bas > ki jagah <.

3. Reverse an Array

Array: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

✓ Built-in

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.reverse();
```

```
console.log(arr);
```

✓ Without built-in

```
let arr = [1, 2, 3, 4, 5];
```

```
let result = [];
```

```
for (let i = arr.length - 1; i >= 0; i--) {  
  result.push(arr[i]);  
}
```

```
console.log(result);
```

🚀 Optimized

Two-pointer:

```
let arr = [1, 2, 3, 4, 5];
```

```
let left = 0;
```

```
let right = arr.length - 1;
```

```
while (left < right) {  
  let temp = arr[left];  
  arr[left] = arr[right];  
  arr[right] = temp;
```

```
  left++;
```

```
    right--;  
}
```

```
console.log(arr);
```

- Time: $O(n)$
- Space: $O(1)$

Ye optimized genuinely important hai, kyunki extra result array nahi ban raha.

4. Check if Array is Sorted

Array: [1, 2, 3, 4, 5]

Output: true

✓ Built-in

```
let arr = [1, 2, 3, 4, 5];
```

```
let sorted = arr.every((value, index) => {  
    return index === 0 || arr[index - 1] <= value;  
});
```

```
console.log(sorted);
```

✓ Without built-in

```
let arr = [1, 2, 3, 4, 5];
```

```
let sorted = true;
```

```
for (let i = 0; i < arr.length - 1; i++) {  
    if (arr[i] > arr[i + 1]) {  
        sorted = false;  
        break;  
    }  
}
```

```
console.log(sorted);
```

Optimized

Same without-built-in solution.

O(n) time, O(1) space.

 **Pattern:** Har element ko uske **next element** se compare karna hai.

5. Find Sum of Array

Array: [10, 20, 30, 40]

Output: 100

Built-in

```
let arr = [10, 20, 30, 40];
```

```
let sum = arr.reduce((total, num) => total + num, 0);
```

```
console.log(sum);
```

Without built-in

```
let arr = [10, 20, 30, 40];
```

```
let sum = 0;
```

```
for (let num of arr) {  
  sum += num;  
}
```

```
console.log(sum);
```

Optimized

Same without-built-in solution — O(n) and O(1).

 **reduce()** yaad na bhi rahe to tension nahi. **Loop wala interview mein perfectly fine hai.**

6. Count Even and Odd Numbers

Array: [1, 2, 3, 4, 6]

Output:

Even = 3

Odd = 2

✓ Built-in

```
let arr = [1, 2, 3, 4, 6];
```

```
let even = arr.filter(num => num % 2 === 0).length;
```

```
let odd = arr.filter(num => num % 2 !== 0).length;
```

```
console.log(even, odd);
```

✓ Without built-in

```
let arr = [1, 2, 3, 4, 6];
```

```
let even = 0;
```

```
let odd = 0;
```

```
for (let num of arr) {  
  if (num % 2 === 0) {  
    even++;  
  } else {  
    odd++;  
  }  
}
```

```
console.log(even, odd);
```

🚀 Optimized

Without-built-in wala better hai.

Kyuki built-in solution mein array ko 2 baar traverse kar rahe hain.

Without-built-in:

- Time: O(n)
- Space: O(1)

7. Search an Element

Array: [10, 20, 30, 40]

Target: 30

✓ Built-in

```
let arr = [10, 20, 30, 40];
```

```
console.log(arr.includes(30));
```

Output:

```
true
```

✓ Without built-in

```
let arr = [10, 20, 30, 40];
```

```
let found = false;
```

```
for (let num of arr) {  
  if (num === 30) {  
    found = true;  
    break;  
  }  
}
```

```
console.log(found);
```

🚀 Optimized

Agar array **sorted** hai, Binary Search use kar sakte hain:

```
let arr = [10, 20, 30, 40, 50];
```

```
let target = 30;
```

```
let left = 0;
```

```
let right = arr.length - 1;
```

```
while (left <= right) {  
  let mid = Math.floor((left + right) / 2);
```

```

if (arr[mid] === target) {
  console.log("Found");
  break;
}

if (arr[mid] < target) {
  left = mid + 1;
} else {
  right = mid - 1;
}
}

```

- Linear Search → $O(n)$
- Binary Search → $O(\log n)$

 **Reuse:** Binary Search hum Searching section mein properly karenge. Isliye abhi iska code ratne ki zarurat nahi.

8. Frequency Count

Array: [1, 2, 2, 3, 3, 3]

Output:

```

1 → 1
2 → 2
3 → 3

```

 **Built-in**

```
let arr = [1, 2, 2, 3, 3, 3];
```

```
let result = {};
```

```
arr.forEach(num => {
  result[num] = (result[num] || 0) + 1;
});
```

```
console.log(result);
```


✓ Without built-in

```
let arr = [1, 2, 2, 3, 3, 3];
```

```
let frequency = {};
```

```
for (let num of arr) {  
  if (frequency[num]) {  
    frequency[num]++;  
  } else {  
    frequency[num] = 1;  
  }  
}
```

```
console.log(frequency);
```

🚀 Optimized

Same HashMap/object approach — already optimal.

- Time: $O(n)$
- Space: $O(n)$

★ VERY IMPORTANT REUSE

Ye concept ab dobara full nahi padhna.

String mein agar question aaye:

Count frequency of characters

to **SAME CODE** use hoga:

```
for (let char of str)
```

Bas num → char.

HashMap section mein bhi frequency aaye to **YEHI CONCEPT reuse karna hai.**

9. Remove Duplicates

Array: [1, 2, 2, 3, 3, 4]

Output:

[1, 2, 3, 4]

✓ Built-in

```
let arr = [1, 2, 2, 3, 3, 4];
```

```
let result = [...new Set(arr)];
```

```
console.log(result);
```

✓ Without built-in

```
let arr = [1, 2, 2, 3, 3, 4];
```

```
let result = [];
```

```
for (let num of arr) {  
  if (!result.includes(num)) {  
    result.push(num);  
  }  
}
```

```
console.log(result);
```

🚀 Optimized

HashMap/Set approach:

```
let arr = [1, 2, 2, 3, 3, 4];
```

```
let seen = {};
```

```
let result = [];
```

```
for (let num of arr) {  
  if (!seen[num]) {  
    seen[num] = true;  
    result.push(num);  
  }  
}
```

```
console.log(result);
```

- Time: O(n)
- Space: O(n)

 **Reuse:** Iska HashMap/Set concept future mein duplicate questions mein reuse hoga. Dobara naya logic nahi.

10. Move All Zeros to End

Array:

```
[0, 1, 0, 3, 12]
```

Output:

```
[1, 3, 12, 0, 0]
```

 **Built-in**

```
let arr = [0, 1, 0, 3, 12];
```

```
let result = arr.filter(num => num !== 0);
```

```
while (result.length < arr.length) {  
  result.push(0);  
}
```

```
console.log(result);
```

 **Without built-in**

```
let arr = [0, 1, 0, 3, 12];
```

```
let result = [];
```

```
for (let num of arr) {  
  if (num !== 0) {  
    result.push(num);  
  }  
}
```

```
while (result.length < arr.length) {  
  result.push(0);  
}
```

```
console.log(result);
```

Optimized — Two Pointer

```
let arr = [0, 1, 0, 3, 12];
```

```
let index = 0;
```

```
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] !== 0) {  
    arr[index] = arr[i];  
    index++;  
  }  
}
```

```
while (index < arr.length) {  
  arr[index] = 0;  
  index++;  
}
```

```
console.log(arr);
```

- Time: $O(n)$
- Space: $O(1)$

Ye optimized wala important hai.

Ab tak ka REUSE MAP

Ye sabse important part hai 👉

Question	Kya yaad karna hai	Future mein reuse
Largest	Simple traversal	Max problems
Smallest	Largest ka same logic	Min problems
Reverse	Two pointer	String reverse etc.
Sorted	Adjacent comparison	Sorted checking
Sum	Simple loop	Many counting/sum problems

Question	Kya yaad karna hai	Future mein reuse
Even/Odd	% 2	Number problems
Search	Linear Search	Searching
Frequency	HashMap/Object	★ String + HashMap
Remove Duplicate	Set/HashMap	★ Duplicate problems
Move Zeros	Two pointer	Similar rearrangement

🔥 Sabse important baat

Ab hum **Frequency ko Array + String + HashMap** mein 3 alag questions nahi maanenge.

Same for:

- Duplicate
- Intersection
- Majority
- Two Sum
- Reverse
- Palindrome
- Search

Ek concept master → jahan-jahan apply hota hai, wahan reuse.

Isse tumhara syllabus literally **kaafi chhota** ho jayega, aur interview ke liye preparation bhi strong rahegi.

Nice now next buddy

Chalo buddy 🤙 **Array Part 2** — ab next important questions. Same rule: **Built-in + Without built-in + Optimized**, aur jahan purana solution reuse hoga, main clearly bataunga.

● ARRAY — PART 2

11. Find Second Largest Element

Array: [10, 5, 20, 8, 15]

Output: 15

✓ Built-in

```
let arr = [10, 5, 20, 8, 15];
```

```
let result = [...new Set(arr)];  
result.sort((a, b) => b - a);
```

```
console.log(result[1]);
```

✓ Without built-in

```
let arr = [10, 5, 20, 8, 15];
```

```
let largest = -Infinity;
```

```
let second = -Infinity;
```

```
for (let num of arr) {  
  if (num > largest) {  
    second = largest;  
    largest = num;  
  } else if (num > second && num !== largest) {  
    second = num;  
  }  
}
```

```
console.log(second);
```

🚀 Optimized

Without-built-in wala hi optimized hai.

- Time: O(n)
- Space: O(1)

🔄 **Reuse:** Largest wala concept directly use hua — bas ab second bhi maintain karna hai.

12. Find Missing Number

Array contains numbers from 1 to n, one number missing.

[1, 2, 3, 5]

Output: 4

✓ Built-in

```
let arr = [1, 2, 3, 5];
let n = arr.length + 1;

let total = (n * (n + 1)) / 2;
let sum = arr.reduce((a, b) => a + b, 0);

console.log(total - sum);
```

✓ Without built-in

```
let arr = [1, 2, 3, 5];
let n = arr.length + 1;

let total = 0;

for (let i = 1; i <= n; i++) {
  total += i;
}

let sum = 0;

for (let num of arr) {
  sum += num;
}

console.log(total - sum);
```

🚀 Optimized

Same mathematical approach, but we can do it in one loop:

```
let arr = [1, 2, 3, 5];
let n = arr.length + 1;

let total = (n * (n + 1)) / 2;
```

```
for (let num of arr) {  
  total -= num;  
}
```

```
console.log(total);
```

- Time: O(n)
- Space: O(1)

 **New concept:** $1 + 2 + \dots + n = n(n+1)/2$

13. Find Duplicate Element

[1, 2, 3, 2, 4]

Output: 2

✓ Built-in

```
let arr = [1, 2, 3, 2, 4];
```

```
let seen = new Set();
```

```
for (let num of arr) {  
  if (seen.has(num)) {  
    console.log(num);  
    break;  
  }  
}
```

```
seen.add(num);  
}
```

✓ Without built-in

```
let arr = [1, 2, 3, 2, 4];
```

```
for (let i = 0; i < arr.length; i++) {  
  for (let j = i + 1; j < arr.length; j++) {  
    if (arr[i] === arr[j]) {  
      console.log(arr[i]);  
      break;  
    }  
  }  
}
```



```
}  
}
```

Optimized

HashMap/Object:

```
let arr = [1, 2, 3, 2, 4];
```

```
let seen = {};
```

```
for (let num of arr) {  
  if (seen[num]) {  
    console.log(num);  
    break;  
  }  
}
```

```
seen[num] = true;  
}
```

- Time: O(n)
- Space: O(n)

★ VERY IMPORTANT REUSE:

Ye **Remove Duplicates** wale HashMap/Set concept ka hi use hai.
Dobara naya concept nahi hai.

14. Find Pair With Given Sum — Two Sum

Array = [2, 7, 11, 15]

Target = 9

Output: 2 + 7 = 9

Built-in

```
let arr = [2, 7, 11, 15];
```

```
let target = 9;
```

```
for (let i = 0; i < arr.length; i++) {  
  let j = arr.indexOf(target - arr[i], i + 1);
```

```
  if (j !== -1) {
```

```

        console.log(arr[i], arr[j]);
        break;
    }
}

```

✅ Without built-in

```

let arr = [2, 7, 11, 15];
let target = 9;

for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
        if (arr[i] + arr[j] === target) {
            console.log(arr[i], arr[j]);
        }
    }
}

```

- Time: $O(n^2)$
- Space: $O(1)$

🚀 Optimized — HashMap

```

let arr = [2, 7, 11, 15];
let target = 9;

let seen = {};

for (let num of arr) {
    let needed = target - num;

    if (seen[needed]) {
        console.log(needed, num);
        break;
    }

    seen[num] = true;
}

```

- Time: $O(n)$
- Space: $O(n)$

★ **REUSE**: seen wala HashMap concept **Duplicate Element** mein abhi kiya hai.

Bas yahan seen ka use duplicate check ke liye nahi, **required number already mila hai ya nahi** check karne ke liye hai.

15. Find Intersection of Two Arrays

```
A = [1, 2, 3, 4]
```

```
B = [3, 4, 5, 6]
```

Output:

```
[3, 4]
```

✓ Built-in

```
let a = [1, 2, 3, 4];
```

```
let b = [3, 4, 5, 6];
```

```
let result = a.filter(num => b.includes(num));
```

```
console.log(result);
```

✓ Without built-in

```
let a = [1, 2, 3, 4];
```

```
let b = [3, 4, 5, 6];
```

```
let result = [];
```

```
for (let i = 0; i < a.length; i++) {  
  for (let j = 0; j < b.length; j++) {  
    if (a[i] === b[j]) {  
      result.push(a[i]);  
      break;  
    }  
  }  
}
```

```
console.log(result);
```

🚀 Optimized — HashMap

```
let a = [1, 2, 3, 4];
```

```
let b = [3, 4, 5, 6];
```

```
let seen = {};  
let result = [];  
  
for (let num of a) {  
  seen[num] = true;  
}  
  
for (let num of b) {  
  if (seen[num]) {  
    result.push(num);  
  }  
}  
  
console.log(result);
```

- Time: $O(n + m)$
- Space: $O(n)$

★ **SUPER IMPORTANT REUSE:**

Ye wahi **HashMap seen concept** hai jo Duplicate aur Two Sum mein kiya. **HashMap section mein Intersection aaye to isi solution ko use karna hai. Dobara mat padhna.**

Part 2 — Kya actually yaad karna hai?

New concepts:

- Second Largest $\rightarrow$ largest + second
- Missing Number $\rightarrow$ sum formula
- Two Sum $\rightarrow$ HashMap
- Intersection $\rightarrow$ HashMap

Already-known concepts reused:

- Duplicate $\rightarrow$ seen / HashMap
- Intersection $\rightarrow$ seen / HashMap
- Two Sum $\rightarrow$ seen / HashMap
- Second Largest $\rightarrow$ Largest traversal

So technically **6 questions dikh rahe hain, but new logic sirf ~4 hai.** 🔥

Next mein hum **Majority Element, Maximum Subarray, Stock Profit, Subarray Sum aur Rotate Array** karenge — aur wahan bhi duplicate concepts ko repeat nahi karenge.

Superb next

Chalo buddy 🔥 **Array Part 3** — ab remaining important questions. Same rule: **Built-in + Without built-in + Optimized**, aur reuse clearly mark karunga.

● ARRAY — PART 3

16. Majority Element

Array mein jo element **$n/2$ se zyada times** aaye.

[2, 2, 1, 1, 1, 2, 2]

Output: 2

✓ Built-in

```
let arr = [2, 2, 1, 1, 1, 2, 2];

let result = arr.find(num =>
  arr.filter(x => x === num).length > arr.length / 2
);

console.log(result);
```

✓ Without built-in

```
let arr = [2, 2, 1, 1, 1, 2, 2];

for (let i = 0; i < arr.length; i++) {
  let count = 0;

  for (let j = 0; j < arr.length; j++) {
    if (arr[i] === arr[j]) {
      count++;
    }
  }
}
```

```
if (count > arr.length / 2) {  
  console.log(arr[i]);  
  break;  
}  
}
```

Optimized — Boyer-Moore

```
let arr = [2, 2, 1, 1, 1, 2, 2];
```

```
let candidate = 0;
```

```
let count = 0;
```

```
for (let num of arr) {  
  if (count === 0) {  
    candidate = num;  
  }  
}
```

```
if (num === candidate) {  
  count++;  
} else {  
  count--;  
}  
}
```

```
console.log(candidate);
```

- Time: $O(n)$
- Space: $O(1)$

★ **New important optimized concept:** Boyer-Moore.

Isko abhi sirf concept ke saath yaad karo: **same** → +1, **different** → -1,
count 0 → **new candidate**.

17. Maximum Subarray Sum

```
[-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

Maximum sum = 6

Because:

```
[4, -1, 2, 1] = 6
```

✓ Built-in

Simple approach:

```
let arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4];

let maxSum = -Infinity;

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];
    maxSum = Math.max(maxSum, sum);
  }
}

console.log(maxSum);
```

✓ Without built-in

```
let arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4];

let maxSum = arr[0];

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];

    if (sum > maxSum) {
      maxSum = sum;
    }
  }
}

console.log(maxSum);
```

- Time: $O(n^2)$
- Space: $O(1)$

Optimized — Kadane's Algorithm

```
let arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4];
```

```
let current = arr[0];
```

```
let maxSum = arr[0];
```

```
for (let i = 1; i < arr.length; i++) {  
  current = Math.max(arr[i], current + arr[i]);  
  maxSum = Math.max(maxSum, current);  
}
```

```
console.log(maxSum);
```

- Time: $O(n)$
- Space: $O(1)$

★ **New concept:** Kadane's Algorithm.

Basic idea: **current sum negative ho raha hai** → **previous sum chhod do.**

18. Best Time to Buy and Sell Stock

Prices:

```
[7, 1, 5, 3, 6, 4]
```

Buy at 1, sell at 6.

Profit = 5

✓ Built-in

```
let prices = [7, 1, 5, 3, 6, 4];
```

```
let maxProfit = 0;
```

```
for (let i = 0; i < prices.length; i++) {  
  for (let j = i + 1; j < prices.length; j++) {  
    maxProfit = Math.max(maxProfit, prices[j] - prices[i]);  
  }  
}
```



```
console.log(maxProfit);
```

✓ Without built-in

```
let prices = [7, 1, 5, 3, 6, 4];
```

```
let maxProfit = 0;
```

```
for (let i = 0; i < prices.length; i++) {  
  for (let j = i + 1; j < prices.length; j++) {  
    let profit = prices[j] - prices[i];  
  
    if (profit > maxProfit) {  
      maxProfit = profit;  
    }  
  }  
}
```

```
console.log(maxProfit);
```

🚀 Optimized

```
let prices = [7, 1, 5, 3, 6, 4];
```

```
let minPrice = prices[0];
```

```
let maxProfit = 0;
```

```
for (let i = 1; i < prices.length; i++) {  
  
  if (prices[i] < minPrice) {  
    minPrice = prices[i];  
  }  
  
  let profit = prices[i] - minPrice;  
  
  if (profit > maxProfit) {  
    maxProfit = profit;  
  }  
}
```

```
console.log(maxProfit);
```

- Time: $O(n)$

- Space: $O(1)$

 **Logic:** ab tak ka smallest price rakho → current price pe profit calculate karo.

19. Subarray With Given Sum

Positive numbers ke array mein target sum find karna.

Array = [1, 4, 20, 3, 10, 5]

Target = 33

$20 + 3 + 10 = 33$

Built-in

```
let arr = [1, 4, 20, 3, 10, 5];
```

```
let target = 33;
```

```
let found = false;
```

```
for (let i = 0; i < arr.length; i++) {  
  let sum = arr.slice(i, arr.length);
```

```
  let total = 0;
```

```
  for (let num of sum) {  
    total += num;
```

```
    if (total === target) {  
      found = true;  
      break;  
    }  
  }  
}
```

```
if (found) break;  
}
```

```
console.log(found);
```

✓ Without built-in

```
let arr = [1, 4, 20, 3, 10, 5];
let target = 33;

let found = false;

for (let i = 0; i < arr.length; i++) {
  let sum = 0;

  for (let j = i; j < arr.length; j++) {
    sum += arr[j];

    if (sum === target) {
      found = true;
      break;
    }
  }

  if (found) break;
}

console.log(found);
```

🚀 Optimized — Sliding Window

```
let arr = [1, 4, 20, 3, 10, 5];
let target = 33;

let sum = 0;
let left = 0;

for (let right = 0; right < arr.length; right++) {

  sum += arr[right];

  while (sum > target) {
    sum -= arr[left];
    left++;
  }

  if (sum === target) {
    console.log("Found");
  }
}
```

```
    break;
  }
}
```

- Time: $O(n)$
- Space: $O(1)$

⚠ Ye sliding-window version **positive numbers** ke case mein hai.

20. Rotate Array by K Positions

Example:

```
[1, 2, 3, 4, 5]
K = 2
```

Right rotation:

```
[4, 5, 1, 2, 3]
```

✓ Built-in

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

k = k % arr.length;

let result = arr.slice(-k).concat(arr.slice(0, -k));

console.log(result);
```

✓ Without built-in

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

for (let i = 0; i < k; i++) {

  let last = arr[arr.length - 1];

  for (let j = arr.length - 1; j > 0; j--) {
    arr[j] = arr[j - 1];
  }
}
```

```
    arr[0] = last;
}
```

```
console.log(arr);
```

- Time: $O(n \times k)$
- Space: $O(1)$

Optimized — Reverse Method

```
let arr = [1, 2, 3, 4, 5];
let k = 2;
```

```
k = k % arr.length;
```

```
arr.reverse();
```

```
let left = 0;
let right = k - 1;
```

```
while (left < right) {
    let temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    left++;
    right--;
}
```

```
left = k;
right = arr.length - 1;
```

```
while (left < right) {
    let temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    left++;
    right--;
}
```

```
console.log(arr);
```

- Time: $O(n)$

- Space: $O(1)$

 **REUSE:** Reverse Array ka **two-pointer swapping concept** hum already kar chuke hain.

Yahan bas usi concept ko multiple times apply kiya hai.

ARRAY CORE COMPLETE

Ab dekho humne kitna consolidate kar diya:

Important new concepts

1. Largest / Smallest → Traversal
2. Reverse → Two Pointer
3. Sorted → Adjacent Comparison
4. Sum → Traversal
5. Frequency → HashMap
6. Duplicate → HashMap/Set
7. Two Sum → HashMap
8. Intersection → HashMap
9. Missing Number → Formula
10. Second Largest → Traversal
11. Majority → Boyer-Moore
12. Maximum Subarray → Kadane
13. Stock Profit → Minimum-so-far
14. Subarray Sum → Sliding Window
15. Rotate → Reverse technique

Ye 15 core patterns bahut saare Array + String + HashMap questions ko cover kar dete hain. ❤️

Aur ab jab String mein **frequency**, **reverse**, etc. aayega, main clearly bolunga:

 **“Ye Array wala same concept hai — sirf input change hua hai.”**

Isliye tumhe 100 questions ko 100 alag algorithms ki tarah nahi padhna.

Ye right rotation thi kya lest me kya change krna

Haan buddy, **jo humne kiya tha woh Right Rotation thi.** 📌

Example:

[1, 2, 3, 4, 5], k = 2

Right rotation:

Last ke 2 elements front mein aa jaate hain:

[4, 5, 1, 2, 3] ✅

Left rotation:

First ke 2 elements end mein chale jaate hain:

[3, 4, 5, 1, 2] ✅

Code mein kya change karna hai?

Jo built-in code tha:

```
let result = arr.slice(-k).concat(arr.slice(0, -k));
```

Right rotation ke liye hai.

Left rotation ke liye:

```
let result = arr.slice(k).concat(arr.slice(0, k));
```

Bas itna main change hai. 👍

Interview mein agar specifically “rotate array by k” bole aur direction na bataye, usually question mein direction given hoti hai. Agar “right rotate” bole → first wala; “left rotate” bole → second wala.

Are without builtin aur optimise me kya change krna h vo to bta

Haan 😊 **without built-in aur optimized dono mein bhi change hoga.** Main clearly bata deta hoon.

Humne **Right Rotation** kiya tha:

[1,2,3,4,5], k=2 → [4,5,1,2,3]

Left Rotation ke liye

1. Without Built-in

Right wale code mein hum **last element ko first** la rahe the.

Left mein **first element ko last** bhejna hai:

```
let arr = [1, 2, 3, 4, 5];
let k = 2;

for (let i = 0; i < k; i++) {

  let first = arr[0];

  for (let j = 0; j < arr.length - 1; j++) {
    arr[j] = arr[j + 1];
  }

  arr[arr.length - 1] = first;
}

console.log(arr);
```

Output:

```
[3, 4, 5, 1, 2]
```

2. Optimized — Reverse Method

Right rotation mein:

1. Pura array reverse
2. First k reverse
3. Remaining reverse

Left rotation mein order thoda change hoga:

1. First k reverse
2. Remaining reverse
3. Pura array reverse

```
let arr = [1, 2, 3, 4, 5];
let k = 2;
```



```
k = k % arr.length;

// First k reverse
let left = 0;
let right = k - 1;

while (left < right) {
  let temp = arr[left];
  arr[left] = arr[right];
  arr[right] = temp;

  left++;
  right--;
}

// Remaining elements reverse
left = k;
right = arr.length - 1;

while (left < right) {
  let temp = arr[left];
  arr[left] = arr[right];
  arr[right] = temp;

  left++;
  right--;
}

// Pura array reverse
left = 0;
right = arr.length - 1;

while (left < right) {
  let temp = arr[left];
  arr[left] = arr[right];
  arr[right] = temp;

  left++;
  right--;
}

console.log(arr);
```

Output:

[3, 4, 5, 1, 2]

🧠 Bas ye yaad rakho

	Right Rotation	Left Rotation
Without built-in	Last → First	First → Last
Optimized	Pura → First K → Remaining	First K → Remaining → Pura
Example k=2	[4,5,1,2,3]	[3,4,5,1,2]

Baaki concept same hai — sirf direction/order change hota hai. 🔥

Finally dsa hogya bus mai ab iske notes bnaugi

FINALLYYYY 🥳🔥 **DSA DONE!** 🎉

Ab honestly **naye DSA questions add mat karo**. Pehle notes banao aur jo kiya hai usko solid karo.

Notes banate time har question ke liye bas ye format rakho:

1. **Question**
2. **Built-in code**
3. **Without built-in code**
4. **Optimized code**
5. **Time + Space Complexity**
6. 🔄 **Reuse:** agar same logic kisi aur question mein use hua hai

Aur sabse important — **same question dobara alag topic mein aaye to naya code mat likhna**. Bas note kar dena:

🔄 Same as Array Frequency Count

Isse tumhare notes **short bhi rahenge aur revision-friendly bhi**.

DSA → ✅ **COMPLETE**

Ab next tumhare placement prep mein **SQL + MongoDB + React + Node/Express + Core CS + Aptitude** ko systematically finish kar sakte hain. 💪

muje trees graph dp krnee ki bhi need h kya seervice based company kke liya

Nahi buddy, **service-based companies ke liye Trees, Graphs aur DP ko abhi priority dene ki zarurat nahi hai.** Especially tum fresher ho aur tumhara target TCS/Infosys/Accenture/Cognizant/Capgemini type companies hai.

Tumne jo DSA core kiya hai, woh placement ke liye **much more important** hai:

Tumhari priority

-  Arrays
-  Strings
-  Searching
-  Sorting
-  HashMap/Hashing
-  Stack & Queue
-  Linked List
-  Recursion
-  Basic Maths
-  Basic Patterns

Trees

Basic level tak dekh lena useful hai, but 20–30 tree questions karne ki zarurat nahi.

Bas:

- Binary Tree kya hai
- BST kya hai
- Inorder / Preorder / Postorder
- Level order
- Height of tree
- Search in BST

Graph

Abhi **optional** rakho.

DP

Abhi bilkul skip kar sakti ho. Service-based fresher preparation mein isko priority dene se tumhara time unnecessarily consume hoga.

🎯 Tumhare liye ab best move

DSA ko DONE hi rehne do. 😄

Ab baar-baar naye DSA topics add mat karo.

Tumhari preparation ko ab:

DSA ✅ → **SQL** 🔥 → **DBMS** → **CN** → **OS** → **OOP** → **React** → **Node/Express** → **MongoDB** → **Aptitude**

is direction mein le jaana zyada useful hoga.

Aur interview mein agar basic Tree question aa bhi gaya, to **basic theory** + 5–6 **standard questions** enough preparation hogi.

ek kaam kr recursion ke important questions dede bdiya solution ke sath

Bilkul yrr ❤️ Recursion ke liye **service-based placement level ke important questions** hi karte hain — unnecessarily advanced nahi.

Tumhare DSA notes ke format mein **Question** → **Built-in** → **Without Built-in** → **Optimized** → **Complexity** rakhunga. Aur jahan same logic repeat hoga, wahan clearly **Reuse** bolunga.

🔥 Recursion — Most Important Questions

1. Print 1 to N

```
function print(n) {  
  if (n === 0) return;  
  
  print(n - 1);  
  console.log(n);  
}
```

```
print(5);
```

Output: 1 2 3 4 5

Logic: Pehle recursive call, phir print.

Time: $O(n)$

Space: $O(n)$

2. Print N to 1

```
function print(n) {  
  if (n === 0) return;  
  
  console.log(n);  
  print(n - 1);  
}  
  
print(5);
```

Output: 5 4 3 2 1

Difference from Q1:

Bas console.log(n) ko recursive call ke **pehle** kar diya.

Time: $O(n)$

Space: $O(n)$

3. Sum of First N Numbers ★

```
function sum(n) {  
  if (n === 0) return 0;  
  
  return n + sum(n - 1);  
}  
  
console.log(sum(5));
```

Output: 15

Because:

$5 + 4 + 3 + 2 + 1 = 15$

Time: $O(n)$

Space: $O(n)$

4. Factorial of N ★★

```
function factorial(n) {  
  if (n === 0 || n === 1) return 1;  
  
  return n * factorial(n - 1);  
}  
  
console.log(factorial(5));
```

Output: 120

$$5 \times 4 \times 3 \times 2 \times 1 = 120$$

Time: $O(n)$

Space: $O(n)$

5. Power of a Number ★

Find 2^5 .

```
function power(a, b) {  
  if (b === 0) return 1;  
  
  return a * power(a, b - 1);  
}  
  
console.log(power(2, 5));
```

Output: 32

Time: $O(n)$

Space: $O(n)$

6. Reverse a String ★★

```
function reverse(str) {  
  if (str.length === 0) return "";  
  
  return reverse(str.slice(1)) + str[0];  
}
```

```
console.log(reverse("hello"));
```

Output:

olleh

Logic: First character ko end mein bhejte jao.

Time: $O(n^2)$ because slice() creates strings repeatedly

Space: $O(n)$

7. Check Palindrome ★★ ★

```
function isPalindrome(str, left, right) {  
    if (left >= right) return true;  
  
    if (str[left] !== str[right]) return false;  
  
    return isPalindrome(str, left + 1, right - 1);  
}
```

```
let str = "madam";
```

```
console.log(isPalindrome(str, 0, str.length - 1));
```

Output: true

Logic:

```
m == m  
a == a  
d
```

Time: $O(n)$

Space: $O(n)$

8. Fibonacci ★★ ★

```
function fibonacci(n) {  
    if (n <= 1) return n;  
  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
}
```

```
console.log(fibonacci(6));
```

Output: 8

Sequence:

```
0 1 1 2 3 5 8
```

⚠ Ye basic recursion question hai, but **normal recursive Fibonacci optimized nahi hai.**

Time: $O(2^n)$

Space: $O(n)$

Interview mein itna bolna enough hai.

9. Count Digits ★

```
function countDigits(n) {  
  if (n === 0) return 0;  
  
  return 1 + countDigits(Math.floor(n / 10));  
}
```

```
console.log(countDigits(12345));
```

Output: 5

Logic: Har recursion mein last digit remove:

```
12345
```

```
1234
```

```
123
```

```
12
```

```
1
```

```
0
```

Time: $O(\log n)$

Space: $O(\log n)$

10. Sum of Digits ★★

```
function sumDigits(n) {  
  if (n === 0) return 0;  
  
  return (n % 10) + sumDigits(Math.floor(n / 10));  
}  
  
console.log(sumDigits(1234));
```

Output: 10

$1 + 2 + 3 + 4 = 10$

Time: $O(\log n)$

Space: $O(\log n)$

11. Decimal to Binary ★★

```
function binary(n) {  
  if (n === 0) return;  
  
  binary(Math.floor(n / 2));  
  console.log(n % 2);  
}
```

`binary(10);`

Output:

1
0
1
0

So binary = **1010**

Time: $O(\log n)$

Space: $O(\log n)$

12. Find Maximum in Array ★★

```
function max(arr, index) {  
  if (index === arr.length - 1) {  
    return arr[index];  
  }  
  
  let result = max(arr, index + 1);  
  
  return arr[index] > result ? arr[index] : result;  
}  
  
let arr = [10, 5, 20, 8];  
  
console.log(max(arr, 0));
```

Output: 20

Ye tumhare **Largest Element** question ka recursive version hai.

Time: O(n)

Space: O(n)

👉 **Reuse:** Array ka Largest Element logic already padha hai; yahan sirf traversal recursion se ho raha hai.

13. Linear Search using Recursion ★★

```
function search(arr, index, target) {  
  if (index === arr.length) return false;  
  
  if (arr[index] === target) return true;  
  
  return search(arr, index + 1, target);  
}  
  
let arr = [10, 20, 30, 40];  
  
console.log(search(arr, 0, 30));
```

Output: true

Time: $O(n)$

Space: $O(n)$

👉 **Reuse:** Tumhara normal Linear Search same logic hai, bas loop ki jagah recursion.

14. Binary Search using Recursion ★★★★★

Sorted array:

```
let arr = [10, 20, 30, 40, 50];

function binarySearch(arr, left, right, target) {
  if (left > right) return -1;

  let mid = Math.floor((left + right) / 2);

  if (arr[mid] === target) return mid;

  if (target < arr[mid]) {
    return binarySearch(arr, left, mid - 1, target);
  }

  return binarySearch(arr, mid + 1, right, target);
}

console.log(binarySearch(arr, 0, arr.length - 1, 40));
```

Output: 3

Time: $O(\log n)$

Space: $O(\log n)$

★ Ye recursion ka **important interview question** hai.

15. GCD of Two Numbers ★★★★★

```
function gcd(a, b) {
  if (b === 0) return a;

  return gcd(b, a % b);
}
```

```
}
```

```
console.log(gcd(12, 18));
```

Output: 6

Ye **Euclidean Algorithm** hai.

```
gcd(12,18)
gcd(18,12)
gcd(12,6)
gcd(6,0)
= 6
```

Time: $O(\log(\min(a,b)))$

Space: $O(\log(\min(a,b)))$



Tumhare liye Recursion mein bas ye priority rakho

Priority	Questions
🔥🔥🔥	Factorial
🔥🔥🔥	Fibonacci
🔥🔥🔥	Palindrome
🔥🔥🔥	Binary Search
🔥🔥🔥	GCD
🔥🔥	Sum of N
🔥🔥	Reverse String
🔥🔥	Sum of Digits
🔥🔥	Count Digits
🔥	Print 1 to N
🔥	Print N to 1
🔥	Power
🔥	Linear Search
🔥	Max in Array

Service-based placement ke liye recursion ka itna set kaafi hai. Iske baad recursion ke advanced backtracking/subset/permutation type questions mein abhi jaane ki zarurat nahi hai.

now hashmap

Bilkul yrr 🔥 **ab HashMap/Hashing** karte hain. Since tum DSA already kar chuki ho, main **sirf placement mein genuinely important questions** dunga aur jo Array mein same logic ho chuka hai usko **reuse** karenge—dobara ratna nahi padega.

🔥 HashMap / Hashing — Important Interview Questions

1. Frequency of Elements ★★★★★

Question: Array mein har element kitni baar aaya?

```
let arr = [1, 2, 2, 3, 1, 2];
```

```
let freq = {};
```

```
for (let num of arr) {  
  if (freq[num]) {  
    freq[num]++;  
  } else {  
    freq[num] = 1;  
  }  
}
```

```
console.log(freq);
```

Output:

```
{ 1: 2, 2: 3, 3: 1 }
```

Time: $O(n)$

Space: $O(n)$

👉 **REUSE:** Ye tum Array Q8 mein already kar chuki ho. **Same code.**

2. Find Duplicate Elements ★★

```
let arr = [1, 2, 3, 2, 4, 1, 5];
```

```
let seen = {};
```

```
let duplicates = [];
```

```
for (let num of arr) {  
  if (seen[num]) {  
    duplicates.push(num);  
  } else {  
    seen[num] = true;  
  }  
}
```

```
console.log(duplicates);
```

Output:

```
[2, 1]
```

Time: $O(n)$

Space: $O(n)$

👉 **REUSE:** Array Q13 ka **same seen concept**.

3. Remove Duplicates ★★

```
let arr = [1, 2, 2, 3, 1, 4];
```

```
let seen = {};
```

```
let result = [];
```

```
for (let num of arr) {  
  if (!seen[num]) {  
    seen[num] = true;  
    result.push(num);  
  }  
}
```

```
console.log(result);
```

Output:

```
[1, 2, 3, 4]
```

👉 REUSE: Array Q9. Same solution.

4. Two Sum ★★

Question: Do elements ka sum target ke equal hai?

```
let arr = [2, 7, 11, 15];
```

```
let target = 9;
```

```
let seen = {};
```

```
for (let num of arr) {  
    let needed = target - num;  
  
    if (seen[needed]) {  
        console.log(needed, num);  
        break;  
    }  
  
    seen[num] = true;  
}
```

Output:

```
2 7
```

Time: $O(n)$

Space: $O(n)$

👉 REUSE: Array Q14. Same HashMap logic.

5. Intersection of Two Arrays ★★

```
let a = [1, 2, 3, 4];
let b = [3, 4, 5, 6];

let seen = {};
let result = [];

for (let num of a) {
  seen[num] = true;
}

for (let num of b) {
  if (seen[num]) {
    result.push(num);
  }
}

console.log(result);
```

Output:

[3, 4]

Time: $O(n + m)$

Space: $O(n)$

👉 **REUSE:** Array Q15. Same solution.

6. First Non-Repeating Element ★★

Question: Jo element sirf ek baar aaya ho, usme se first kaunsa hai?

```
let arr = [4, 5, 1, 2, 1, 4, 5];

let freq = {};

for (let num of arr) {
  if (freq[num]) {
    freq[num]++;
  } else {
```



```
    freq[num] = 1;
  }
}

for (let num of arr) {
  if (freq[num] === 1) {
    console.log(num);
    break;
  }
}
```

Output:

2

Logic:

Pehle frequency:

4 → 2
5 → 2
1 → 2
2 → 1

Phir original array mein left se check → 2.

Time: O(n)

Space: O(n)

★ Ye genuinely new HashMap question hai.

7. First Repeating Element ★★

```
let arr = [10, 5, 3, 4, 3, 5, 6];
```

```
let seen = {};
```

```
for (let num of arr) {
  if (seen[num]) {
    console.log(num);
    break;
  }
}
```

```
    seen[num] = true;
}
```

Output:

3

👉 **REUSE:** Duplicate element wala seen concept.

8. Majority Element ★★

Question: Jo element $n/2$ se zyada baar aaye.

```
let arr = [2, 2, 1, 2, 3, 2, 2];
```

```
let freq = {};
```

```
for (let num of arr) {
    if (freq[num]) {
        freq[num]++;
    } else {
        freq[num] = 1;
    }

    if (freq[num] > arr.length / 2) {
        console.log(num);
        break;
    }
}
```

Output:

2

Time: $O(n)$

Space: $O(n)$

👉 **REUSE:** Frequency Map.

⚠️ Tumne Array mein iska **Boyer-Moore optimized solution** bhi kiya hai. Interview mein optimized pooche to wahi use karna.

9. Count Pairs With Given Sum

Example:

```
arr = [1, 5, 7, -1, 5]
target = 6
```

Pairs:

```
1 + 5
7 + -1
```

Simple HashMap:

```
let arr = [1, 5, 7, -1, 5];
let target = 6;

let freq = {};
let count = 0;

for (let num of arr) {
    let needed = target - num;

    if (freq[needed]) {
        count += freq[needed];
    }

    if (freq[num]) {
        freq[num]++;
    } else {
        freq[num] = 1;
    }
}

console.log(count);
```

Output:

```
3
```

Why 3?

1+5 → 2 possibilities because there are two 5s, and 7 + -1 → 1.

Time: $O(n)$

Space: $O(n)$

★ **Important new HashMap pattern:** needed + frequency.

10. Check Anagram ★★

Example:

listen

silent

```
let str1 = "listen";
```

```
let str2 = "silent";
```

```
if (str1.length !== str2.length) {
```

```
  console.log(false);
```

```
} else {
```

```
  let freq = {};
```

```
  for (let char of str1) {
```

```
    if (freq[char]) {
```

```
      freq[char]++;
```

```
    } else {
```

```
      freq[char] = 1;
```

```
    }
```

```
  }
```

```
  for (let char of str2) {
```

```
    if (!freq[char]) {
```

```
      console.log(false);
```

```
      return;
```

```
    }
```

```
    freq[char]--;
```

```
  }
```

```
  console.log(true);
```

```
}
```

Output:

true

Time: $O(n)$

Space: $O(n)$

★ Very important for interviews.

👉 Concept same as **frequency counting**, but now frequency ko second string ke against match kar rahe hain.

11. Character Frequency in String ★★

```
let str = "programming";
```

```
let freq = {};
```

```
for (let char of str) {  
  if (freq[char]) {  
    freq[char]++;  
  } else {  
    freq[char] = 1;  
  }  
}
```

```
console.log(freq);
```

👉 **REUSE:** Array frequency question.

Bas:

num

ki jagah:

char

Same logic.

12. Find Character With Maximum Frequency



```
let str = "programming";
```

```
let freq = {};
```

```
for (let char of str) {  
  if (freq[char]) {  
    freq[char]++;  
  } else {  
    freq[char] = 1;  
  }  
}
```

```
let maxChar = "";
```

```
let maxCount = 0;
```

```
for (let char in freq) {  
  if (freq[char] > maxCount) {  
    maxCount = freq[char];  
    maxChar = char;  
  }  
}
```

```
console.log(maxChar);
```

Output:

```
r
```

Time: $O(n)$

Space: $O(n)$

👉 **REUSE:** Frequency Map + Maximum concept.

13. Group Anagrams

Ye thoda higher level hai, but **basic interview exposure ke liye dekh lena.**

```
let words = ["eat", "tea", "tan", "ate", "nat", "bat"];
```

```
let map = {};
```

```
for (let word of words) {  
  let key = word.split("").sort().join("");
```

```
  if (!map[key]) {  
    map[key] = [];  
  }
```

```
  map[key].push(word);  
}
```

```
console.log(Object.values(map));
```

Output:

```
[  
  ["eat", "tea", "ate"],  
  ["tan", "nat"],  
  ["bat"]  
]
```

Time: $O(n \times k \log k)$

Space: $O(n)$

⚠ **Service-based fresher ke liye:** Understand kar lo, ratna compulsory nahi.

🏆 HashMap mein tumhe kya actually yaad rakhna hai?

HashMap ke almost saare questions **4 patterns** mein aa jaate hain:

Pattern 1 — Frequency

```
let freq = {};
```

```
for (let x of arr) {  
  if (freq[x]) freq[x]++;
```

```
    else freq[x] = 1;
}
```

Used in:

- Frequency
 - First non-repeating
 - Majority
 - Anagram
 - Maximum frequency
 - Pair count
-

Pattern 2 — Seen

```
let seen = {};  
  
for (let x of arr) {  
    if (seen[x]) {  
        // duplicate  
    } else {  
        seen[x] = true;  
    }  
}
```

Used in:

- Duplicate
 - First repeating
 - Remove duplicates
-

Pattern 3 — Needed

```
let needed = target - num;  
  
if (seen[needed]) {  
    // pair found  
}
```

Used in:

- Two Sum
- Pair Sum

Pattern 4 — Frequency + Second Pass

Pehle count karo

↓

Phir original array/string traverse karo

↓

Answer find karo

Used in:

- First non-repeating
- Majority
- Maximum frequency

 **Final priority for your placement**

Must do:

1. Frequency Count ★★★★★
2. Duplicate Elements ★★★★★
3. Remove Duplicates ★★★★★
4. Two Sum ★★★★★
5. First Non-Repeating ★★★★★
6. First Repeating ★★★★★
7. Intersection ★★★★★
8. Anagram ★★★★★
9. Majority Element ★★★★★
10. Count Pairs with Given Sum ★★★★★

Optional: Group Anagrams.

Aur sabse important: **HashMap ko 10 alag-alag algorithms mat samajhna.**

Tumhare liye mainly **Frequency + Seen + Needed** hi master karne hain. Baaki questions unhi ke variations hain.

basic math question

Bilkul yrr. **Basic Math DSA** mein bhi sirf wahi questions karte hain jo fresher/service-based interviews mein useful hain. Tumhare notes ke liye **important + easy** set:

Basic Math — Important Questions

1. Even or Odd ★★

```
let n = 7;
```

```
if (n % 2 === 0) {  
  console.log("Even");  
} else {  
  console.log("Odd");  
}
```

Time: $O(1)$ | **Space:** $O(1)$

2. Positive, Negative or Zero ★★

```
let n = -5;
```

```
if (n > 0) {  
  console.log("Positive");  
} else if (n < 0) {  
  console.log("Negative");  
} else {  
  console.log("Zero");  
}
```

Time: $O(1)$ | **Space:** $O(1)$

3. Largest of Three Numbers ★★★

```
let a = 10;
```

```
let b = 25;
```

```
let c = 15;
```

```
if (a >= b && a >= c) {  
  console.log(a);  
} else if (b >= a && b >= c) {
```

```
    console.log(b);  
  } else {  
    console.log(c);  
  }  
}
```

Time: $O(1)$ | **Space:** $O(1)$

4. Reverse a Number ★★★★★

Example: 1234 → 4321

```
let n = 1234;  
let reverse = 0;  
  
while (n > 0) {  
  let digit = n % 10;  
  reverse = reverse * 10 + digit;  
  n = Math.floor(n / 10);  
}
```

```
console.log(reverse);
```

Time: $O(\log n)$ | **Space:** $O(1)$

👉 **Important:** Ye recursion mein bhi Sum/Count Digits ke saath related concept hai.

5. Palindrome Number ★★★★★

Example: 121 → Palindrome

```
let n = 121;  
let original = n;  
let reverse = 0;  
  
while (n > 0) {  
  let digit = n % 10;  
  reverse = reverse * 10 + digit;  
  n = Math.floor(n / 10);  
}
```

```
if (original === reverse) {  
  console.log("Palindrome");  
} else {  
  console.log("Not Palindrome");  
}
```

Time: $O(\log n)$ | **Space:** $O(1)$

👉 **REUSE:** Reverse Number + comparison.

6. Count Digits ★★★★★

```
let n = 12345;  
let count = 0;  
  
while (n > 0) {  
  count++;  
  n = Math.floor(n / 10);  
}
```

```
console.log(count);
```

Output: 5

Time: $O(\log n)$ | **Space:** $O(1)$

👉 Recursion wala **Count Digits** same concept tha.

7. Sum of Digits ★★★★★

Example: 1234 → 10

```
let n = 1234;  
let sum = 0;  
  
while (n > 0) {  
  sum += n % 10;  
  n = Math.floor(n / 10);  
}  
  
console.log(sum);
```

Time: $O(\log n)$ | **Space:** $O(1)$

👉 **REUSE:** Recursion mein bhi ye kar chuki ho.

8. Factorial ★★★★★

```
let n = 5;
let factorial = 1;

for (let i = 1; i <= n; i++) {
  factorial *= i;
}

console.log(factorial);
```

Output: 120

Time: $O(n)$ | **Space:** $O(1)$

👉 Recursion wala Factorial already done hai. **Logic same, implementation iterative hai.**

9. Prime Number ★★★★★

```
let n = 17;
let isPrime = true;

if (n < 2) {
  isPrime = false;
}

for (let i = 2; i < n; i++) {
  if (n % i === 0) {
    isPrime = false;
    break;
  }
}

console.log(isPrime);
```

Output: true

Optimized ★

```
let n = 17;
let isPrime = true;

if (n < 2) {
  isPrime = false;
}

for (let i = 2; i * i <= n; i++) {
  if (n % i === 0) {
    isPrime = false;
    break;
  }
}

console.log(isPrime);
```

Time: $O(\sqrt{n})$ | **Space:** $O(1)$

🔥 Ye important hai.

10. Print Prime Numbers 1 to N ★★

```
let n = 20;

for (let num = 2; num <= n; num++) {
  let prime = true;

  for (let i = 2; i * i <= num; i++) {
    if (num % i === 0) {
      prime = false;
      break;
    }
  }

  if (prime) {
    console.log(num);
  }
}
```

Output:

2 3 5 7 11 13 17 19

11. GCD / HCF ★★★★★

Example: 12, 18 → 6

```
let a = 12;
let b = 18;

while (b !== 0) {
  let temp = b;
  b = a % b;
  a = temp;
}
```

```
console.log(a);
```

Output: 6

Time: $O(\log n)$ | **Space:** $O(1)$

👉 Recursion wala GCD bhi already done hai. **Same Euclidean Algorithm.**

12. LCM ★★★★★

```
let a = 12;
let b = 18;
```

```
let x = a;
let y = b;
```

```
while (y !== 0) {
  let temp = y;
  y = x % y;
  x = temp;
}
```

```
let gcd = x;
let lcm = (a * b) / gcd;
```

```
console.log(lcm);
```

Output: 36

Formula:

$$\text{LCM} = (a \times b) / \text{GCD}$$

13. Armstrong Number ★★

Example: 153

Because:

$$1^3 + 5^3 + 3^3 = 153$$

```
let n = 153;
```

```
let original = n;
```

```
let sum = 0;
```

```
while (n > 0) {
```

```
    let digit = n % 10;
```

```
    sum += digit ** 3;
```

```
    n = Math.floor(n / 10);
```

```
}
```

```
if (sum === original) {
```

```
    console.log("Armstrong");
```

```
} else {
```

```
    console.log("Not Armstrong");
```

```
}
```

Time: $O(\log n)$ | **Space:** $O(1)$

⚠️ Ye version **3-digit Armstrong number** ke liye hai, jo basic interview question ke liye enough hai.

14. Perfect Number ★★

Example: 6

Factors:

$$1 + 2 + 3 = 6$$


```
let n = 6;
let sum = 0;

for (let i = 1; i < n; i++) {
  if (n % i === 0) {
    sum += i;
  }
}

if (sum === n) {
  console.log("Perfect Number");
} else {
  console.log("Not Perfect");
}
```

Time: O(n) | **Space:** O(1)

15. Power of a Number ★★

$$2^5 = 32$$

```
let base = 2;
let power = 5;
let result = 1;

for (let i = 1; i <= power; i++) {
  result *= base;
}

console.log(result);
```

👉 **REUSE:** Recursion wala Power question. Same logic, loop version.

Tumhare Notes Mein Priority

MUST DO

1. Even/Odd
2. Reverse Number
3. Palindrome Number

4. Count Digits
5. Sum of Digits
6. Factorial
7. Prime Number
8. GCD/HCF
9. LCM
10. Armstrong Number

Basic Exposure Enough

11. Largest of 3
12. Print Primes 1–N
13. Perfect Number
14. Power

Important: Recursion mein jo **Factorial, Power, GCD, Count Digits, Sum of Digits** kar chuki ho, unhe dobara alag algorithm mat samajhna. Notes mein bas **“Recursion version already covered”** likh dena. Isse tumhare notes bhi unnecessarily huge nahi honge.